Compression and Verification

Layered Emergence, Structural Convergence, and Verifiable Execution in Complex Software Systems

The Compression–Assurance Working Notes

An independent technical report

https://github.com/ghwinter/assay

Correspondence via the repository issue tracker.
Source of this report: https://github.com/ghwinter/assay/tree/master/docs/paper

2026

Abstract

We formalise one question. *Do large software systems converge in their high-level abstractions, structures and architectures, in the way that low-level ABIs, calling conventions and stack frames converge?* The intuition is decomposed into three propositions of differing strength, each given a decidable or falsifiable form.

Four theses organise the work. The **compression thesis**: a complex system is comprehensible to a bounded intelligence exactly to the extent that it is compressible; comprehension is task-constrained lossy compression whose optimal descent depth is the stationarity condition of a rate–distortion Lagrangian. The **necessity thesis**: any system with concurrently live entities and contended finite resources, whose operational semantics is well defined, has already defined six functions—identity and lifecycle, allocation and reclamation, scheduling, naming and addressing, communication and synchronisation, and fault isolation and recovery; these may be built or adopted, but not omitted. The **separation thesis**: generation may be free, probabilistic and untrusted, while admission must be deterministic, mechanical and exhaustively inspectable; formal-layer guarantees come from a minimal trusted kernel, intent-layer guarantees from adversarial consensus, and the two are stitched by artefact binding. The **design thesis**: architecture is manufactured separation of scales—nature obtains levels free through renormalisation, software must pay for them—and design is forward compression while comprehension is inverse compression, two reading directions of one structure.

The formal results are as follows. A level's *oblivious set* as the kernel of its abstraction map, making "what this level may ignore" computable and reducing the reality of a level to a compression ratio; Galois connections for inter-level semantic preservation; a *contract operad* whose well-definedness reduces the composability of local guarantees to well-typed colouring and acyclicity of the interface graph, thereby explaining why feedback and retry invalidate existing composition arguments; a *level family* distinguishing cross-domain isomorphism of the six functions from transport along problem shape; near-decomposability as interface sufficiency, yielding a diagnostic for *leak pairs*; a *coarse-graining criterion* making the convergence thesis falsifiable within a single project; an amortisation inequality $\Theta(nm) \to \mathcal{O}(n+m)$ giving the phase-transition threshold of internal infrastructuralisation; expected read-set as a measurable architectural objective, with invariant centralisation proved to minimise it; an *observability rank* criterion for whether comprehension can converge at all, separating insufficient observation from insufficient sampling; *governable complexity* $C = \mathbb{E}[\rho] \cdot \lambda$ against comprehension capacity $K = hb$, yielding a stability condition and a proof that architecture fixes the scaling exponent of required headcount; technical debt as a description-length gap with a rate–distortion reading of refactoring; a Hoeffding bound $\exp(-2n(1/2-p)^2)$ for consensus reliability, dominated by a correlated error mode q that is in turn reduced to an incentive-design problem; and a consolidated assurance condition combining two-layer trust with artefact binding.

Twelve falsifiable hypotheses with operationalisations, controls, falsifiers and risks are given, together with a six-phase roadmap, a paper-centred interdisciplinary literature map, and an explicit list of non-claims rejecting the use of Gödel incompleteness, Rice's theorem, the

open status of P versus NP, near-decomposability, and analogies to the Langlands programme or Landauer's principle, as load-bearing universal laws.

Keywords: software architecture; structural convergence; abstraction levels; information compression; minimum description length; formal verification; proof-carrying code; semantic preservation; trusted computing base; consensus; intent gap

Contents

Abstract **i**

1 Introduction **1**
1.1 The questions . . . 1
1.2 Three gaps in existing treatments . . . 2
1.3 Core theses . . . 2
1.4 Epistemic discipline . . . 3
1.5 Contributions . . . 3
1.6 Structure . . . 4
1.7 Terminology . . . 5

2 Formal framework **6**
2.1 Levels and abstraction maps . . . 6
2.2 Oblivious sets and compression . . . 7
2.3 Architecture as a typed relational graph . . . 8
2.4 Comprehension as task-constrained lossy compression . . . 9
2.5 Contracts, traces and assurance . . . 10
2.6 Two modes of trust generation . . . 11
2.7 Contract operads and composition . . . 12
2.8 Notation . . . 13

3 Structural Convergence **15**
3.1 Three Convergence Claims of Different Strength . . . 15
3.2 Constraint Systems and Solution-Set Geometry . . . 16
3.3 Mechanisms of Convergence . . . 17
3.4 Convergence Strength and Domain Binding . . . 18
3.5 Relations Rather Than Components . . . 18
3.6 Verification Design . . . 19
3.7 Counterexamples and Limits . . . 20

4 Internal Infrastructuralisation **21**
4.1 The Phenomenon and the Terms . . . 21
4.2 Participant–Resource Structure . . . 22
4.3 The Integration Cost Theorem and the Phase-Transition Threshold . . . 22
4.4 The Six Functions . . . 23
4.5 Build or Adopt . . . 25
4.6 Cross-Domain Comparison . . . 25
4.7 Transport and Re-Derivation . . . 26
4.8 The Coarse-Graining Criterion . . . 27
4.9 Virtualisation and Deep Modules . . . 28

4.10 Functional Differentiation in Agent Systems . . . 28

5 Comprehension: Inverse Compression 30
5.1 Failure of Sequential Reading . . . 30
5.2 Cognitive Maps . . . 31
5.3 Operational Criteria for Comprehension . . . 31
5.4 Reading Order . . . 32
5.5 Detection Criteria for Module Boundaries . . . 33
5.6 Active Identification and Passive Reading . . . 33
5.7 The Observability Criterion . . . 34
5.8 Budget and Stopping Conditions . . . 35
5.9 Implications for Machine Comprehension . . . 36

6 Design: forward compression 38
6.1 Read-sets and the objective function . . . 38
6.2 Near-decomposability . . . 39
6.3 Technical debt . . . 40
6.4 Governable complexity and comprehension capacity . . . 41
6.5 The design procedure . . . 42
6.6 Cross-cutting concerns and the limits of layering . . . 42
6.7 Design checklist . . . 43

7 From Intent to Verifiable Execution 45
7.1 Dimensions of Correctness . . . 45
7.2 The Verifiable Refinement Chain . . . 46
7.3 Transition Edges and Evidence . . . 46
7.4 Composition Conditions and Failure Modes . . . 48
7.5 The Intent Gap . . . 49
7.6 The Asymmetry Between Verification and Generation . . . 50
7.7 Grades of Evidence . . . 51
7.8 Side Effects and Constraint Mechanisms . . . 52
7.9 Runtime Assurance . . . 53
7.10 Artefact Binding . . . 53
7.11 The Reliability Bound of the Consensus Layer . . . 54
7.12 The Incentive Structure of the Intent Layer . . . 55
7.13 The Consolidated Assurance Condition . . . 55

8 Levels as Units of Verification-Obligation Decomposition 58
8.1 The Cognitive Scale . . . 58
8.2 The Level Contract . . . 59
8.3 Decomposition of Verification Obligations . . . 59
8.4 The Bearer of Responsibility . . . 60
8.5 Level-Language Mismatch . . . 61

9 The Falsifiable Research Programme 63
9.1 Methodology: From Interpretive Framework to Testable Hypotheses . . . 63
9.2 Twelve Falsifiable Hypotheses . . . 63
9.3 Three Priority Research Directions . . . 66
9.4 Six-Stage Implementation Roadmap . . . 69
9.5 Success Criteria . . . 69

10 Literature Map and Positioning **71**
10.1 Six Groups of Literature . . . 71
10.2 The Five Layers Rearranged by Mathematical Function . . . 73
10.3 The Unoccupied Intersection . . . 73
10.4 Relation to Existing Theory . . . 74

11 Objections and Limits **76**
11.1 List of Non-Claims . . . 76
11.2 Arguments Removed from the Load-Bearing Structure . . . 77
11.3 Possible Objections and Responses . . . 79
11.4 Open Problems . . . 80
11.5 Methodological Limitations . . . 80

12 Conclusion **82**
12.1 Answers to the four questions . . . 82
12.2 Compression as the through-line . . . 83
12.3 The core proposition . . . 83
12.4 Closing remarks . . . 83

A Supplementary Proofs and Arguments **85**
A.1 Notes on Proposition 3.5 . . . 85
A.2 Complete Argument for Theorem 4.4 . . . 85
A.3 Supplement to the Item-by-Item Argument for Theorem 4.9 . . . 86
A.4 The Read-Set Lower Bound of Theorem 6.3 . . . 86
A.5 Exhaustiveness of the Three Grounds of Proposition 7.12 . . . 87
A.6 Tightness of the Bound in Theorem 7.27 . . . 87
A.7 A Stronger Form of Proposition 7.28 . . . 87
A.8 Notes on Proposition 5.8 . . . 88

B Templates **89**
B.1 Architecture Map Template . . . 89
B.2 Module Contract Template . . . 89
B.3 Decision Record Template . . . 90

C Glossary **91**

List of Tables

2.1 The two modes of trust generation 12
2.2 Notation used throughout. 13

4.1 Independent implementations of the same set of management duties in different domains 26
4.2 Agent systems rediscovering the six functions 29

5.1 The minimum field set of a subsystem cognitive record 36

6.1 Design checklist for large systems 43

7.1 The five dimensions of correctness and their mechanical checkability 45
7.2 Five transition edges, typical evidence and critical gaps 47
7.3 Grades of evidence 51
7.4 Artefact evidence list 52
7.5 Classification of side effects and the corresponding mechanical mechanisms . . . 52

8.1 Instantiation of the level-contract quadruple on a relational database engine . . 60
8.2 Distribution of assurance types across levels (example) 61

9.1 Summary of hypotheses and experimental designs 66
9.2 Research implementation roadmap 69

10.1 The five layers rearranged by mathematical function 73
10.2 Relation of this paper to existing theory 74

11.1 Arguments removed from the load-bearing structure 77

B.1 Architecture map 89

Chapter 1

Introduction

1.1 The questions

The following three questions share one structure. They are stated here in the form in which they can be discussed, proved and falsified.

Problem 1.1 (Generality of low-level determinism)**.** Software at the execution level is strikingly uniform: ABIs and calling conventions, the call stack, heap and object lifetime, control and data flow, synchronisation primitives and thread scheduling. That uniformity is imposed by a shared computational model, hardware architecture, language runtime and operating-system interface. Does the same determinism extend upwards? Do most large software systems, normally biased and normally engineered, converge on the same or similar abstractions, structures and architectures?

Problem 1.2 (Coercion by scale)**.** When a system crosses some order of magnitude, does it necessarily begin to design and optimise for recurrent problems of one kind, and thereby grow a unified internal infrastructure? An operating-system kernel defines how memory is managed and allocated, how threads and processes are defined and scheduled, and what runtime interface is offered upwards. A browser, a game engine, an asynchronous runtime, a distributed store and a relational engine address the same class of questions at different levels. Once the code base is large enough, do all of them exhibit the same kind of reasoning—unified function, unified interfaces, managed planning, runtime structure?

Problem 1.3 (Comprehension and design)**.** Software is written by humans and machines, but it is not easy to understand. A large system can hardly be read sequentially from the micro level. At what level of abstraction should it be approached, and how does comprehension lead into development without dissolving into a sea of code? The question bears equally on machines: an intelligence cannot descend into unbounded low-level information, and must retain an understanding of first principles—call this L_0—together with the levels above it, L_1, L_2, L_3, and so on. What an intelligence needs is compressed information. Conversely, how should a large complex system be designed so that its purpose is achieved?

The third question has a contemporary variant that moves it from comprehension to warranty:

Problem 1.4 (Correctness without review)**.** If machine generation drives the cost of code towards zero and no human reads the implementation line by line, how is correctness guaranteed *mechanically* and *non-self-referentially*? How is it guaranteed that the stated purpose is achieved, and that no unauthorised side effect accompanies it? A concrete instance: a hand-written call takes 1 ns; a generated implementation takes 1 μs. Functionally identical, purpose achieved, and a performance side effect introduced.

1.2 Three gaps in existing treatments

Each question has local counterparts in the literature: architectural convergence in pattern languages and complex-systems theory, coercion by scale in information hiding and near-decomposability, comprehension and design in cognitive models and multi-view modelling, correctness in program logic, refinement calculi and verified compilation. Treating them as a whole exposes three structural gaps.

The strength of the convergence claim is not graded. "Large software architectures converge" contains at least three propositions of very different strength: recurrent local architectural patterns (well supported by engineering experience); measurable structural similarity above a random baseline after abstraction (plausible and worth systematic testing); and convergence of nearly all large systems on one high-level architecture (unsupported, and probably too strong). Conflating them makes the claim untestable.

Necessity claims lack a load-bearing structure. "Scale forces a unified infrastructure" is a necessity claim, but it is usually stated as an empirical observation, and therefore cannot answer where the necessity comes from, what fixes the threshold, or why some equally large systems grow no kernel. The answers given below are that the necessity comes from the completeness of the operational semantics, the threshold from an amortisation inequality, and the apparent exceptions from systems that *adopt* a kernel rather than build one.

Comprehension, design and assurance are three readings of one structure. Comprehension is treated as a cognitive problem, design as an engineering problem and assurance as a logical problem, in three separate literatures. They are in fact different reading directions of one object: a level is the unit into which assurance obligations decompose; comprehension recovers that decomposition and design prescribes it.

1.3 Core theses

Four theses organise the paper. Each is annotated with its formal location and its strength: **[P]** proved, **[A]** argued under stated assumptions, **[H]** advanced as a falsifiable hypothesis.

Principle 1.5 (Compression)**.** **[A]** A complex system is comprehensible to a bounded intelligence if and only if it is compressible. Comprehension is task-constrained lossy compression: description length is minimised subject to preserved task-relevant predictive power. The optimal descent depth is therefore the stationarity condition of a rate–distortion Lagrangian, that is, the point at which marginal predictive gain vanishes.

Principle 1.6 (Necessity)**.** **[A]** Any system with concurrently live entities and contended finite resources, whose operational semantics is well defined, has already defined six functions: identity and lifecycle, allocation and reclamation, scheduling, naming and addressing, communication and synchronisation, and fault isolation and recovery. These may be built or adopted from a host—an operating system, a language runtime, a framework—but they cannot be omitted from the semantics.

Principle 1.7 (Separation)**.** **[A]** Generation may be free, probabilistic, open and untrusted; admission must be deterministic, mechanical and exhaustively inspectable. Guarantees on the formal layer (specification → implementation → binary → execution) come from mechanical decision by a minimal trusted kernel. Guarantees on the intent layer (intent → specification) come from adversarial consensus, and that layer admits no mechanical solution. The two are

stitched by artefact binding: only the verified binary is admitted to run, and the environmental assumptions are checked or monitored on it.

Principle 1.8 (Design)**. [A]** Architecture is manufactured separation of scales. Nature obtains levels free from scale separation and renormalisation; software levels do not emerge spontaneously and must be paid for in interfaces, encapsulation and invariants, and they leak. Design is therefore forward compression and comprehension is inverse compression, two reading directions of one structure. The measurable objective of architecture is the expected read-set $\mathbb{E}[\rho]$ of a typical change.

The four theses and their formal results are referred to collectively as the **compression–assurance framework**. The name is taken from the two ends of the claim: *compression* is the mechanism of comprehension and design, *assurance* is the mode in which trust is anchored. The central assertion is that a complex system is comprehensible to a bounded agent if and only if it is compressible, and that it is compressible if and only if it admits a separation of scales in which every level has an explicitly declared oblivious set.

1.4 Epistemic discipline

The material crosses engineering intuition, complex-systems theory, information theory, mathematical logic and social epistemology, and the risk of overreach is high. Five rules are observed throughout.

(i) **Strength labelling.** Every claim is labelled as a theorem, proposition, conjecture, hypothesis or heuristic. Heuristics are not written as theorems.

(ii) **Non-claims list.** Chapter 11 states explicitly what the paper does *not* claim, including several "authoritative theorems" that are declined.

(iii) **Explicit assumptions.** Any conclusion that depends on an environmental assumption states it, and any guarantee that has a boundary marks it.

(iv) **Falsifiability.** Chapter 9 reduces the core claims to twelve falsifiable hypotheses with operationalisations, controls, falsifiers and risks.

(v) **Verification not isomorphic to the conclusion.** Where the paper argues that generation must be untrusted and verification independent, that rule applies to the paper itself: its load-bearing propositions must be independently checkable and must not depend on any unproved conjecture.

Rule (v) constrains the paper's own rhetoric. Several claims have been removed from the load-bearing structure: Gödel incompleteness and Rice's theorem are not used to prove that a gap between intent and specification is principled (Chapter 7, §7.5); the open status of P versus NP is not used to support "verification is cheaper than generation" (§7.6); and near-decomposability is not treated as a universal law (Chapters 3 and 11). Each of these gains strength by being unloaded rather than reinforced.

1.5 Contributions

Formal contributions:

(C1) The *oblivious set* of a level as the kernel of its abstraction map, which makes "what this level need not care about" a computable object (Definition 2.7).

(C2) Galois connections as the condition for inter-level semantic preservation (Definitions 2.2, 2.3, Proposition 2.4).

(C3) A *contract operad* whose well-definedness reduces the composability of local guarantees to well-typed colouring and acyclicity of the interface graph (Definition 2.23, Proposition

2.25).

(C4) A *level family* $\pi : \mathcal{L} \to \mathcal{P}$ that separates cross-domain isomorphism of the six functions from transport along problem shape (Definition 4.14, Proposition 4.15).

(C5) A *coarse-graining criterion* under which the convergence thesis can be falsified inside a single project (Definition 4.18, Conjecture 4.19).

(C6) An amortisation inequality $\Theta(nm) \to \mathcal{O}(n+m)$ giving the phase-transition threshold of internal infrastructuralisation (Theorem 4.4).

(C7) Conditional necessity of the six functions from the well-definedness of the operational semantics (Theorem 4.9), and the resulting build-or-adopt criterion (Corollary 4.11).

(C8) A criterion for over- and under-constraint that renders "unique below, manifold above" as solution-set geometry (Proposition 3.5).

(C9) Expected read-set as the measurable objective of architecture, with a proof that invariant centralisation minimises it (Definition 6.1, Theorem 6.3).

(C10) Technical debt as a description-length gap (Definition 6.10), with a rate–distortion reading of refactoring.

(C11) An observability-rank criterion for whether comprehension can converge, separating insufficient observation from insufficient sampling (Definition 5.9, Proposition 5.10).

(C12) A governable-complexity stability condition and a scaling law for required headcount (Definition 6.13, Proposition 6.14).

(C13) A Hoeffding bound for consensus reliability and the proof that a correlated error mode dominates it, reducing the governing parameter to an incentive-design problem (Theorem 7.27, Proposition 7.28, §7.12).

(C14) A consolidated assurance condition combining two-layer trust with artefact binding (Definition 2.20, Theorem 7.34).

Methodological contributions:

(M1) A descent protocol that converts comprehension of a large system from sequential reading into layered modelling and active identification.

(M2) An executable list of seam detectors for locating real module boundaries in an unfamiliar code base.

(M3) A design procedure that replaces adjectival goals with an optimisable objective and a stability condition.

(M4) Twelve falsifiable hypotheses with experimental designs.

(M5) A cross-disciplinary research position that couples social epistemology to proof-carrying code, which the literature map identifies as unoccupied.

1.6 Structure

Chapter 2 establishes the formal language: levels, abstraction maps, oblivious sets, contracts, traces, assurance and trust. Chapter 3 addresses Problem 1.1, grading the convergence claim and stating its strength law. Chapter 4 addresses Problem 1.2, proving the conditional necessity of the six functions and deriving the phase-transition threshold. Chapter 5 addresses the first half of Problem 1.3: comprehension as inverse compression. Chapter 6 addresses the second half: design as forward compression. Chapter 7 addresses Problem 1.4 with a verifiable refinement chain and a two-layer trust structure. Chapter 8 promotes levels from a comprehension aid to the unit into which assurance obligations decompose. Chapter 9 states the falsifiable programme. Chapter 10 positions the work. Chapter 11 lists objections and limits. Chapter 12 concludes.

1.7 Terminology

Several terms drift between communities. They are fixed once here; the full glossary is Appendix C, and the Chinese rendering is fixed term by term in the companion terminology file.

Term	Meaning in this paper
level	A tuple (representation space, behaviour set, observation map, environment assumptions, cost model, invariants), defined relative to a study object and a task; a cognitive scale, not a physical directory.
oblivious set	The differences a level can never distinguish; the kernel of its abstraction map.
separation of scales	The condition under which adjacent levels each support a valid description and the lower level's implementation details do not appear in the upper level's semantics.
internal infrastructuralisation	The process by which a system promotes recurrent management problems to unified mechanisms on which the upper layers then depend.
the six functions	Identity and lifecycle; allocation and reclamation; scheduling; naming and addressing; communication and synchronisation; fault isolation and recovery.
read-set	The set of modules that must be read to complete a typical change.
assurance	A traceable evidential relation holding a property within stated premises and a stated scope.
mechanical assurance	Assurance by a single deterministic, exhaustively inspectable decision procedure.
consensus assurance	Assurance by multiple independent, adversarial participants.
artefact binding	The identity relation between the verified artefact and the executed artefact.
intent gap	The distortion between an intent and its formal projection.

One terminological caution is load-bearing. A *level* is a cognitive scale, not a physical tier. The same component occupies different levels in different tasks: an asynchronous runtime is infrastructure to an application developer and a complex system in its own right to its maintainer. Levels are defined relative to the study object and the task (Chapter 8).

Chapter 2

Formal framework

This chapter establishes the shared formal language. It does not invent new mathematics; it names objects that engineering practice already uses but does not define: what a level may ignore, under what conditions semantic preservation holds across levels, how far comprehension should descend, and within what scope a guarantee holds.

2.1 Levels and abstraction maps

Definition 2.1 (Level)**.** A **level** is a six-tuple

$$\Lambda \;=\; \big(X,\ \mathcal{B},\ \mathcal{O},\ \mathcal{A},\ \mathcal{C},\ \mathcal{I}\big),$$

where X is the **representation space** of states admissible at that level; $\mathcal{B} \subseteq X^{\mathbb{N}}$ is the **behaviour set** of admissible execution traces; $\mathcal{O}$ is the **observation map** projecting the level's behaviour onto the observable algebra of the level above; $\mathcal{A}$ is a set of **environmental assumptions** on which the level's conclusions depend; $\mathcal{C} : \mathcal{B} \to \mathbb{R}^{k}_{\geq 0}$ is a **cost model** with k components (time, space, energy, bandwidth); and $\mathcal{I} \subseteq \mathcal{B}$ is the **invariant set** that must hold under every admissible transition.

The six components are not of equal weight. X and $\mathcal{B}$ describe what exists, $\mathcal{O}$ describes what cannot be seen, $\mathcal{A}$ describes what is depended upon, $\mathcal{C}$ describes what it costs, and $\mathcal{I}$ describes what must not break. Chapter 8 argues that $\mathcal{O}$ and $\mathcal{I}$ decide whether a level is real: a level that cannot state its observation map and its invariants is not a level but a directory.

Definition 2.2 (Abstraction and concretisation)**.** Let Λ_i and Λ_{i+1} be adjacent levels, with the smaller index the upper level. The **abstraction map** $\mathcal{O}_i : \mathcal{B}_{i+1} \to \mathcal{B}_i$ projects lower-level behaviour onto upper-level behaviour; the **concretisation map** $\gamma_i : \mathcal{B}_i \to 2^{\mathcal{B}_{i+1}}$ expands an upper-level behaviour into the set of its lower-level implementations. If there are partial orders $\sqsubseteq_i$ and $\sqsubseteq_{i+1}$ such that

$$b \sqsubseteq_{i+1} \gamma_i(b') \quad\Longleftrightarrow\quad \mathcal{O}_i(b) \sqsubseteq_i b', \qquad \forall b \in \mathcal{B}_{i+1},\ b' \in \mathcal{B}_i, \tag{2.1}$$

then $(\mathcal{O}_i, \gamma_i)$ is a **Galois connection**, written $\mathcal{O}_i \dashv \gamma_i$.

A Galois connection states precisely what abstraction discards and what it retains, and is the central tool of the abstract-interpretation framework [20]. Where an inter-level relation can be expressed as a Galois connection, the semantic-preservation condition becomes mechanically decidable; where it cannot, one must fall back on simulation relations or on an explicit human argument.

Definition 2.3 (Inter-level soundness)**.** Let $A \in \mathcal{A}_i$ be an environmental assumption declared at the upper level, and let $\mathcal{B}_{i+1} \restriction A$ denote the lower-level behaviours admissible under A. Then Λ_i is **sound** with respect to Λ_{i+1} under A if

$$\mathcal{O}_i\big(\mathcal{B}_{i+1} \restriction A\big) \;\subseteq\; \mathcal{B}_i. \tag{2.2}$$

Proposition 2.4 (Composition of soundness)**.** *Let there be a chain $\Lambda_0, \Lambda_1, \ldots, \Lambda_n$, with increasing index denoting increasing concreteness, and suppose that for each i there exists $A_i \in \mathcal{A}_i$ satisfying (2.2). If*

(a) observation compatibility*: $\mathcal{O}_i \circ \mathcal{O}_{i+1} = \mathcal{O}_i^{(i+2)}$, where $\mathcal{O}_i^{(i+2)}$ is the direct observation from $\mathcal{B}_{i+2}$ to $\mathcal{B}_i$; and*

(b) assumption closure*: $\mathcal{O}_i(\mathcal{B}_{i+1} \restriction A_i) \subseteq \mathcal{B}_i \restriction A_{i-1}$, that is, the assumptions satisfied below suffice to discharge those above,*

then $\mathcal{O}_0^{(n)}\big(\mathcal{B}_n \restriction \bigcap_i A_i\big) \subseteq \mathcal{B}_0$.

Proof. Induction on n. The case $n = 1$ is (2.2). Condition (b) ensures that at each inductive step the upper level's environmental assumption holds of the upper-level behaviour set, so that (2.2) can be applied level by level. Condition (a) ensures that the composite observation agrees with the level-by-level observations, so that the conclusion holds at the top level. ∎

Remark 2.5 (Why composition fails). Proposition 2.4 is the precise form of "how do local guarantees compose into an end-to-end guarantee", and equally the precise form of why they often do not: composition requires both conditions simultaneously. In practice (a) fails when observation maps disagree—for instance when "the transaction committed" denotes different observations at the application layer and the storage layer—and (b) fails when environmental assumptions are not closed, for instance when the upper layer assumes that "a write is durable" while the lower layer's durability depends on an undeclared flush policy. Chapter 7 uses these two failure modes as the skeleton of its evidence-boundary table.

Remark 2.6 (Relation to the source model of the companion report). Definition 2.1 extends the four-tuple $M_i = (X_i, \mathrm{Beh}_i, \mathrm{Obs}_i, A_i)$ used in the companion research report by adding $\mathcal{C}$ and $\mathcal{I}$, which Chapters 6 and 7 require in order to place resource guarantees and invariant enforcement inside the level object. The companion report describes inter-level semantics by an *interpretation relation* $R_i \subseteq \mathrm{Beh}_{i+1} \times \mathrm{Beh}_i$, requiring that for every admissible target behaviour b_{i+1} of a successful transformation $x_{i+1} = T_i(x_i)$ there exists a source behaviour b_i with $R_i(b_{i+1}, b_i)$. That is a *simulation condition* and is more general than the set inclusion of (2.2); the two coincide when R_i is induced by a function, that is when $R_i = \{(b, \mathcal{O}_i(b))\}$. This paper uses R_i where a general relation is needed and $\mathcal{O}_i$ where the abstraction is functional. The companion report stresses that, across different languages and observation domains, R_i may be a forward simulation, a backward simulation, a bisimulation, a contextual equivalence or a refinement relation, and that one must not assume a single relation throughout. That qualification is accepted here.

2.2 Oblivious sets and compression

The next two definitions are the formal centre of the paper. They turn "what this level need not care about" from an engineering phrase into a computable object.

Definition 2.7 (Oblivious set)**.** The **oblivious set** of level Λ_i with respect to the level below is the kernel of the abstraction map:

$$\mathrm{Obl}(\Lambda_i) \;:=\; \ker \mathcal{O}_i \;=\; \big\{(b, b') \in \mathcal{B}_{i+1}^2 \;:\; \mathcal{O}_i(b) = \mathcal{O}_i(b')\big\}. \tag{2.3}$$

If $(b, b') \in \mathrm{Obl}(\Lambda_i)$, the level is **oblivious** to the difference between b and b'.

Proposition 2.8 (Basic properties)**.** $\mathrm{Obl}(\Lambda_i)$ *is an equivalence relation on* $\mathcal{B}_{i+1}$*, and the quotient satisfies*

$$\mathcal{B}_{i+1}/\,\mathrm{Obl}(\Lambda_i) \;\cong\; \mathcal{O}_i(\mathcal{B}_{i+1}).$$

Proof. The three properties of an equivalence relation follow from those of $=$. The surjection $\mathcal{O}_i : \mathcal{B}_{i+1} \to \mathcal{O}_i(\mathcal{B}_{i+1})$ induces a bijection $\mathcal{B}_{i+1}/\ker \mathcal{O}_i \to \mathcal{O}_i(\mathcal{B}_{i+1})$, by the universal property of the quotient. ■

Proposition 2.8 yields a direct consequence: *abstraction is quotienting by the kernel*. Everything a level exposes to the level above is the quotient of lower-level behaviour by that kernel. Hence:

- "what this level guarantees" is an element of the quotient $\mathcal{O}_i(\mathcal{B}_{i+1})$;
- "what this level does not guarantee" is what has been quotiented away, namely $\mathrm{Obl}(\Lambda_i)$;
- "what this level may ignore" is every lower-level detail compatible with $\mathrm{Obl}(\Lambda_i)$.

The third item has the deepest consequence: it makes ignorance a property that can be declared, checked and held to account. A module that has never declared its oblivious set has an incomplete contract.

Remark 2.9 (A criterion for pattern realism). Proposition 2.8 is isomorphic to an independent philosophical criterion. Dennett's criterion for a *real pattern* states that a pattern is real exactly when describing the system in terms of it is more economical than describing the raw data [63]. Under that criterion, whether a level exists is not an ontological question but a question of compression ratio—precisely the quantity measured by Definition 2.10.

Three consequences follow, and are used in later chapters.

(a) Levels are neither discovered natural kinds nor arbitrary conventions; they are fixed points of a compression operator.

(b) The convergence thesis of Chapter 3 needs no separate philosophical defence: if two systems compress independently under similar constraints, that their fixed points are similar follows from properties of the compression operator rather than from an additional assumption about the world.

(c) The criterion is quantitative, so whether a given level is real is measurable rather than arguable.

Definition 2.10 (Compression ratio)**.** Let μ be a reference measure on $\mathcal{B}_{i+1}$ and let H denote the corresponding entropy functional. The **compression ratio** of level Λ_i is the information annihilated:

$$\kappa_i \;=\; H_\mu\big(\mathcal{B}_{i+1}\big) \;-\; H_{\mathcal{O}_i \# \mu}\big(\mathcal{O}_i(\mathcal{B}_{i+1})\big), \tag{2.4}$$

that is, the information that cannot be recovered under $(\mathcal{O}_i, \gamma_i)$. The wording is borrowed from information theory [55] and algorithmic information theory [57].

Compression is not itself the objective: a system that guarantees nothing attains arbitrarily large κ. The objective is to maximise κ *subject to preserving task-relevant predictive power*, which is the subject of §2.4.

2.3 Architecture as a typed relational graph

Describing a system by levels alone is insufficient. Chapter 3 argues that cross-system similarity resides mainly in *relations* rather than in components, so the relations themselves must be typed.

Definition 2.11 (Architecture relational graph)**.** The **architecture relational graph** of a system is a typed directed multigraph

$$\mathcal{G} \;=\; \big(V,\; E_{\mathrm{abs}} \sqcup E_{\mathrm{dep}} \sqcup E_{\mathrm{cmp}} \sqcup E_{\mathrm{ctl}} \sqcup E_{\mathrm{ctr}}\big), \tag{2.5}$$

where V is a set of components and the five edge classes are:

(a) E_{abs}, **abstraction**: $u \to v$ when several details of v are expressed uniformly by u; annotated with which semantics of v are retained and which are hidden.
(b) E_{dep}, **dependency**: $u \to v$ when u requires v to discharge its own duty; annotated with whether the dependency is on an abstract interface or on a concrete implementation.
(c) E_{cmp}, **composition**: $u \to v$ when u is constituted by v; annotated with ownership versus mere use.
(d) E_{ctl}, **control**: $u \to v$ when execution passes from u to v; annotated with synchronous call, asynchronous message, event callback or data-driven dispatch, and with who schedules.
(e) E_{ctr}, **contract**: $u \to v$ when u promises something to v and requires something of v; annotated with preconditions, postconditions, invariants and failure semantics.

Keeping the five classes separate is not formalism for its own sake. Consider a single call graph: it would compress four distinct facts into one edge class—that a thread pool composes threads and queues, that it depends on synchronisation primitives, that it is invoked through a submission interface, and that it promises to accept no new tasks after shutdown—and would lose all structural information in doing so. The seam detectors of Chapter 5 are, in effect, procedures for recovering this five-way distinction.

Definition 2.12 (Abstract skeleton)**.** A **skeleton map** ϕ replaces each component by its role label in some duty classification and retains the type of each relation while discarding its instance. The image $\phi(\mathcal{G})$ is the **abstract skeleton** of $\mathcal{G}$.

The central proposition of Chapter 3 is stated with ϕ: $\phi(\mathcal{G}_1) \approx \phi(\mathcal{G}_2)$ can hold even when $\mathcal{G}_1$ and $\mathcal{G}_2$ have nothing in common at the source level.

2.4 Comprehension as task-constrained lossy compression

Definition 2.13 (Task)**.** A **task** is a pair $T = (Q, d)$, where Q is a family of **queries** $q : \mathcal{B} \to \mathcal{A}_q$ mapping system behaviour to observable answers, and d is a loss functional on answers.

Typical queries are: if I change this, which behaviours change? under what conditions does this path fail? who enforces this invariant? which modules must change to add a new entity type?

Definition 2.14 (T-sufficiency)**.** Let M be a model of system S at some level of abstraction. M is **T-sufficient** for S if

$$\forall q \in Q: \quad q\big(\mathcal{B}(M)\big) = q\big(\mathcal{B}(S)\big), \tag{2.6}$$

or, in the approximate form, $\mathbb{E}_{q\sim Q}\big[d\big(q(\mathcal{B}(M)), q(\mathcal{B}(S))\big)\big] \le D_T$.

Definition 2.15 (Description length and the rate–distortion objective)**.** The definition rests on the minimum description length principle [56, 40, 41]. Let $\ell(M)$ be the description length of M under a fixed encoding, and let $L_T(M) = \mathbb{E}[d(q(\mathcal{B}(M)), q(\mathcal{B}(S)))]$ be the task loss. Comprehending S means solving

$$M^\star \;=\; \arg\min_{M}\; \ell(M) \qquad \text{s.t.} \qquad L_T(M) \le D_T, \tag{2.7}$$

or equivalently the Lagrangian form $\min_M\; \ell(M) + \lambda\, L_T(M)$.

Proposition 2.16 (Stopping rule for descent)**.** *Parameterise the model family by descent depth, $M_0 \succ M_1 \succ \cdots$, with increasing index meaning increasing retention of lower-level detail, and suppose $\ell(M_{k+1}) \geq \ell(M_k)$ and $L_T(M_{k+1}) \leq L_T(M_k)$. Then the optimal depth $k^\star$ satisfies the stationarity condition*

$$\left.\frac{\partial L_T}{\partial \ell}\right|_{k^\star} = -\frac{1}{\lambda}, \tag{2.8}$$

and once $\partial L_T/\partial \ell = 0$ beyond some depth, further descent is strictly dominated.

Proof. The Lagrangian is $\mathcal{L}(k) = \ell(M_k) + \lambda L_T(M_k)$. First-order optimality gives $\mathrm{d}\ell/\mathrm{d}k + \lambda\,\mathrm{d}L_T/\mathrm{d}k = 0$, that is $\mathrm{d}L_T/\mathrm{d}\ell = -1/\lambda$. If $\mathrm{d}L_T/\mathrm{d}\ell = 0$ on an interval while $\mathrm{d}\ell/\mathrm{d}k > 0$, then $\mathrm{d}\mathcal{L}/\mathrm{d}k = \mathrm{d}\ell/\mathrm{d}k > 0$ and the objective increases monotonically, so further descent is strictly dominated. ∎

Proposition 2.16 yields a rule usable in practice.

Principle 2.17 (Descent depth)**.** To make a change correctly it suffices to descend until the invariant governing that change becomes visible, and then to stop. Further descent increases description length without reducing task loss.

The rule explains both why reading all the source is not comprehension and why reading only an architecture diagram is not comprehension either: the former passes the stationary point into the dominated region, the latter has not yet reached T-sufficiency.

2.5 Contracts, traces and assurance

Definition 2.18 (Behaviour contract)**.** A **behaviour contract** is an eight-tuple

$$S = \big(P,\ Q,\ E^+,\ E^-,\ R,\ I,\ F,\ A\big), \tag{2.9}$$

where P is a precondition; Q a postcondition; E^+ the set of permitted side effects; E^- the set of forbidden side effects; R a resource budget (time, space, bandwidth, energy); I the invariants that must hold throughout execution; F the failure, cancellation, retry and recovery semantics; and A the environmental assumptions (hardware, operating system, load, external services).

Equation (2.9) is not claimed to be a recognised specification format. Its purpose is to state that *correctness is not compressed into a return value but defined as a set of constraints on the whole execution.* That is the starting point of Chapter 7.

Definition 2.19 (Traces and the admissible set)**.** Let B be an executable artefact and E a runtime environment (processor, instruction set, operating system, loader, configuration, external services and devices). Let $\mathrm{Tr}(B, E)$ be the set of *observable execution traces* B may produce in E, including input and output, system calls, state mutations, I/O, concurrency events and failure behaviour, and let $\mathrm{Allowed}(S)$ be the traces permitted by contract S. Then B **satisfies** S in E if

$$\mathrm{Tr}(B, E) \subseteq \mathrm{Allowed}(S). \tag{2.10}$$

Inclusion (2.10) is far stronger than output correctness: it constrains *all possible behaviours*, including side effects that were never intended.

Definition 2.20 (Assurance and its condition)**.** An **assurance** is a four-tuple (property assured, object, premises, evidence). A property φ holds within scope Σ exactly when

$$\begin{aligned} &\text{environmental assumptions hold} \\ &\wedge\ \text{the contract is explicit and authorised} \\ &\wedge\ \text{the verifier accepts the implementation and its proof} \\ &\wedge\ \text{the transformations preserve the required semantics} \\ &\wedge\ \text{the execution boundary is effective} \\ &\wedge\ \text{the object executed is the object verified} \\ &\quad\Longrightarrow\ \varphi \text{ holds within the declared scope } \Sigma. \end{aligned} \tag{2.11}$$

The last conjunct—called here the **fourth pillar**—is frequently omitted, but it is not a detail: even if a verifier correctly accepts a binary, the preceding conclusion does not transfer if the deployment system actually runs a different file, or if loading changes a critical premise. Its formal expression is an identity assertion:

$$\operatorname{digest}\big(B_{\text{deployed}}\big) \;=\; \operatorname{digest}\big(B_{\text{verified}}\big) \quad\wedge\quad E_{\text{deployed}} \models A_{\text{claimed}}. \tag{2.12}$$

Equation (2.12) is why content addressing, reproducible builds and load-chain integrity belong to the theory rather than to deployment housekeeping.

2.6 Two modes of trust generation

Definition 2.21 (Trusted computing base)**.** For an assurance regime, the **trusted computing base** TCB is the set of components that must be trusted unconditionally for the assurance to hold: the proof checker and its execution, the logic and semantic definitions in use, the identity between the program a proof is about and the program that executes, the compilation, linking, loading and runtime mechanisms, and the assumed hardware and environmental conditions.

TCB minimisation is the precise restatement of the intuition that a system must not verify itself. Its exact content is this: a mechanical system does not *betray*, but it can be defective, can fail, and can embody an incomplete model. The basis of trust is therefore not that the mechanism is error-free, but that its decision does not depend on the generator's will. The two are not the same.

Definition 2.22 (Mechanical and consensus assurance)**. Mechanical assurance** is provided by a single deterministic, exhaustively inspectable decision procedure whose objects are finite, static and unambiguous symbolic objects, such as proof terms, typing derivations or constraint systems. **Consensus assurance** is provided by the adversarial scrutiny of multiple independent participants, and its objects are high-dimensional, dynamic and partly self-referential judgements, such as whether a specification faithfully expresses an intent.

The two modes are complementary rather than competing. Table 2.1 states the difference.

The load-bearing row is failure shape. Mechanical assurance fails discretely—not because it is more reliable, but because its object is a finite static object and the check can be exhausted. Consensus assurance fails continuously, because its object is intentional and can only be approached by a network. §7.11 gives the precise form of that continuity and proves a counter-intuitive consequence: the governing variable of consensus reliability is not the number of participants but the independence of their errors.

Table 2.1: The two modes of trust generation

Dimension	Mechanical assurance	Consensus assurance
Generation	A minimal decision procedure examines symbolic objects	Independent parties scrutinise adversarially and sign
Object	Formalised properties	Judgements that cannot be formalised
Strength	Absolute under the declared premises	Statistically reliable, not absolute
Failure mode	Defective decision procedure; lapsed premise	Collusion, capture, *cognitive isomorphism*
Failure shape	Discrete (defective or not)	Continuous (higher independence, lower failure)
Cost	Proportional to the complexity of the property	Proportional to participant count and independence
Disciplinary analogue	Checking a mathematical proof	Peer review, a jury, the judiciary

2.7 Contract operads and composition

Proposition 2.4 gives sufficient conditions for composition, but it lists them one by one. This section supplies the algebraic structure of contracts, so that the composition conditions become well-definedness conditions on that structure rather than a checklist.

Definition 2.23 (Contract operad)**.** A **contract operad** $\mathcal{O}$ consists of:

(i) a set of **colours** Col, whose elements are behaviour contracts (Definition 2.18);

(ii) for each $n \geq 0$ and colours $S_0, S_1, \ldots, S_n$, a set of **operations** $\mathcal{O}(S_1, \ldots, S_n;\, S_0)$ whose elements are *composites taking* $S_1, \ldots, S_n$ *as subcomponent contracts and satisfying* S_0;

(iii) **substitution**: from $f \in \mathcal{O}(S_1, \ldots, S_n;\, S_0)$ and $g_i \in \mathcal{O}(T_{i,1}, \ldots, T_{i,m_i};\, S_i)$ for $i = 1..n$,

$$f \circ (g_1, \ldots, g_n) \;\in\; \mathcal{O}\big(T_{1,1}, \ldots, T_{1,m_1}, \ldots, T_{n,1}, \ldots, T_{n,m_n};\, S_0\big);$$

(iv) **units** $\mathrm{id}_S \in \mathcal{O}(S;\, S)$.

Substitution is required to be associative and unital. Informally, a colour is what a part promises, an operation is how parts are assembled, and substitution refines a part without altering the promise of the whole. The definition is the standard one for an operad, coloured by contracts [67].

Definition 2.24 (Interface graph and freeness)**.** Let $f \in \mathcal{O}(S_1, \ldots, S_n;\, S_0)$. Take the subcomponents as vertices and draw a directed edge $u \to v$ when the guarantee of u is required by an assumption of v. The resulting digraph is the **interface graph** of f. If it is acyclic, f is said to be **freely generated** by its interface graph.

Proposition 2.25 (Structural form of the composition conditions)**.** *Conditions (a) and (b) of Proposition 2.4 are respectively equivalent to two structural conditions on the contract operad:*

(a) ***Well-typed colouring****: observation compatibility (condition (a) of Proposition 2.4) holds if and only if substitution is well defined on colours, that is, the composite never demands of a subcomponent an observation its colour does not declare;*

(b) ***Environmental closure****: assumption closure (condition (b) of Proposition 2.4) holds if and only if the interface graph is acyclic, that is, the composite can be built uniquely bottom-up from its subcomponents.*

Proof. For (a): if the observation maps are compatible, the projection of each subcomponent onto the interface agrees with the composite's observation, so substitution introduces no new colour requirement; conversely, if substitution is well typed on colours, every observable of the composite is traceable to an observation declared by some subcomponent colour, which is observation compatibility.

For (b): if the interface graph is acyclic, a topological order exists and substitution along that order yields the unique construction, with assumption closure verified level by level. Conversely, if there is a cycle $u_1 \to u_2 \to \cdots \to u_k \to u_1$, then the assumption of u_1 depends on the guarantee of u_k, whose guarantee depends (around the cycle) on the assumption of u_1; the two sides of the substitution define each other and the composite has no well-founded construction. To give it meaning one must supply a fixed-point semantics, that is, a separate existence-and-uniqueness condition for that cycle. ■

Part (b) accounts for two of the three composition failures in §7.4, and explains why two established practices must introduce an additional hypothesis:

- contract composition for dynamical systems must assume that the interconnection is *well posed*, which is exactly the fixed-point semantics supplied for a feedback cycle;
- software contracts fail on network-retry loops because retry turns an acyclic call graph into a cyclic interface graph, and the colouring is not re-verified once the cycle appears.

Remark 2.26 (Feedback is a structural change). Proposition 2.25 reduces composability to acyclicity of the interface graph. This has a non-trivial consequence: *introducing feedback is not a local change.* Any edit that turns an acyclic interface graph into a cyclic one—retry, cache write-back, bidirectional synchronisation, an event loop—invalidates the existing composition argument, regardless of whether the components remain individually correct. This agrees with engineering experience: such edits are a high-incidence region for regressions.

2.8 Notation

Table 2.2: Notation used throughout.

Symbol	Meaning
Λ_i	level: the six-tuple (Definition 2.1)
$\mathcal{O}_i \dashv \gamma_i$	Galois connection between levels (Definition 2.2)
$\mathrm{Obl}(\Lambda_i) = \ker \mathcal{O}_i$	oblivious set (Definition 2.7)
κ_i	compression ratio (Definition 2.10)
$\mathcal{G}$	architecture relational graph (Definition 2.11)
$\phi(\mathcal{G})$	abstract skeleton (Definition 2.12)
$T = (Q, d)$	task (Definition 2.13)
$\ell(M)$	description length (Definition 2.15)
$L_T(M)$	task loss (Definition 2.15)
S	behaviour contract: the eight-tuple (Definition 2.18)
$\mathrm{Tr}(B, E)$, $\mathrm{Allowed}(S)$	observable traces, permitted traces (Definition 2.19)
$\mathfrak{A}$	assurance (Definition 2.20)
TCB	trusted computing base (Definition 2.21)
$\mathcal{O}$, Col	contract operad and its colours (Definition 2.23)
R_i	inter-level interpretation relation (Remark 2.6)
$\rho(c)$	read-set of change c (Definition 6.1)

Symbol	Meaning
$\mathcal{K}_6$	the six functions (Definition 4.8)
D	technical debt (Definition 6.10)
$C(t)$, K	governable complexity, comprehension capacity (Definition 6.13)
$\mathcal{J}$	observation operator (Definition 5.9)
$\operatorname{rank}_{\mathcal{T}} \mathcal{J}$	observability rank on the task subspace (Definition 5.9)
q	correlated error mode (§7.11)

Chapter 3

Structural Convergence: Over-Constrained Solution Sets and Engineering Attractors

This chapter addresses Problem 1.1. High-level structure does converge, but by a different mechanism from the low level, and claims of different strength must be stated separately; the strength of convergence depends on domain binding, and its carrier is relations rather than components. The chapter closes with a falsifiable verification design.

3.1 Three Convergence Claims of Different Strength

Hypothesis 3.1 (The weak claim: recurrence of local patterns)**.** Software in different domains repeatedly adopts the same set of local structural solutions: layering and staging, separation of mechanism from policy, separation of control from data, events and state machines, separation of declarative description from execution engine, separation of front end from back end, and resource ownership and lifecycle management. What matters about these structures is not that they share a name, but that they share a *structural motivation*.

The weak claim has substantial engineering experience behind it, and it is also the weakest: it is almost impossible to falsify, and therefore has limited explanatory power.

Hypothesis 3.2 (The medium claim: structural similarity after abstraction)**.** For large software systems that come from different domains, are developed by different teams and carry different implementation details, once differences of implementation language, component naming and domain semantics have been eliminated, their high-level architectures exhibit a structural similarity *significantly above a random baseline* in some definable abstract space.

The medium claim is the core claim, and the operationalisable one. It can be operationalised (§3.6), and whether it holds depends on no metaphysical assumption.

Hypothesis 3.3 (The strong claim: single-point convergence)**.** Almost all large software that is normally biased eventually converges on one and the same high-level architecture.

The strong claim is unsupported by the evidence and is probably too strong. Its principal defect is that it conflates "there exists an attractor" with "there is only one attractor".

This paper takes Hypothesis 3.2 (the medium claim) as its core claim, treats Hypothesis 3.1 (the weak claim) as its degenerate form, and rejects Hypothesis 3.3 (the strong claim) as an overstatement.

3.2 Constraint Systems and Solution-Set Geometry

Definition 3.4 (Constraint system and solution set)**.** Let the design space be a d-dimensional manifold $\mathcal{D}$. A design problem is defined by c independent constraints $g_1, \ldots, g_c : \mathcal{D} \to \mathbb{R}$, and its **solution set** is $\mathcal{S} = \{x \in \mathcal{D} : g_j(x) = 0,\ j = 1..c\}$.

Proposition 3.5 (Over- and under-constraint)**.** *Under the notation of Definition 3.4, if the constraints are mutually independent and transversal, then*

(a) when $c \geq d$, $\mathcal{S}$ is generically a finite set (or empty);
(b) when $c < d$, $\mathcal{S}$ is generically a submanifold of dimension $\geq d - c$;
(c) when $c \ll d$, the dimension of $\mathcal{S}$ tends to d, that is, the solution set is almost unconstrained.

Proof. Under transversality, each independent constraint lowers the codimension by 1. By the transversality theorem (Thom), codimension is additive: $\operatorname{codim} \mathcal{S} = c$, hence $\dim \mathcal{S} = d - c$ (when $d - c \geq 0$); when $c \geq d$ the dimension is 0, that is, a discrete point set; when $c < d$ the dimension is positive, that is, a submanifold. ■

Proposition 3.5 gives a clean explanation of "unique below, manifold above", but the wording must be handled with care. It does *not* say that the low level really is formalised as a constraint system; it says that the effective dimension of the low-level design space is small and its effective constraints numerous, so that the solution set is degenerate in the geometric sense.

Example 3.6 (The low level)**.** Consider defining a function calling convention for a fixed instruction-set architecture. The design freedoms include where arguments are passed (registers or stack), who is responsible for saving registers, stack frame layout, where the return address is stored, how variadic arguments are passed, alignment requirements, and compatibility with existing binaries, debuggers, exception unwind tables, the system-call interface and the dynamic linker. The great majority of these constraints are not negotiable preferences but are fixed by *objects that already exist*: existing binaries must keep working, the debugger must be able to walk the stack, exceptions must unwind, and the kernel must be able to receive system calls. Hence $c \geq d$, and the solution set degenerates to very few feasible points, in practice often to a single point.

Example 3.7 (The high level)**.** Consider choosing an architecture for a task-processing system. The design freedoms include: function calls within a monolith, multi-process message passing, microservice RPC, event-driven asynchronous processing, the actor model, distributed workflow engines, and their continuous parameters along the following dimensions: deployment granularity, state ownership, consistency model, synchronous or asynchronous, failure boundaries, and data partitioning. Here the problem-level constraints are markedly fewer than the degrees of freedom, and $\mathcal{S}$ is a high-dimensional manifold.

Why, then, does the high level still converge? Because there is a *second set of constraints*, which come not from the problem but from the human organisations that build and maintain the system:

Principle 3.8 (Compression of the solution set by organisational constraints)**.** The under-constrained high-dimensional solution manifold at the problem level is compressed by organisational constraints into a few sharp peaks. These constraints include: the cost of communication between teams, the ceiling on cognitive load, the limits of deployment and operations capacity, the market for technical talent, the sunk cost of an existing technology stack, and regulatory and compliance requirements.

One corollary follows: *high-level convergence is not towards engineering optimality but towards structures navigable by a bounded organisation.* The strength law below makes this precise.

3.3 Mechanisms of Convergence

(M1) **Complexity constraint**. Let a system have N components. If dependencies are left unorganised, the number of potential directed dependencies reaches $N(N-1)$, and the undirected count $N(N-1)/2$. Maintainability therefore requires structure to be imposed on dependencies: modularisation, layering, explicit interfaces, constraints on dependency direction, subsystem boundaries. These techniques are not identical, but all of them solve one and the same problem: *limiting the propagation of complexity.* The classical argument for modularisation and information hiding is [49].

(M2) **Information-flow constraint**. A large class of software can be abstracted as "input → processing → state change → output" (with feedback, concurrent events, external side effects and continuously running state machines superimposed). Where different duties have different rates of change, error modes and resource requirements, separating them yields a concrete benefit: parsing from execution, business rules from data access, pure computation from side effects, and the control plane from the data plane.

(M3) **State-management constraint**. A system must answer: who owns a given state? who may modify it? when is it created and destroyed? how do several components observe its changes? how is concurrent access coordinated? how is consistency restored on failure? These questions are highly general, and therefore repeatedly produce similar structures: transaction and lock management, process and handle management, the state owner and event loop of a GUI, replicated state machines, entity lifecycle and system update.

(M4) **Cognitive constraint**. Large software is constrained not only by machines but by the limits of developer comprehension. If implicit relations hold between all components, then even a system that runs in the computational sense leaves its developers unable to predict the effect of a change. Humans therefore tend to build hierarchies, namespaces, modules, explicit interfaces, composable abstractions and locally comprehensible state. *Software architecture organises not only computation but comprehensibility under bounded cognition.*

(M5) **Evolutionary constraint**. Large software is not designed in one pass; it passes through initial implementation, feature growth, performance optimisation, fault repair, team handover, module refactoring and compatibility maintenance. Some decisions reveal their cost only as evolution proceeds, which produces a selection effect. Evolution in practice does not guarantee optimality: legacy baggage, compatibility requirements, organisational structure and sunk cost can all keep an undesirable architecture alive for a long time.

Mechanisms (M1)–(M3) come from the objective structure of the problem; mechanisms (M4)–(M5) come from humans and history. This distinction corresponds to the strength law of §3.4: the first three mechanisms produce cross-domain similarity, the last two produce within-domain similarity and path dependence.

The classical statements of mechanisms (M4) and (M5) are decomposition into levels under cognitive load [39] and the determination of system structure by communication structure [50]. The oscillation between a "no final state" view of architecture and multiple stable forms is treated in [61]; the practice by which organisational shape drives architecture back through team topologies appears in bounded contexts and domain-driven design [62].

3.4 Convergence Strength and Domain Binding

Definition 3.9 (Domain binding)**.** Let the semantics of level Λ depend on a set of domain axioms $\mathrm{Ax}(\Lambda)$ (for example, "an account balance may be negative" or "a frame must be presented within 16.6 ms"). Define the **domain binding** of Λ as

$$\mathrm{Dom}(\Lambda) \;=\; \frac{\big|\mathrm{Ax}(\Lambda)\big|}{\big|\mathrm{Ax}(\Lambda)\big| + \big|\mathrm{Ax}_{\mathrm{general}}(\Lambda)\big|}, \tag{3.1}$$

that is, the fraction of the level's axioms that are domain-specific. $\mathrm{Dom} = 0$ means that the level is determined entirely by general axioms (such as memory locality or the cost of concurrency coordination); $\mathrm{Dom} = 1$ means that the level is entirely domain convention.

Conjecture 3.10 (The strength law of convergence)**.** *Let* $\mathrm{Conv}(\Lambda)$ *be the structural similarity of level* Λ *under cross-system comparison (§3.6 gives the operationalisation). Then* Conv *is monotonically decreasing in* Dom*:*

$$\Lambda_1 \preceq \Lambda_2 \implies \mathrm{Conv}(\Lambda_1) \;\geq\; \mathrm{Conv}(\Lambda_2) \qquad \textit{whenever} \qquad \mathrm{Dom}(\Lambda_1) \leq \mathrm{Dom}(\Lambda_2). \tag{3.2}$$

Conjecture 3.10 has observable qualitative support: the closer to the physical and network layers, the stronger the convergence (the eight fallacies of distributed computing are hard constraints with no room for negotiation); the closer to the business domain, the weaker the convergence (an insurance core system and a recommender system are almost never isomorphic in their domain models); the closer to organisational structure, the more it is governed by communication structure; the closer to the maturity phase of a technology, the more its form converges; and the closer to core and high-risk modules, the more conservatively it converges.

This paper does not call Equation (3.2) a law, because it depends on the definability of Dom and the measurability of Conv, both of which must be established experimentally. Its value is to convert a vague directional judgement into a falsifiable monotonicity hypothesis.

3.5 Relations Rather Than Components

Principle 3.11 (Relations first)**.** Cross-system structural commonality resides mainly in *relations* rather than in *components*. Different systems need not have the same modules, but they may have similar relational graphs:

- which parts decide and which parts execute;
- which parts own state and which parts may only read it;
- which parts define stable contracts and which parts are allowed to change;
- which parts coordinate globally and which parts work independently;
- which dependencies may cross a boundary and which must be forbidden.

In the language of Definitions 2.11 and 2.12: what must be sought is not a universal architectural template but a set of structural relations that recur across different systems, together with the *conditions under which they hold*. This is the same line of thought as the six functions of Chapter 4, viewed from two sides: the six functions supply coordinates for *management duties*, the relational graph supplies coordinates *between duties*.

Example 3.12 (Abstract skeletons of a database and a browser)**.** A database engine comprises an SQL parser, a query optimiser, an execution engine, a buffer pool, a transaction manager and a storage engine; a browser comprises HTML/CSS/JavaScript parsers, a style and layout engine, a script execution engine, a rendering engine, a network subsystem and a cache persistence mechanism. Their module names, internal algorithms and concrete duties all differ. Yet under a suitable skeleton map ϕ, both present as: an *input and interface*

layer (receiving requests, parsing input, establishing an execution context); an *interpretation and planning layer* (converting external intent into an internally executable representation); an *execution and coordination layer* (scheduling work, coordinating subsystems, managing the execution process); and a *state and resource layer* (managing data, caches, lifecycle and persistence).

The value of Example 3.12 is limited: this skeleton is already broad enough to describe almost any system. An additional requirement on the abstraction level is therefore needed:

Principle 3.13 (Two-way constraint on the abstraction level)**.** An abstraction level worth studying must satisfy two directions at once: *upwards*, it must be able to identify structure common across systems; *downwards*, it must retain enough discriminating power that different systems remain distinguishable at that level. The more abstract the level, the more implementation differences are eliminated, but the more easily the description loses discriminating power; there is a proper middle level.

The coordinates of the six functions in Chapter 4 are designed precisely to satisfy this two-way requirement: they are narrower than "input–processing–output" (and therefore discriminating) and broader than any concrete module list (and therefore comparable across domains).

3.6 Verification Design

For the medium claim (Hypothesis 3.2) to be a scientific claim, it must be able to answer "compared with what". A complete test design has five steps.

Step 1: Build the architecture graph. For each system, build a graph $\mathcal{G} = (V, E)$ whose nodes are components or duties and whose edges are calls, dependencies, control flow or data flow. By Definition 2.11, *relations of different types must be recorded separately*, so that entirely different relations are not conflated.

Step 2: Define the abstraction map. Map concrete components to more general categories, for example: input and parsing, planning and decision, execution and coordination, state and persistence, communication and boundary, resource and lifecycle management. These categories must not be defined so broadly in advance that any system will appear similar.

Step 3: Choose discriminating structural metrics. Candidate metrics include: the degree of modularity of the dependency graph; the depth of the hierarchy; the proportion of cross-module dependencies; the concentration of state ownership; the degree of coupling between control flow and data flow; the network centrality of core components; and the direction of dependencies between duties.

Step 4: Construct a sound control group. It is not enough to compare a few well-known systems and find that all of them are modular. The observed similarity must be compared against an appropriate random baseline, controlling for system size, domain, platform, architectural style and historical technology stack.

Step 5: Distinguish genuine convergence from common origin. Two systems may adopt the same architectural pattern because they independently solved similar problems; because they used the same framework; because they were shaped by the same standard;

because they drew on the same design literature; or because they inherited the same historical architecture. *Only after these factors are controlled is there better reason to speak of independent structural convergence.*

Chapter 9 writes this design out as Hypotheses H_1 and H_2.

3.7 Counterexamples and Limits

The cases that invalidate or degrade the convergence claim must be listed explicitly:

(a) **Path dependence**. Once a structure is locked in by an ecosystem, a toolchain or a standard, it may persist for a long time even if suboptimal. The strength of the lock-in is proportional to the cost of migration and independent of the quality of the structure locked in.

(b) **Phase transition in size**. At the two sizes of 10^2 and 10^9 entities, the optimal structure is qualitatively different: the consistency model, the sharding strategy and the failure model all change. There is therefore no such thing as "the architecture of a large system", only "an architecture within some range of sizes". This directly limits any statement that "all large systems converge".

(c) **Organisational politics**. The determination of system structure by communication structure (Conway's law) means that even when the technical problem is identical, different organisations will produce different system boundaries. This is both part of the reason for convergence (when organisational shapes converge) and part of the reason for divergence (when organisational shapes differ).

(d) **Strongly coupled systems**. When the internal coupling of a system cannot be localised (for example, a tightly coupled real-time control system or a monolithic kernel), module boundaries may be forced rather than chosen, and comparing "architecture" itself becomes less meaningful.

(e) **Arbitrariness in the choice of abstraction level**. The choice of metrics in Steps 3 and 4 itself contains researcher degrees of freedom. Different duty classification systems yield different similarity conclusions. This is the most fragile methodological link in this line of work and must be mitigated by pre-registering the metrics and by robustness tests across several classification systems.

The falsifiable part of this chapter is Hypothesis 3.2 and Conjecture 3.10: if, after controlling for size, domain, platform and framework, cross-system structural similarity is not significantly above a random baseline, or if similarity is not monotone in Dom, then the central claim of this chapter is weakened (Hypotheses H_1 and H_2, Chapter 9). The weak claim 3.1 and Principle 3.11 are unaffected by this, because they are interpretive frameworks rather than empirical assertions.

Chapter 4

Internal Infrastructuralisation: The Conditional Necessity of the Six Functions

This chapter addresses Problem 1.2. The central claim is that any sufficiently complex software system reinvents an operating system inside its own problem domain. The cause is not that engineers imitate a kernel; it is that a set of semantic obligations cannot be evaded at scale. The chapter gives the premises, the proof and the criteria for that claim, together with a falsifiable test that can be carried out inside a single project.

4.1 The Phenomenon and the Terms

The observed phenomenon is this: once a system grows past a certain size, it begins to promote mechanisms that were previously scattered across it into unified system-level capabilities. Specifically:

- several facilities need to allocate and release resources;
- several subsystems need to manage object lifetime;
- several components need to execute asynchronous tasks;
- several modules need to share state and coordinate modifications;
- several facilities need unified error handling, logging and diagnostics;
- several execution paths need to obey the same thread-safety and concurrency rules.

If each module solved these problems on its own, the system would accumulate a mass of duplicated implementations, inconsistent behaviour and coupling that is hard to control. The system therefore begins to handle them centrally, producing common mechanisms, interfaces, managers, runtimes and execution models.

Definition 4.1 (Internal infrastructuralisation)**. Internal infrastructuralisation** is a system's passage through three successive stages:

(i) **commonality identification**: recurrent requirements are identified as one class of problem (a merger in problem space);

(ii) **mechanism unification**: the commonality is promoted to a common capability, so that different facilities access shared resources through common interfaces and obey common lifecycle, error-handling and concurrency rules (centralisation and standardisation of mechanism);

(iii) **structural unification**: upper-layer facilities no longer solve all lower-level problems directly, but build on unified rules (unification of structure).

In shorthand, commonality identification → mechanism unification → upper-layer composition.

Mechanism unification does *not* mean that all logic is gathered into one central module. A mature architecture can achieve unification *through protocols, interfaces and distributed ownership*. What is unified is the *rules and contracts*, not necessarily the *implementations and ownership*. That distinction is the key to avoiding a god object, and Chapter 6 states it as a design principle.

4.2 Participant–Resource Structure

Definition 4.2 (Participant–resource structure)**.** A **participant–resource structure** is a five-tuple

$$\Pi = \big(E,\ \mathcal{R},\ \to,\ \text{Ready},\ \text{Fault}\big), \tag{4.1}$$

where E is a set of concurrently live entities, $\mathcal{R} = \{r_1, \ldots, r_m\}$ is a finite set of shared resources, $\to$ is the transition relation of the operational semantics, Ready $\subseteq E \times \text{Conf}$ indicates which entities may execute in a given configuration, and Fault is a set of fault events.

The two quantities that matter are $n = |E|$ and $m = |\mathcal{R}|$. Section 4.3 proves that the independent variable deciding whether a unified infrastructure is needed is the structural complexity $n \times m$, not lines of code.

Definition 4.3 (Unified substrate)**.** A **unified substrate** for Π is a set of mechanisms $\mathcal{U}$ such that the interaction of every entity $e \in E$ with every resource $r \in \mathcal{R}$ proceeds through an interface declared by $\mathcal{U}$ rather than being implemented directly by e.

4.3 The Integration Cost Theorem and the Phase-Transition Threshold

Theorem 4.4 (Integration cost)**.** *Let the participant–resource structure Π have n entities and m shared resources. Assume:*

(a) without a substrate, every entity that participates in a resource r must implement correct interaction with r independently, including independent handling of the invariants of r;

(b) under maximal coupling, every entity participates in all m resources;

(c) with a substrate, every entity uses at most k substrate interfaces, and the substrate itself defines the semantics of each of the m resources once; the fixed cost of the substrate is F (design, implementation and long-term maintenance).

Then the number of interacting pairs is $\Theta(nm)$ without a substrate and $\mathcal{O}(nk + m)$ with one. The substrate is better than no substrate in expectation if and only if

$$n \cdot m \ >\ c\,(nk + m) \ +\ F \tag{4.2}$$

where c is the unit cost coefficient of one interaction.

Proof. Without a substrate: correct interaction of entity e with resource r requires an independent implementation, so the number of interacting pairs is $\sum_e |\mathcal{R}(e)| \le nm$, with equality under assumption (b), that is $\Theta(nm)$. In addition, for each resource r the n independent implementations must agree on the invariants of r; this is a consistency obligation whose number of failure modes is $\Omega(n)$ (any single implementation that deviates breaks the global invariant).

With a substrate: the semantics of the m resources are defined once each, at cost $\Theta(m)$; every entity interacts with the substrate through at most k interfaces, at cost $\mathcal{O}(nk)$; add the fixed cost F. The total is therefore $\mathcal{O}(nk + m) + F$.

Comparing the two expressions and normalising by the unit cost coefficient c gives (4.2). ■

Corollary 4.5 (Phase-transition threshold)**.** *Because k is the* bounded *interface usage of each entity, $\mathcal{O}(nk + m)$ is linear in n, whereas $\Theta(nm)$ is a product linear in both n and m. Hence when either n or m grows substantially, inequality* (4.2) *must hold. The phase transition occurs at $nm \sim F/c$, that is, when the* product of the number of entities and the number of resources *exceeds the order of the fixed cost of the substrate.*

Corollary 4.5 makes precise the intuition that "the code volume reaches some order of magnitude", and supplies its correct form:

Principle 4.6 (The threshold is not lines of code)**.** The independent variable deciding whether a unified substrate must be built is the product of the *number of live entities* and the *number of shared resources*, together with the *frequency of recurrent problems*, the *number of interactions between state and resources*, the *complexity of concurrent execution*, the *rate of module evolution*, and the *demands on consistency, performance and reliability.* Lines of code are only a proxy for these quantities.

This explains a fact that ought to be puzzling: two programs of a million lines each may have entirely different internal architectures. One may be a large body of mutually independent generated code, data files or linear business logic (m small, n small); the other may have complex shared state, asynchronous tasks and many interdependent subsystems (n and m both large). Only the second develops a need for a unified runtime, resource management and coordination mechanisms.

The central reduction (Section 4.2) supplies a second explanation here: when the number of live entities in a system jumps from 10^2 to 10^6, naive management undergoes a *phase transition.* For 100 objects, conditional branching and raw allocation suffice; for a million concurrent tasks, cache entries or connections, the potential interactions between entities are $O(N^2)$ and the complexity is infeasible, so pooling, indexing, layering, asynchrony, batching and cache locality are forced in. That set of techniques is exactly the full inventory of "kernel technology".

Example 4.7. A relational database engine builds its own memory-context system because a general-purpose allocator fragments and cannot track leaks under a load that "creates and destroys thousands of tuples per query". It therefore constructs allocation and bulk reclamation over a tree of contexts, which is an object pool inside the database domain. An asynchronous runtime builds its own work-stealing scheduler because operating-system threads are too heavy in the face of a million concurrent operations, and therefore constructs $M{:}N$ scheduling, which is user-space time-sharing. A jump in order of magnitude forces kernel technology in: this is the first necessity.

4.4 The Six Functions

Definition 4.8 (The six functions)**.** For a participant–resource structure Π, the **six functions** $\mathcal{K}_6$ are defined as

(F1) **Identity and lifecycle**: the identity, creation, liveness and termination relations of entities;

(F2) **Allocation and reclamation**: the allocation and reclamation policies for finite resources;
(F3) **Scheduling**: the selection function that chooses the next executor among ready entities;
(F4) **Naming and addressing**: the mechanisms that make entities discoverable, referenceable and traversable;
(F5) **Communication and synchronisation**: the interaction primitives between entities and the composition rules for concurrent transitions;
(F6) **Fault isolation and recovery**: the containment relations of faults and the replayability of state.

Theorem 4.9 (Conditional necessity of the six functions)**.** *Let* $\Pi = (E, \mathcal{R}, \to, \text{Ready}, \text{Fault})$ *be a participant–resource structure whose operational semantics is well defined (that is, in every reachable configuration the next transition is determined, or the non-determinism it permits is explicitly specified). If* $|E| > 1$ *and* $\mathcal{R}$ *is finite and contended, then the six functions of* $\mathcal{K}_6$ *have necessarily already been defined, either in the system's own code or in its host (operating system, language runtime, framework).*

Structural argument. It is shown item by item that the absence of any single function renders the semantics incomplete (an undefined behaviour exists).

(F1) Identity and lifecycle: if entities were indistinguishable, Ready could not be defined as a function on the set of entities; if entities had no creation and termination relations, the variation of E over time would be unspecified and $\to$ would be undefined at steps where "an entity appears or disappears".

(F2) Allocation and reclamation: $\mathcal{R}$ is finite and contended, so "more resources are requested at once than can be supplied" is a reachable state. Without an allocation policy, $\to$ is undefined in that state. If the resources of terminated entities are not reclaimed, available resources decrease monotonically and the system reaches resource exhaustion in finitely many steps; $\to$ is then undefined in the forward-looking sense.

(F3) Scheduling: if $|E| > 1$ and several entities are ready at once, $\to$ is no longer a function but a relation. Without a specified choice among ready entities, the "permitted non-determinism" is not delimited and the semantics is incomplete.

(F4) Naming and addressing: an interaction primitive (see F5) must designate the object of the interaction. Without a referencing mechanism, the object of an interaction cannot be expressed.

(F5) Communication and synchronisation: if entities interact, the global transition is not the simple product of local transitions; without composition rules, the semantics of concurrent transitions is undefined.

(F6) Fault isolation and recovery: Fault $\neq \varnothing$ (faults are a physical and engineering given). Without containment relations, a local deviation has global semantics, which is equivalent to the system having no well-defined behaviour under fault; without recovery rules, the system never returns to a specified state after the first fault.

In sum, the absence of any one function destroys the premise of well-definedness. ■ ■

Remark 4.10 (The nature of the theorem). Theorem 4.9 is a *conditional necessity* result, not an unconditional theorem. Its conditions are $|E| > 1$, $\mathcal{R}$ finite and contended, and faults reachable. Its proof is a structural enumeration argument, not a derivation in some formal system. It can be falsified by exhibiting a well-defined system that satisfies the premises and lacks one of the six. That judgement is itself open to challenge.

The wording of Theorem 4.9, "either in the system's own code or in its host", is essential. It separates two questions: whether the functions necessarily appear, and whether they must be built in-house.

4.5 Build or Adopt

Corollary 4.11 (Build-or-adopt dichotomy)**.** *By Theorem 4.9 the six functions must be defined, but where they are defined is a design choice. That choice is decided by the* domain criticality *of the invariants:*

(a) if the invariant carried by a function is domain-critical*, that is, the invariant can hold only under a guarantee owned by this system, the function must be built in-house;*

(b) if the invariant carried by a function is general*, that is, a general-purpose host can provide a guarantee of sufficient strength, the function should be adopted;*

(c) implementing a general function in-house produces the failure mode this paper calls reinvention of commodity parts*: a worse general-purpose mechanism re-implemented at a higher long-term cost of comprehension and maintenance.*

This criterion explains why the boundary between "application" and "platform" falls exactly where it does: *the set of invariants you are forced to own is the boundary of your kernel.*

Example 4.12. A game engine must build its own entity–component storage and frame scheduling, because a general-purpose host cannot provide the guarantee of a "sub-millisecond deterministic frame budget". A database must build its own storage engine and write-ahead log, because a general-purpose file system cannot provide the guarantee of "recovery to a deterministic state after a crash". Conversely, a web application should adopt garbage collection, thread scheduling and memory allocation, because the invariants of these functions are general and building them in-house yields only a worse version.

Remark 4.13 (A sharper criterion: monotonicity). Corollary 4.11 decides build versus adopt by whether the invariant is domain-critical. For functions of the "coordination" kind there is a sharper criterion that can be checked statically: *whether the computation is monotone.* Where monotonicity holds [69] (outputs grow with inputs and do not depend on negation or on aggregate global knowledge), coordination is not required and can be pushed down to the participants; non-monotonicity (uniqueness constraints, negation, threshold aggregation, non-commutative updates) forces some site to carry coordination.

This sharpens the claim for function (F5) (communication and synchronisation) and part of (F3) (scheduling) of Section 4.4 from "centralise once the scale is reached" to "centralise once non-monotonicity appears". For monotone subsystems the k of the integration cost theorem (Theorem 4.4) can be reduced to nearly a constant, because participants need not agree on shared state.

4.6 Cross-Domain Comparison

Theorem 4.9 predicts the pattern of Table 4.1: each column is a system from a different company, a different language, a different era and a different domain, while each row is the same organ. The table does not claim that these mechanisms can be identified one with another; it compares them along the same dimension of *management duty.*

Conversely, the *form* these six organs take in different hosts varies widely, and that variation is the concrete content of "the trunk converges while the branches diverge" in Chapter 3. The sources of that variation in form can be listed systematically:

(a) **Host memory model**. Systems with manual memory management (C/C++/Rust) must build their own allocator and even their own reclaimer (engine object reclamation, Blink's garbage collection in the browser); systems with managed memory spend the same effort on pause behaviour, because allocation is given and the pain point becomes the tail of the latency distribution.

Table 4.1: Independent implementations of the same set of management duties in different domains

Function	Linux	Chromium	Tokio	Bevy	Unreal	PostgreSQL
Identity/ lifecycle	task_struct, inode, file	Frame, Document, Node	Task, JoinHandle	Entity, Component	UObject, Actor	backend, xid, tuple
Allocation/ reclamation	buddy, slab, page cache	PartitionAlloc, Oilpan	slab-style task allocation	archetype column store	FMemory, GC	MemoryContext, shared_buffers
Scheduling	EEVDF, workqueue	task queue, RendererScheduler	work-stealing, waker	Schedule/System parallel executor	TaskGraph, GameThread	postmaster, autovacuum
Naming/ addressing	kobject, sysfs, fd	DOM, Web IDL	Handle, waker	Entity ID	FName, reflection metadata	OID, system catalog
Communication/ synchronisation	syscall, netlink, IPC	Mojo IPC	mpsc, oneshot	Event, Resource	delegates, replication	libpq, LWLock, extension hooks
Isolation/ recovery	namespace, cgroup, OOM	site isolation, renderer process sandbox	panic propagation, cancellation	state machine isolation	crash reporter	MVCC snapshots, WAL replay

(b) **Host concurrency model**. The runtime model determines the shape of the scheduling organ: a game engine uses frame-driven cooperative scheduling (predictable latency over throughput), a server event-driven preemptive scheduling (throughput first). Frame schedulers and work-stealing schedulers are therefore two subspecies of one organ.

(c) **Granularity of domain primitives**. A general-purpose kernel offers pages, threads, file descriptors and event polling; but a game engine needs frames, components and assets, a database needs buffer pools, transactions and version chains, and a browser needs document nodes, security contexts and cross-process objects. A mismatch in primitive granularity forces each domain to build its own infrastructure.

(d) **Evolutionary stage**. These internal operating systems are themselves continuously refactored, and each refactoring converges towards "a more complete set of the six functions" rather than diverging from it.

Even the operating system itself is circumvented by its users: high-performance network and storage stacks redo I/O by polling in user space, asynchronous I/O interfaces are redesigned, and the operating system cannot escape being "reinvented" by its own user space. This is not anomalous; it is a direct consequence of Corollary 4.5: when the $n \times m$ of user space crosses the threshold, user space grows a new kernel.

4.7 Transport and Re-Derivation

Table 4.1 shows the same set of functions recurring across domains. But "recurrence" admits two entirely different explanations, with entirely different predictions:

(a) **Isomorphism**: the six functions are six implementations of one structure and can be translated into one another;

(b) **Transport along the base**: the six functions are the fibres of some more general structure over different *problem shapes*, and when the shape changes the structure must be re-derived, not carried over.

The original treatment gives only (a). Distinguishing the two requires making the base explicit, namely "problem shape".

Definition 4.14 (Problem shape and level family)**.** A pair $\pi : \mathcal{L} \to \mathcal{P}$ is a **level family**, where

- $\mathcal{P}$ is a set of **problem shapes**, determined by the resource form, concurrency structure, failure modes and time scales of the domain;

- $\mathcal{L}$ is a set of layer implementations;
- the fibre $\pi^{-1}(p)$ is the family of layer implementations feasible at problem shape p.

π maps "an implementation" to "what shape of problem it solves". Reindexing π gives **transport along the base**: given $p \to p'$, an implementation at p is moved to p'.

Proposition 4.15 (What transport preserves)**.** *Let $p \to p'$ be a map between problem shapes. If the map is an* equivalence *(the same resource form, concurrency structure, failure modes and time scales), then the* slots *of $\mathcal{K}_6$ (the question each function must answer) are preserved under transport; the form of the implementation may change, but the slots are neither added nor removed. If the map is not an equivalence, the semantic requirement of at least one slot changes, and the implementation must be re-derived.*

Argument. Each item of $\mathcal{K}_6$ is uniquely determined by the semantic obligation of Theorem 4.9: identity by the distinguishability of entities, allocation by the finiteness and contention of resources, scheduling by the cardinality of the ready set, naming by the object of interaction, communication by the mode of composition of concurrent transitions, and isolation by the propagation range of faults. These six obligations depend only on the four components of problem shape. When the map preserves the four components, the obligation set is unchanged; when any component changes, the semantic requirement of the corresponding obligation changes with it. ■

Corollary 4.16 (An observable discriminator)**.** *By Proposition 4.15, differences in form fall into two classes:* implementation differences of the same slot over different bases *(for instance frame-driven and event-driven schedulers, whose bases have different time scales), and* differences of slot*. The former is not a counterexample; the latter is.*

Corollary 4.16 makes the middle proposition of Chapter 3 (Hypothesis 3.2) testable more rigorously: two systems must first be aligned on the base, and only then compared slot by slot. Comparing modules directly without aligning the base is the most common error in existing comparative studies of architecture.

Remark 4.17 (The status of this section). Definition 4.14 and Proposition 4.15 are a *structural proposal*, not an application of an established theorem of category theory. Their role is to make explicit a question that has not previously been posed, whether the recurrence is isomorphism or transport, and to give a discriminating corollary (Corollary 4.16). Promoting π to a strict fibration and proving the properties of the reindexing functor is unfinished work (item O in Chapter 11).

4.8 The Coarse-Graining Criterion

Theorem 4.9 establishes that the six functions necessarily *exist*, but not that they are the most *salient* structure. A stronger proposition is that the six functions are the *surviving set* of system structure under coarse-graining.

Definition 4.18 (Coarse-graining sequence)**.** Let the architecture relational graph of a system be $\mathcal{G}$ (Definition 2.11). A **coarse-graining** is a quotient map $q : \mathcal{G} \to q(\mathcal{G})$ that merges components while preserving relation types. A sequence of quotient maps $q_1, q_2, \ldots$ (each step strictly coarser than the last) is a **coarse-graining sequence**. A structure $\mathcal{S}$ **survives** under that sequence if there exists N such that for all $n \geq N$, $\mathcal{S}$ remains recognisable in $q_n(\mathcal{G})$.

Conjecture 4.19 (The six functions are the relevant structure)**.** *Under the coarse-graining sequence of a normally biased large system, the structures that survive are exactly the slots of $\mathcal{K}_6$ and their relational graph; the remaining structures (concrete modules, algorithm choices, data-layout details) are washed out at a sufficiently coarse granularity.*

The value of Conjecture 4.19 is that it yields a falsification procedure *inside a single project*: no cross-project statistical comparison is needed, only repeated coarse-graining of one code base, observing the surviving set. If the surviving set is not the six functions, the conjecture is refuted and the convergence thesis of Chapter 3 requires a different explanation.

Principle 4.20 (Form of implementation)**.** Coarse-graining can be implemented entirely mechanically: starting from a module dependency graph, three operations are applied repeatedly: (i) merge nodes whose fan-in and fan-out fall below a threshold; (ii) collapse transitive dependency edges; (iii) erase nodes that carry no invariant ($\mathrm{S}(\iota) = \varnothing$ in Definition 6.2). Each step records the change in the surviving structure. The whole experiment needs one repository and one dependency graph, with no labels and no manual classification, and is therefore unaffected by the "freedom in choosing a duty-classification system" noted in Section 3.7.

Chapter 9 states Conjecture 4.19 as Hypothesis H_{10}.

Remark 4.21 (Relation to renormalisation). The operations of Principle 4.20 are formally the same as coarse-graining in the renormalisation group: short-range degrees of freedom are eliminated and one observes whether long-range structure converges to a fixed point. The fixed point here is the six-function slots. It must be stated plainly that this does *not* mean that software possesses a renormalisation-group flow: software has no separation of scales (the discussion preceding Principle 4.22), hence no exponential decay of genuinely irrelevant operators. What is the same is the *operation*, not the *mechanism*.

4.9 Virtualisation and Deep Modules

The phrase "compress complexity and provide the upper layers with a high-level language and symbols" can be made precise. The core work of every infrastructure layer is one and the same action:

Principle 4.22 (Virtualisation)**.** An infrastructure layer translates what is *physical, continuous and volatile* into *discrete, stable, composable* domain-level symbols.

Example 4.23. Operating system: physical memory → virtual address space; block devices → files; hardware interrupts → signals. Database: physical pages → multi-version tuples → "rows" in SQL. Browser: byte streams → document trees; socket events → event-loop callbacks. Game engine: raw bytes in memory → components and assets; the time of one frame → a schedule table. Asynchronous runtime: event-poll bitmap → the Future of `async`/`await`.

In the vocabulary of module design [49, 53], these are all *deep modules*: minimal interface, very deep implementation. Behind one allocation call stands an entire allocator system; behind one task creation stands an entire scheduler. The duty of an infrastructure layer is to keep producing deep modules, sealing complexity below and exposing clean symbols above.

This connects directly to the theory of comprehension in Chapter 5: a deep module is precisely a level of very high compression ratio (Definition 2.10) that quotients away a large amount of lower-level difference while retaining the predictive power the upper layer needs. It also explains why good infrastructure is at once "high-performance" and "easy to understand": both are consequences of the same quotient map.

4.10 Functional Differentiation in Agent Systems

If Theorem 4.9 holds, the most recently emerged complex computational domain should be re-enacting this evolution. Table 4.2 gives the observation.

Table 4.2: Agent systems rediscovering the six functions

Function	**Counterpart in the agent domain**
Allocation and reclamation	the context window as fast storage; retrieval augmentation as page-fault handling; external memory as swap space; context compression as memory compaction
Communication and synchronisation	tool calls or function calls as system-call primitives
Scheduling	the agent main loop, multi-agent orchestration
Identity and lifecycle	lifecycle management of sessions, tasks and subgoals
Naming and addressing	tool registration and discovery protocols
Isolation and recovery	sandboxing, retries, checkpoints and rollback

The value of this observation is that it is a *natural experiment*: no one is deliberately imitating a kernel; complexity itself has pressed out the same anatomy. It also forms the empirical basis of Hypothesis H_3 (Chapter 9).

Theorems 4.4 and 4.9 hold under their stated premises; what can be challenged is the premises and the rigour of the structural argument. What is empirically falsifiable is whether the appearance of the six functions is predicted by $n \times m$ rather than by lines of code (Hypotheses H_3 and H_{10}), and whether there exists a well-defined system that satisfies the premises yet lacks one of the functions over the long run.

Chapter 5

Comprehension: Inverse Compression

This chapter addresses the first half of Problem 1.3: for a large system that cannot be read through from the micro level, at what level should it be understood, and how does comprehension lead into development. The central claim is that comprehension is inverse compression, whose optimal depth is given by the stationarity condition of a rate–distortion objective and whose feasibility is decided by observability rank.

5.1 Failure of Sequential Reading

A natural way to read is to follow the physical order and the control flow of the source code:

$$\texttt{main()} \to \texttt{module_a} \to \texttt{module_b} \to \texttt{function_c} \to \texttt{helper_d} \to \texttt{allocator} \to \texttt{operating_system} \to \cdots$$

One then keeps following downwards and ends in infinite recursion. The failure is not a lack of patience but structural:

Principle 5.1 (Calling relations are not cognitive dependencies)**.** The function-call relation is the topology of *implementation*; the cognitive-dependency relation is the topology of *comprehension*. The two are not isomorphic. In studying transaction isolation one may need to know how the transaction manager uses locks and snapshots, but not necessarily understand the entire virtual-memory subsystem of the operating system; conversely, in investigating anomalous memory usage the high-level business semantics of the transaction manager may be temporarily irrelevant.

The more fundamental reason concerns the relation between the direction of generation and the direction of comprehension. A system is designed *top-down* (intent first, then architecture, and only then the unfolding of code); the inverse process should therefore take the same direction. Reading code bottom-up is like understanding a cat by counting all its molecules: feasible in principle, absurd in practice.

The concession "feasible in principle" has a practical consequence: sequential reading is not *impossible* but *dominated*. Proposition 2.16 gives its precise form: beyond the stationary point, description length increases while task loss no longer falls.

5.2 Cognitive Maps

Confronted with a large monolithic repository, one should build a conceptual map before a file map. The division of labour is:

- a file map answers "where the code is";
- a conceptual map answers "what the system is made of, why these parts exist, who depends on whom, and what rules allow them to collaborate".

Definition 5.2 (The four cognitive maps)**.** For a system S and a task T, the four cognitive maps are:

(i) **system panorama**: compresses S into 5–15 major responsibility domains and answers: which responsibility domain is the most important? Why does each responsibility domain exist? What are the most important dependency relations among them? Which mechanisms are shared by several responsibility domains? Which states and behaviours decide the correctness of the whole system?

(ii) **key subsystem map**: for each key subsystem, answers eight questions: responsibility; boundary (what it is responsible for and what it is not); state (what state it owns and who may modify it); mechanism; interface; constraints; failure behaviour; and implementation entry points (which source files and tests can verify the above conclusions).

(iii) **key execution path map**: converts the static architecture into a model of dynamic behaviour. For example, the real questions behind a query path "input → parsing → planning → execution → storage access → return": when is the query plan created? Where is state kept during execution? Who manages the transaction snapshot? How is an error propagated upwards when a storage access fails? Which steps hold locks? Which operations may trigger disk I/O? Which resources must be released when a query is cancelled?

(iv) **contract map**: records the relations among components by each of the five edge classes of Definition 2.11, in particular the *oblivious set* of each layer, that is, what that layer is allowed never to care about.

Map (iv) is this paper's addition to conventional architecture documentation. Its necessity follows directly from Proposition 2.8: a contract that has not declared its oblivious set has not declared its compression boundary, and hence one cannot judge how far the level above may rely on it.

5.3 Operational Criteria for Comprehension

"Comprehension" must be operationalised, or it cannot be decided whether comprehension has been achieved. This paper adopts a five-capability criterion.

Definition 5.3 (Criteria for comprehension)**.** An agent (human or machine) is said to **understand** a subsystem M to the degree required by a task T if and only if it can do the following:

(U1) **Explain**: state why this subsystem exists;
(U2) **Predict**: predict what behaviour a given input, state change or error will cause;
(U3) **Locate**: decide which components to inspect when a fault occurs;
(U4) **Modify**: find the correct extension points and the constraints that must be respected when a feature is to be added;
(U5) **Verify**: know which tests or implementation details can substantiate its own judgement.

This paper calls a state that attains (U1)–(U5) **operational abstract comprehension**.

The relation between Definition 5.3 and Definition 2.14 is direct: (U2) is the behavioural statement of T-sufficiency; (U1), (U3) and (U4) are concrete query families in T; (U5) is the requirement that one can trace back to lower-level evidence, namely the two clauses "sufficiently accurate" and "verifiable" of Definition 2.15.

One diagnostic corollary matters here: if one can only recite the architecture diagram but cannot predict what a change will affect, the current abstraction is not yet sufficient. This and the seam detectors of §5.5 are one thing seen from two directions.

5.4 Reading Order

Principle 5.4 (Descent protocol)**.** Form the model top-down, descend to verify on demand, and never read sequentially.

The steps are as follows.

The Discipline Layer. Read the README, architecture documents, architecture decision records, design papers and the kernel documentation tree. About 1% of the time establishes about 80% of the skeleton model. The output is a hand-drawn organ diagram: which large blocks the system has, what each is responsible for, and how data flows.

Runtime Observation. Run the system first, read it afterwards. Runtime behaviour exposes structure and is far more honest than static code: observe the process and thread model; set breakpoints or trace the whole journey of one typical request; read logs and metrics, *the vocabulary of the logs is the system's self-description.* One end-to-end trace is worth ten thousand lines of static reading.

Interface Boundaries. Read the header files, public interfaces and exported symbols of modules in dependency order, not their implementations. An interface is a compression boundary (Proposition 2.8) and states what this layer promises; how it discharges that promise is not yet needed. What is wanted at this point is an organ atlas, not a tissue section.

Data Shape. The data model is the most stable abstraction and control flow the most volatile. Establish first: what the core entities are, who owns their lifecycle, which state machines exist, and what is writable and what is read-only. Once the shape of the data is in hand, control flow is only data rearranged over time.

Descent on Demand. Descend into code only with a concrete task in hand: fix this defect, add this feature. Reading is then directed retrieval rather than wandering. Descent follows the V-shape "call site → implementation → underlying mechanism": stop as soon as the current problem can be explained, then return to the upper layer and reassess the model.

Prediction and Verification. At each layer, write the mental model down first ("I guess the executor fetches the next tuple like this"), then verify it. *Being able to predict behaviour is comprehension.* If the prediction is right, the compression model is effective; if it is wrong, that spot is exactly where the system is most counter-intuitive.

Write-Back to the Contract Map. Write the newly acquired knowledge back into the contract map of Definition 5.2, filling in that layer's new guarantees, new assumptions and new oblivious set. This step is the only means of preventing cognitive drift: without write-back, the layered model gradually drifts away from the real system.

Remark 5.5 (An operational ratio). Understand about 10% of the code in depth while knowing where the other 90% is and who owns it. The ratio is not a precise target but a quantified reminder of the principle that comprehension is not exhaustion.

5.5 Detection Criteria for Module Boundaries

Architecture is "the places where a change is localised". The key operation of comprehension is therefore *finding seams*. The criteria below are ordered by reliability, and all can be executed mechanically or semi-mechanically.

(S1) **Where types stop crossing**. Where is data re-encoded, wrapped or degraded to a generic type (byte arrays, strings, maps)? This is a strong signal of a module boundary: where the type system stops providing guarantees is where a human must supply the contract.

(S2) **Units that can be tested in isolation**. Where does a test suite exist that can be run without starting the whole system? This is the sharpest criterion: *only a unit that can be verified in isolation is a real module*. Its reliability comes from Definition 2.14: a unit test is essentially the assertion that "the interface observation is a sufficient statistic of behaviour".

(S3) **Serialisation, version fields and copy boundaries**. Where are there version fields, serialisation, deep copies, cross-process or cross-network calls? Crossing such a boundary makes a contract visible and falsifiable.

(S4) **Centralised enforcement sites of invariants**. Where are there assertions, database constraints, type-level constraints or contract checks? Where invariants are enforced centrally is usually where a level boundary lies.

(S5) **Independent targets of the build system**. Where does the build configuration define an independent compilation target or package? This is a *necessary* but not a *sufficient* condition for a boundary: an independent target is not necessarily a real module, but a real module nearly always has one.

(S6) **Co-change graphs and incident history**. This is the most underrated technique. Examine the commit history: which files tend to be modified together in the same change? *A co-change graph gives semantic coupling, whereas an import graph gives syntactic coupling.* Where the two disagree is exactly where the architecture documentation disagrees with reality. Incident and defect histories then give the real invariant gaps.

Principle 5.6 (Summary of the seam criteria)**.** A module is real if and only if its interface observation is a sufficient statistic of its external behaviour; a boundary is real if and only if, once it has been crossed, changes on the other side no longer need to be understood.

Chapter 6 inverts Principle 5.6 and uses it as a design objective.

5.6 Active Identification and Passive Reading

Principle 5.7 (Comprehension as system identification)**.** Comprehending a complex system is essentially *system identification*: constructing a model that predicts its behaviour and quantifying the uncertainty of that model. The most efficient form of system identification is the "perturb–observe" loop.

Passive reading is an extremely poor way to sample, because it *has no error signal*. Every judgement it produces is unfalsifiable, and therefore cannot be updated. Active probing is the opposite: form a falsifiable prediction first ("if I change this, only these two modules should

be affected"), make a minimal change or run a test, then observe the prediction error. *The error is the part of the structure you cannot see.*

Proposition 5.8 (The advantage of active probing)**.** *Let comprehension be an iterative update of a model M, in which each observation o supplies information $I(\text{structure}; o \mid M)$ about the true structure. The observation distribution of passive reading does not depend on M, so its expected information gain is constant in M; the observation distribution of active probing depends on the current prediction $\hat{y}(M)$, and its expected information gain is proportional to the prediction error $\mathbb{E}\,|y - \hat{y}(M)|$. Hence, wherever the prediction error is non-zero, active probing has strictly higher marginal information gain.*

Argument. The samples drawn by passive reading are independent of M, hence $p(o \mid M) = p(o)$ and the information gain is constant. Active probing selects the observations that maximise predictive disagreement (D-optimal or uncertainty sampling in experimental design), so $p(o \mid M)$ concentrates on the region where the current predictions of M are uncertain; by the definition of information gain $I = \mathbb{E}[\log p(o \mid M) - \log p(o)]$, that quantity is non-negative and vanishes when $p(o \mid M) = p(o)$, that is, when prediction and truth agree. Hence active probing is strictly better wherever the prediction error is non-zero. ■

For AI the conclusion carries more weight: the advantage of AI is not that it can "read" more code but that it can *probe* at high throughput, generating hypotheses, running tests, making minimal changes, inspecting co-changes, reading incident records, and then updating the model. That is a frequency humans cannot match.

5.7 The Observability Criterion

§2.4 defines comprehension as task-constrained compression, and Proposition 2.16 gives the stopping point of descent. Both presuppose that continued observation can improve the model. This section identifies a stronger limitation: *when observation is insufficient, increasing the number of observations changes nothing.*

Definition 5.9 (Observation operator and task subspace)**.** Let $\mathcal{B}$ be the behaviour space of the system and $\mathcal{T} \subseteq \mathcal{B}$ the *task-relevant difference subspace*, that is, the subspace spanned by the differences that the query family Q of Definition 2.13 can distinguish. Let $\mathcal{J} : \mathcal{B} \to \mathcal{Y}$ be the observation operator *available* to the agent: it maps behaviour to what the agent can actually read (source code, logs, traces, test results, runtime metrics). Define the **observability rank** on $\mathcal{T}$ as

$$\operatorname{rank}_{\mathcal{T}} \mathcal{J} \;=\; \dim \mathcal{J}\big(\mathcal{T}\big), \tag{5.1}$$

the dimension of the task-relevant differences that remain distinguishable after $\mathcal{J}$. The construction is isomorphic to the observability criterion of linear system theory [72].

Proposition 5.10 (The observability criterion)**.** *An agent can identify the system to T-sufficiency (Definition 2.14) if and only if*

$$\operatorname{rank}_{\mathcal{T}} \mathcal{J} \;=\; \dim \mathcal{T}. \tag{5.2}$$

When (5.2) fails, the kernel of $\mathcal{J}$ on $\mathcal{T}$ is non-trivial: there exist two executions e_1, e_2 whose available observations are all identical while their task-relevant behaviour differs. In that case no number of additional observations can reduce task loss*, because new observations still lie in the range of the same $\mathcal{J}$.*

Proof. If (5.2) holds, then $\mathcal{J}$ is injective on $\mathcal{T}$, task-relevant differences are uniquely recoverable from observations, and T-sufficiency follows from Definition 2.14. If it fails, then $\ker \mathcal{J} \cap \mathcal{T} \neq \{0\}$; take a pair of executions $e_1 \neq e_2$ corresponding to a non-zero element of that kernel (in the task-relevant sense). The two are indistinguishable under $\mathcal{J}$, so every $\mathcal{J}$-measurable decision rule gives the same answer and is wrong on at least one of them. Adding observations does not change $\mathcal{J}$, and therefore does not change that kernel. ■

Proposition 5.10 separates "insufficient comprehension" into two cases that are entirely different in engineering terms:

Principle 5.11 (Distinguishing two kinds of comprehension failure)**.** (a) **rank deficiency**: the observation operator itself cannot distinguish task-relevant differences. The only remedy is to *change the observation*: add invariant checks, add trace points, add assertions, expose internal state. This is a design action, not a reading action.

(b) **full rank but insufficient samples**: observation suffices to distinguish, but has not yet been collected. The remedy is to *probe more*, with its efficiency governed by Proposition 5.8.

Confusing the two is the main reason advice such as "read more code" fails: when the problem belongs to (a), additional reading is information-theoretically ineffective.

Remark 5.12 (Measuring the cost of comprehension correctly). The cost of case (b) is a *sample complexity*, not an amount of reading. If the contract hypotheses are confined to a hypothesis class $\mathcal{H}$ (the set of operations in Definition 2.23 provides a natural candidate), then the number of probes required to identify the system to T-sufficiency at error ε has a lower bound decided by the distinguishing power of $\mathcal{H}$ on T, typically of the form $\Omega(d/\varepsilon)$, where d is the capacity dimension of the hypothesis class.

This accounting yields a testable prediction: *the factor by which designed probing beats exhaustive reading is quantifiable*, and that factor grows with the capacity of the hypothesis class. Hypothesis H_5 of Chapter 9 thereby acquires a quantitative form.

Example 5.13 (A typical shape of rank deficiency)**.** A service keeps all its state in process memory and exposes only the return values of requests. If the task is to decide "whether two concurrent requests can read each other's partial updates", then no number of return-value observations suffices, because the difference lies entirely outside the return values. The correct action is not to read more code but to introduce an observable: a runtime assertion of a concurrency invariant, or a test with interleaving injection. The example shows that the distinction of Principle 5.11 is practical rather than merely taxonomic.

5.8 Budget and Stopping Conditions

§2.4 has given the formal stopping criterion (Proposition 2.16). This section gives its engineering version and the allocation of budget.

Principle 5.14 (Allocation of the descent budget)**.** Allocate the comprehension budget in order of information gain per unit cost, not by code volume or directory order. The typical high-yield entry points, in order, are: external interface definitions and the data model > one trace of an end-to-end execution path > invariant enforcement sites > co-change hotspots > specific implementation details. Low-yield entry points are: sequential reading of implementation bodies, reading through a utility function library, and reading through generated code.

Principle 5.15 (Stopping conditions)**.** Stop descending as soon as any of the following holds:

(a) you can name every invariant governing the current change;
(b) further descent no longer changes your prediction of "what this change will affect";
(c) the cost of obtaining the next piece of evidence exceeds the expected rework cost of the current change.

Condition (a) corresponds to Principle 2.17; condition (b) to Proposition 2.16; condition (c) is an economic judgement: comprehension is not a matter of more being better, but of *enough*.

5.9 Implications for Machine Comprehension

The difficulty facing AI is not only a limited context length. Even when a model can read a large amount of code, three distinct problems remain: too much information drowns the important information; local pieces of information depend on one another in complex ways; and comprehension of details does not automatically form a correct system-level model. Enlarging the context window alone therefore does not solve every problem.

Principle 5.16 (A software cognitive architecture for AI)**.** An effective machine cognitive architecture should have four layers:

(i) **global cognitive model**: system goals, architecture diagram, key invariants, main data flows, core design decisions;
(ii) **subsystem cognitive model**: component responsibilities, interfaces, state ownership, dependency relations, lifecycle and error semantics;
(iii) **implementation-level index**: key functions, type definitions, call chains, source locations, tests and runtime evidence;
(iv) **verification and feedback**: compilation, tests, static analysis, execution traces, profiling, used to correct the models above.

Its key property is: *an upper layer model need not replicate all lower-level information, but need only retain enough information for the agent to find the correct lower-level evidence when it is required.*

Definition 5.17 (Traceable subsystem cognitive record)**.** The cognitive record of each subsystem should contain at least nine fields, listed in Table 5.1.

Table 5.1: The minimum field set of a subsystem cognitive record

Field	Role
Purpose	Why this subsystem exists
Contract	What guarantees it provides upwards (that is, $\mathcal{O}_i(\mathcal{B}_{i+1})$)
Oblivious	What this layer is allowed never to care about (that is, $\mathrm{Obl}(\Lambda_i)$)
State	What state it owns and who may modify it
Dependencies	Which other components it depends on
Invariants	Which conditions must never be violated, and where they are enforced
Lifecycle	How key objects are created, run and destroyed
Execution paths	How important requests or tasks pass through the system
Evidence	Which source files, tests or experiments support the above conclusions
Uncertainty	What has not been verified, and which assumptions may be wrong

The last two entries of Table 5.1, together with the Oblivious entry, are additions of this paper, for the following reasons.

On Uncertainty: AI cannot treat an architecture summary it has generated itself as fact. It needs to know which conclusions come from actual source code, which come from tests, and

which are only provisional inferences. This is the same requirement as the epistemic discipline of Chapter 11.

On Oblivious: as stated above, it is the compression boundary itself. Without this entry, a cognitive record describes only "what there is" and not "what may be disregarded", and so cannot support the modification capability (U4).

On dynamic update: when the code changes, the cognitive record must be updated in step. A usable machine comprehension system should therefore not be a static knowledge base but a *bidirectionally traceable* relation between the source code and the abstract model.

Principle 5.18 (Readability turns from a virtue into infrastructure)**.** The "comprehension interface" of a system is becoming a first-class engineering deliverable. Documentation used to be an appendix written for human readers; now machine-readable architecture descriptions, the practice of interface-as-documentation, and agent-facing working conventions are becoming components that a system must design, because the readership of code now includes a non-human intelligence that relies only on the compression layer.

The empirical content of this chapter is Hypothesis H_5 (Chapter 9): a comprehension process that uses a layered model together with active probing achieves a higher task success rate per unit cost than exhaustive reading. Propositions 2.16, 5.8 and 5.10 hold under the stated model; Definition 5.3 is a working definition whose value depends on whether it can be used to distinguish "comprehension" from "recitation", which requires experimental test.

Chapter 6

Design: forward compression

If comprehension is inverse compression, design is forward compression: compression boundaries are built in advance for future readers. This chapter turns that symmetry into an optimisable objective function, states its dynamical constraint, and shows where layering fails.

6.1 Read-sets and the objective function

The measurable objective of architecture is not an adjective such as "decoupling" or "high cohesion" but a number that can be counted.

Definition 6.1 (Change read-set)**.** Let the code base consist of a set of modules $\mathcal{M}$, namely $\bigcup_{c\in\mathcal{C}} \rho(c)$, where $\mathcal{C}$ is the set of changes. For a change c, its **read-set** $\rho(c) \subseteq \mathcal{M}$ is the set of modules that must be read in order to complete c correctly. The **architectural quality metric** of the system is

$$Q(\mathcal{M}) \;=\; \mathbb{E}_{c\sim\mathcal{C}}\big[\,|\rho(c)|\,\big], \tag{6.1}$$

that is, the mean read-set size weighted by the historical change distribution.

Definition 6.1 is operational: it can be estimated directly from version history, by counting for each merged change the number of top-level modules, files and directories it touches and taking the resulting distribution. This is not a perfect proxy (touching is not having to read), but it is *measurable*, and "degree of modularity" is not.

Definition 6.2 (Enforcement sites of an invariant)**.** Let $\mathcal{I}$ be the set of invariants of the system. For $\iota \in \mathcal{I}$, its **enforcement-site set** $\mathrm{S}(\iota) \subseteq \mathcal{M}$ is the set of all modules that *must be implemented correctly for ι to be guaranteed*. Enforcement is **effective** if ι is not violated in any execution.

Theorem 6.3 (Centralisation of invariants)**.** *Suppose that enforcement of invariant ι is effective and that a change c may break ι, that is, there exists a legal sequence of changes that breaks ι in the absence of an understanding of* $\mathrm{S}(\iota)$*. Then*

$$|\rho(c)| \;\geq\; \big|\mathrm{S}(\iota)\big|. \tag{6.2}$$

Hence, other things equal, minimising $\mathbb{E}[|\rho|]$ requires minimising $\sum_{\iota\in\mathcal{I}} |\mathrm{S}(\iota)|$, whose lower bound is attained by enforcing each invariant in a single module*.*

Proof. If c may break ι, an agent executing c must know where and how ι is guaranteed; otherwise it cannot judge whether c is safe. Every element of $\mathrm{S}(\iota)$ is a necessary source of

evidence for that judgement, so $\rho(c) \supseteq \mathrm{S}(\iota)$, which gives (6.2). Summing over ι: $\mathbb{E}[|\rho|] \geq \mathbb{E}[|\bigcup_\iota \mathrm{S}(\iota)|]$, with equality in the optimistic case in which the $\mathrm{S}(\iota)$ are disjoint, where the right-hand side is $\sum_\iota |\mathrm{S}(\iota)|$. Every non-empty $\mathrm{S}(\iota)$ contains at least one module, so the lower bound is attained at $|\mathrm{S}(\iota)| = 1$. ■

Corollary 6.4 (Optimal location of enforcement sites)**.** *Among all candidates satisfying the effectiveness constraint, the optimal enforcement sites are the modules of* least fan-in*, that is, those on which fewest other modules depend and which are therefore least likely to be circumvented by modification.*

Remark 6.5 (Limits of Theorem 6.3). The theorem has one premise: enforcement must be *effective*. In practice, concentrating an invariant in a single place often fails, because callers can circumvent that place. This is the real reason why "scattered invariants" recur in engineering. It is not ignorance but a defence against the risk of circumvention. The correct reading of Theorem 6.3 is therefore: *scattering invariants is a defence with a cost; where invariants can genuinely be centralised (for instance by a type system, private constructors, capability tokens or database constraints that make circumvention impossible), centralisation is strictly better than scattering.* This explains why "encoding invariants in types" has engineering value beyond expressiveness: it lowers $|\mathrm{S}(\iota)|$ to 1 without increasing the risk of circumvention.

6.2 Near-decomposability

Simon's near-decomposability [39] is the classical explanation of why a level structure is stable: interaction within a layer is dense, interaction between layers is sparse. But "sparse interaction" is not itself testable. This section gives a testable equivalent characterisation.

Definition 6.6 (Near-decomposable)**.** Let module M have an interface I. For an execution e of M, let $\mathrm{trace}_I(e)$ be the sequence of interactions of e on I and $\mathrm{behav}_{\mathrm{out}}(e)$ the behaviour of e observable outside M. Then M is **near-decomposable** with respect to I if there exists a function f such that

$$\forall e: \quad \mathrm{behav}_{\mathrm{out}}(e) \;=\; f\big(\mathrm{trace}_I(e)\big), \tag{6.3}$$

that is, the observation on I is a *sufficient statistic* for the external behaviour of M.

The relation between Definition 6.6 and Definition 2.14 (T-sufficiency) is immediate: take the query family Q of a task T to be "observations of external behaviour"; near-decomposability is then exactly the T-sufficiency of M with respect to its interface. Simon's qualitative principle is thereby translated into a decidable condition.

Proposition 6.7 (Leak pairs)**.** *M is* not *near-decomposable with respect to I if and only if there exist two executions e_1, e_2 such that*

$$\mathrm{trace}_I(e_1) = \mathrm{trace}_I(e_2) \qquad \textit{and} \qquad \mathrm{behav}_{\mathrm{out}}(e_1) \neq \mathrm{behav}_{\mathrm{out}}(e_2). \tag{6.4}$$

Such a pair is called a ***leak pair****.*

Proof. ($\Leftarrow$) If such e_1, e_2 exist, then no function f can satisfy both $f(\mathrm{trace}_I(e_1)) = \mathrm{behav}_{\mathrm{out}}(e_1)$ and $f(\mathrm{trace}_I(e_2)) = \mathrm{behav}_{\mathrm{out}}(e_2)$, since the arguments on the two sides are equal while the assigned values are not; hence (6.3) fails. ($\Rightarrow$) If (6.3) fails, no f satisfies the condition, and by the definition of a function there must exist two points with the same argument and different assigned values, which is (6.4). ■

Corollary 6.8 (A diagnostic procedure)**.** *Proposition 6.7 yields an* executable *diagnostic: construct two scenarios identical at the interface but differing in internal state or external environment, and observe whether the external behaviour diverges. If it diverges, the interface is not a sufficient statistic, and the module boundary leaks there.*

Corollary 6.8 turns "does this module's abstraction leak" from an aesthetic judgement into a differential test. Its practical value is that *whether an abstraction leaks is measurable, not merely felt.* The engineering commonplaces of "shadow state", "implicit globals" and "cross-request cache pollution" are all concrete instances of leak pairs.

Principle 6.9 (An equivalent design objective)**.** By Corollary 6.8, the testable objective of design can be written as: *for every claimed module boundary, the interface observation should be a sufficient statistic for external behaviour; the existence of a leak pair is the failure of that boundary.*

6.3 Technical debt

"Technical debt" is usually a metaphor in engineering usage. This section gives a definition that is precise and still useful.

Definition 6.10 (Technical debt)**.** Given an encoding, let the description length of the system code be $\ell(\text{code})$ and let its **shortest generating description** be $\ell^\star = \min\{\,|d| : d \text{ generates code}\,\}$, that is, the shortest program that generates that code in the Kolmogorov sense. Define the **technical debt**

$$D \;=\; \ell(\text{code}) \;-\; \ell^\star. \tag{6.5}$$

The definition is defensible but non-computable ($\ell^\star$ is not computable), so its value lies in the *direction* and the *criterion* it supplies, not in a numerical value.

Principle 6.11 (How technical debt accrues and is repaid)**.** Technical debt grows when *a change is not re-abstracted*: if a modification is not re-abstracted, the gap between the code and its "shortest generating description" widens. Therefore:

(a) refactoring = recompression, a survival necessity for holding the compression ratio above the comprehensibility threshold;
(b) a system that is not refactored does not stop growing; it accumulates incomprehensibility;
(c) deciding whether a refactoring is worthwhile is equivalent to deciding whether it significantly lowers D, that is, whether it makes the system easier to generate from a shorter description.

Principle 6.11(c) gives a practical criterion: if a refactoring does not let you say in fewer words what the module does, it is probably only reformatting.

Remark 6.12 (A rate–distortion reading of refactoring). Regard $(\ell(M), L_T(M))$ as a point in the rate–distortion plane [71] (Definition 2.15). The current state of a system is a point on the curve or outside it: *outside the curve* means that a model of the same description length attains lower loss, that is, that there is pure room for improvement; refactoring is then moving along the curve or moving the point back onto it.

This reading yields a testable prediction: *the slope of the rate–distortion curve, $\partial L_T/\partial \ell$, should predict which refactorings pay off.* A refactoring at a point where the slope is zero does not lower task loss and is therefore not worth doing; where the slope is large, it is worth doing even at high cost. If that predictive power does not hold in real projects, the choice of objective function in this chapter requires revision (Chapter 9, hypothesis H_{12}).

6.4 Governable complexity and comprehension capacity

Section 6.1 defines architectural quality as a static quantity $\mathbb{E}[\rho]$. Real systems, however, have a *rate*: changes arrive at some frequency, and an organisation's reading capacity is finite. This section supplies that dynamics; it is the only place in the paper where time is a variable.

Definition 6.13 (Governable complexity and comprehension capacity)**.** Let $\lambda(t)$ be the change arrival rate (changes per unit time) and let $\mathbb{E}[\rho]$ be the expected read-set (Definition 6.1). Define the **governable complexity**

$$C(t) \;=\; \mathbb{E}\big[\rho\big] \cdot \lambda(t), \tag{6.6}$$

that is, the information throughput required to maintain the system, in units of module-reads per unit time. Let the **comprehension capacity** of the organisation be

$$K \;=\; h \cdot b, \tag{6.7}$$

where h is the number of people who can be applied to comprehension and b is the comprehension bandwidth of one person (module-reads per unit time). The system is in a **governable state** if $C(t) \le K$ for all relevant t.

Proposition 6.14 (Architecture fixes the scaling exponent)**.** *Let a system have n features, each changing independently at rate λ_0, so that $\lambda \propto n$. Let the read-set $\mathbb{E}[\rho]$ scale as n^α in n, with $\alpha \ge 0$ fixed by the architecture, $\alpha = 0$ corresponding to complete localisation and $\alpha = 1$ to unconstrained coupling. Then the governable complexity scales as*

$$C \;\propto\; n^{1+\alpha}. \tag{6.8}$$

From $C \le K$, the required headcount scales as $h \propto n^{1+\alpha}$. Therefore:

(a) if the architecture presses α to 0, headcount grows linearly with system size and the system can persist indefinitely;

(b) if $\alpha = 1$, headcount must grow as the square *of size, and the system must become unstable at some size.*

Proof. By Definition 6.13, $C = \mathbb{E}[\rho] \cdot \lambda$. Substituting $\lambda = n\lambda_0$ and $\mathbb{E}[\rho] = \Theta(n^\alpha)$ gives $C = \Theta(n^{1+\alpha})$. From $C \le K = hb$ it follows that $h \ge C/b = \Theta(n^{1+\alpha})$. ∎

Corollary 6.15 (The actual role of architecture)**.** *Proposition 6.14 assigns architecture a precise duty: it does not lower the cost of an individual change but* lowers the exponent with which cost grows with size. *This explains why architectural benefit is invisible in small systems and decisive for survival in large ones, and why the charge of "architectural over-engineering" always appears to hold at small scale.*

Corollary 6.16 (Derivation of two classical conclusions)**.** *Equation (6.8) yields two classical conclusions:*

(a) ***Relation between organisation size and system size****: from $h \propto n^{1+\alpha}$, organisation size must grow with system size, and the growth exponent is fixed by the architecture. This is the* dynamical *version of the correspondence between organisational structure and system structure [50]; that correspondence is a static statement (structure mirrors organisation), whereas the dynamical version gives the governability bound.*

(b) ***Adding people makes a project slower****: increasing h raises K linearly, but at the same time raises the number of changes in flight and hence λ, and raises $\mathbb{E}[\rho]$ through the interfaces it adds. When the growth of the latter two exceeds that of the former, C/K worsens rather than improves. This is an in-model explanation of the empirical rule that adding people to a late project makes it later.*

Principle 6.17 (Design implication)**.** By Proposition 6.14, the criterion of an architecture review should not be "is the read-set small" but *"what is the scaling exponent of the read-set in size"*. A system whose read-set is currently small but grows rapidly with size is more dangerous than one whose read-set is currently larger but scales flatly. This requires hypothesis H_4 to be strengthened: the value of a read-set lies mainly in its *scaling*, not in its *magnitude*.

Remark 6.18 (The only dynamics)*.* The paper has so far been static: structure, invariants, read-sets and contracts are all time-invariant quantities. Definition 6.13 is the only dynamical variable, because it is the only place where "what an organisation can process" and "what the system demands" must match in *rate*. Elsewhere time can be eliminated.

6.5 The design procedure

(D1) **Fix the invariants first**. Before writing any code, write down the propositions that must hold forever of this system (for example "the total balance is conserved", "an I/O thread never blocks", "the version number increases monotonically"). Invariants are the gravitation of a level; all structure crystallises around them. By Theorem 6.3, each invariant should designate a single enforcement site and state why circumvention is impossible.

(D2) **Choose the primitives**. Atoms, components, messages, transactions, entities, and so on. Once the primitives are chosen they become the shared language of all subsequent discussion of the system. *Choosing the primitives is the most important single decision at this level*, because it fixes the shape of every expression thereafter.

(D3) **Fix the data model**. Entities, relations, lifecycles, ownership. A data model is an order of magnitude more stable than logic, so it should be frozen first.

(D4) **Cut the organs**. The basis of modularisation is not "similar functionality" but *information that changes together being hidden together* [49]. The test for each module is that its interface is small enough to hold in mind at once and its implementation deep enough to be replaced wholesale [53]. The interface is the compression summary you deliberately maintain; the implementation is the compressed content that may be rewritten.

(D5) **Make the physical laws executable**. An invariant written in a document rots; only encoded in a type system, assertions, contract tests and property tests does it endure. *Types are architectural documentation enforced by the compiler.* This is the confluence of "designing for comprehensibility" and "designing for correctness": an invariant that a machine can check need not be maintained from memory by a human or an AI.

(D6) **Record decisions**. Write down why each choice was made and what was given up. Code is the result; a decision record is part of the generating rules, and it spares the reader performing inverse compression the most expensive guessing.

(D7) **Refactoring = recompression**. See Principle 6.11.

One meta-rule runs through all seven:

Principle 6.19 (Design for the reader's mental bandwidth)**.** Design for the mental bandwidth of your reader, not for your typing speed. Code is read far more often than it is written, and the users of an architecture are future comprehenders.

6.6 Cross-cutting concerns and the limits of layering

The failure modes of layering must be acknowledged explicitly. One class of concerns cannot be localised to any layer:

- security and authorisation (every module may introduce a vulnerability);

- multi-tenant isolation (spanning storage, computation and network);
- distributed tracing (which must run through every call path);
- transactions and consistency (across the boundaries of state owners);
- observability and quotas (required at every point of resource consumption).

Principle 6.20 (Cross-cutting concerns are independent dimensions)**.** For cross-cutting concerns, the mature treatment is to acknowledge them as *independent dimensions* (policy plane, capability model, taint propagation, service mesh) rather than forcing them into the hierarchy and pretending that they are a layer.

This principle follows directly from the discussion in Chapter 3: a level structure is supported by near-decomposability, and near-decomposability presupposes sparse interaction between layers. When a concern has a non-trivial interaction with every module, it breaks near-decomposability by definition, and therefore cannot be represented as "a layer". The result of forcing the representation is that the level structure fails, and the failure is usually misdiagnosed as "insufficiently thorough layering".

6.7 Design checklist

The following checklist turns this chapter into executable items. Each item should be answered explicitly at a design review.

Table 6.1: Design checklist for large systems

No.	Check
A1	Which propositions must hold forever of the system? Where is each enforced? Why is circumvention impossible?
A2	Is the enforcement site of each invariant unique? If not, has the cost (the read-set) been explicitly accepted?
A3	What are the primitives of this level? Are they the shared language of all subsequent discussion?
A4	Has the data model been frozen before the logic? Are entities, relations, lifecycles and ownership explicit?
B1	Is each module's interface small enough to hold in mind at once?
B2	Is each module deep enough to be replaced wholesale?
B3	For each claimed boundary, does a leak pair exist? (That is: two executions with the same interface trace and different external behaviour.)
B4	Has each module's *oblivious set* been written down explicitly?
B5	Which modules change together? Has this been verified against co-change history rather than intuition alone?
C1	How many unrelated modules must be modified to add a new feature? (The measured read-set.)
C2	Does replacing the underlying implementation force large-scale modification of the business logic?
C3	When a subsystem fails, where does the failure propagate?
C4	Does adding a task type require rewriting the shared scheduling logic?
C5	Does concurrent access break state invariants? Where is this detected?
C6	Are performance bottlenecks concentrated in a few critical paths?
D1	Have the invariants been encoded in executable form (types, assertions, contract tests, property tests)?
D2	Can each module's contract be verified in isolation? If not, is it a module?
D3	Does each critical subsystem have a traceable record of what is known about it (Table 5.1)?
D4	Does the decision record cover why each choice was made and what was given up?

No.	Check
D5	Are cross-cutting concerns treated as independent dimensions rather than disguised as a layer?
E1	Does the system need a unified substrate? Judge by $n \times m$, not by lines of code.
E2	Of the six functions, which are built and which are adopted? The criterion is the domain-criticality of the invariants.
E3	For the parts that are built, are their invariants ones that a general-purpose host cannot provide?
E4	For the parts that are adopted, is the strength of the guarantee offered by the host sufficient to support this layer's contract?

Theorem 6.3 and Propositions 6.7 and 6.14 hold under the stated models. The empirical content is hypotheses H_4 and H_{12}: that the scaling exponent of the read-set outperforms read-set magnitude and conventional size metrics; that centralising invariants (where circumvention is impossible) lowers both the read-set and the defect rate; and that the rate–distortion slope predicts the benefit of a refactoring. If all three are refuted, the choice of objective function in this chapter requires revision.

Chapter 7

From Intent to Verifiable Execution

This chapter addresses Problem 1.4: if the cost of coding tends to zero and no human reviews the implementation line by line, how is correctness guaranteed mechanically. The answer has three parts: decompose correctness into dimensions that can be treated separately, define the intermediate process as a verifiable refinement chain, and organise trust as a composite of two modes of generation. The chapter also identifies an entry point on that chain which cannot be mechanically closed, and the sense in which consensus can approximate it.

7.1 Dimensions of Correctness

An AI rewrites a 1 ns call into a 1 μs call: functionally identical, but a thousand times slower. The example shows that "correct" is not a single dimension.

Definition 7.1 (The five dimensions of correctness)**.** See Table 7.1.

Table 7.1: The five dimensions of correctness and their mechanical checkability

Dimension	Question to be answered	Mechanically checkable?
Functional correctness	Do the outputs and the state changes meet the requirements?	For an explicit specification, provable or testable
Side-effect correctness	Has the program performed unauthorised I/O, writes, network requests or state changes?	Constrainable by effect specifications, permission boundaries and runtime isolation
Resource correctness	Do time, memory, bandwidth and energy meet the budget?	Partly provable statically; partly measurable only, or statistically guaranteed in a specified environment
System correctness	After components are composed, are concurrency, failure recovery and state consistency still correct?	Partly formally provable; partly requiring model checking, fault injection and runtime verification
Correctness of goal expression	Is the specification the machine executes really the goal the human wants?	**Not derivable automatically from a program-correctness proof**

The last row of Table 7.1 is the crux of the whole problem: *a program may implement a wrong specification correctly.* A compiler that is entirely correct can compile a wrong program into machine code entirely correctly; a formal proof is equally unable to rescue a specification that omits a critical requirement. The historical statement of this distinction is the separation of verification from validation in software requirements engineering [52].

7.2 The Verifiable Refinement Chain

Definition 7.2 (Verifiable refinement chain)**.** A **verifiable refinement chain** consists of seven stages:

(G1) **Purpose and intent**: what is to be achieved; which outcomes count as success; which side effects are unacceptable; how to trade off conflicting goals.

(G2) **System and environment model**: a representation of runtime states, events, operations, resources, operating-system interfaces, the external environment and observable behaviour.

(G3) **Formal behaviour contract**: permitted behaviour, forbidden behaviour, invariants that must hold, progress conditions, resource budgets and environmental assumptions, as in Definition 2.18.

(G4) **Realisability decision**: check whether the contract is self-contradictory, and whether the goal is genuinely achievable under the specified resources, hardware, operating system and environmental conditions.

(G5) **Implementation synthesis and search**: generators, search algorithms or other synthesisers produce candidate implementations. A candidate implementation is not trusted in advance.

(G6) **Correctness proof from abstract behaviour to binary**: check the candidate implementation together with its compilation, linking and other transformations, and prove that the generated machine code satisfies the behaviour contract in the specified environment.

(G7) **Attested deployment and actual execution**: ascertain that what is running is the verified binary, execute it in the specified environment, and monitor, constrain and handle failures for the conditions that cannot be guaranteed statically.

The significance of this chain lies not in adding seven process steps but in *assigning a different mathematical task to each.* The model (G2) should not be a natural-language description but an interpretable model of system state, behaviour and environment; the realisability decision (G4) asks whether the requirements are satisfiable; synthesis (G5) searches for possible implementations; (G6) establishes a proof from the implementation to the required behaviour; and (G7) ensures that the object actually executed is the object the proof is about.

Principle 7.3 (What crosses a boundary)**.** At every boundary what crosses is not only an artefact but also *what that artefact stands for*, and *the evidence for why it may be accepted.*

Remark 7.4 (The entry point that cannot be mechanically replaced)*.* Verification results and counterexamples can feed back into (G2), (G3) and (G4); only the attestation of intent in (G1) cannot be replaced by a general code-correctness proof. The reason is given in §7.5.

7.3 Transition Edges and Evidence

The seven stages of Definition 7.2 compress into five transition edges, each with its typical evidence and its critical gap. Table 7.2 states them.

Remark 7.5 (Heterogeneity of the relations R_i)*.* By Remark 2.6, the interpretation relation on different transition edges need not be of the same kind. For I→R and R→S the available "relation" may be only a traceability mapping rather than a formal relation [1, 2]; for P→C→B it may be a compiler's semantics-preservation theorem [22, 24], a single translation verification [23] or a translation attestation [26]; for B→E it may be a decision by runtime monitoring [34]. Forcing them into one kind of relation is a mistake.

Table 7.2: Five transition edges, typical evidence and critical gaps

Edge	**Input → output**	**Typical proof/evidence**	**Key assumptions and principal gaps**
I→R	Natural language/dialogue → structured requirements, domain facts and items requiring clarification	Stakeholder attestation, links to requirement sources, examples and counterexamples, conflict reports	Intent has no automatically given mathematical truth value; external facts and conflicts of interest cannot be settled by type checking alone
R→S	Requirements → formal properties/contracts	Traceability matrices, requirement-level tests, consistency/satisfiability/realisability analysis, vacuity and mutation analysis	A formal mapping must preserve *prohibitions and positive examples* alike; an omitted requirement cannot be derived from internal consistency alone
S→P	Specification → program/system candidates	Synthesis algorithms, SMT, model checking, Hoare/type/dependent-type proofs, counterexample-guided inductive synthesis	Finite DSLs, logic fragments and search budgets; over-strong premises and vacuous specifications must be prevented
P→C→B	Source program → stage-by-stage intermediate representations → machine code/deployable artefact	Verified-compiler theorems, per-translation verification/attestation, proof-carrying code	Source and target semantics must be defined; linking, loading, assembly, the runtime, third-party libraries and hardware are not necessarily within the proof's scope
B→E	Binary and configuration → behaviour in the real environment	Deployment integrity, system verification, runtime monitoring, isolation and capability mechanisms, performance and scheduling analysis	The model must be aligned with the real CPU, operating system, network, scheduling, failure and service assumptions

Remark 7.6 (Existing engineering on each edge). Table 7.2 is not a purely speculative scheme: verified compilation [22] and verified compiler back ends [24] on P→C→B, kernel-level refinement proofs [25], and proof-carrying code [15] have already achieved substantial progress for specific languages, compilers, kernels and systems. What remains genuinely unresolved is how to extend these high-assurance local results to a wider range of targets and a greater variety of runtime environments, and to systems whose implementations may be generated arbitrarily or adjusted dynamically.

7.4 Composition Conditions and Failure Modes

Proposition 2.4 already gives sufficient conditions for composition. Three concrete counterexamples of composition failure are added here; their role is to show that "locally correct" does not entail "globally correct".

Example 7.7 (Three composition failures)**.** (a) The source program is semantically correct, but the linker selects the wrong library.
(b) Each service satisfies its interface contract on its own, but the contracts' assumptions about network retry are mutually incompatible.
(c) The functional proof holds, but the latency budget does not cover cache contention.

The common structure of the three is that one of conditions (a) or (b) of Proposition 2.4 is violated.

Read through the contract algebra (Definition 2.23, Proposition 2.25), the three counterexamples are no longer mutually unrelated; they fall into three classes of failure:

Corollary 7.8 (Common structure of the three counterexamples)**.** *(a) Linker selects the wrong library:* colour mismatch*, the colour of the linked implementation being unequal to the colour declared at the link point;*
(b) Contracts' assumptions about network retry are mutually incompatible: a cycle appears in the interface graph*, retry turning an acyclic call graph into a cyclic one while the colouring is not re-verified after the cycle appears (Remark 2.26);*
(c) The latency budget does not cover cache contention: a missing contract component*, the resource component R never entering the colour, so that cost is treated as an implementation detail.*

The three are, respectively, a colour whose extension is too small, a changed interface graph, and a missing contract component.

The conclusion is that composition rules must explicitly incorporate these interface conditions together with *non-functional conditions*.

Remark 7.9 (The holistic character of assurance). Corollary 7.8(c) and Remark 7.23 together show that no verification conclusion actually asserts "property φ holds"; what it asserts is the conjunction

$$\varphi \ \wedge \ A_1 \ \wedge \ A_2 \ \wedge \cdots \wedge A_k,$$

where the A_i are the full set of environmental assumptions. The number k cannot be enumerated in advance, and an assumption A_i becomes visible only when it is actually violated. This is the form taken by the Duhem–Quine thesis [64] in software verification: there is no decisive test of a single proposition, only a test of the whole conjunction. Its immediate consequence is that the strength of "the tests pass" is strictly limited by the *set of assumptions that were not enumerated*. This also explains why Table 7.3 must attach a "what must not be claimed" list to every class of evidence: that list is the known A_i.

7.5 The Intent Gap

This is the section that requires the most care.

Problem 7.10 (The intent gap)**.** How is an intent I, first expressed by a human in natural language, guaranteed to be transformed into a correct formal specification Φ? The hope is that $I \equiv \Phi$, where the equivalence means that "the behaviour Φ requires fully reflects the human's real goal".

A popular argument holds that, by Gödel's incompleteness theorem and Rice's theorem, the gap between intent and specification is *principled* and *can never be mechanically closed.* The argument does not hold.

Proposition 7.11 (Why this paper does not accept that argument)**.** *Gödel's incompleteness theorem concerns the limits of* formal provability *in formal systems satisfying certain conditions; it does not directly state that a machine cannot decide whether a requirements specification matches a particular person's real intent. Rice's theorem [28] states that non-trivial semantic properties of general programs are undecidable; that is a fundamental limit on general static analysis, but it is not a theorem about the relation between a person's mental intent and a formal specification. Deriving "intent can never be fully formalised" from it requires additional philosophical and cognitive assumptions. Turing's original result [27] likewise applies only to explicitly encoded computational problems and general algorithmic claims.*

Does the intent gap exist, then? It does, but the grounds are *epistemic*, not *logical.*

Proposition 7.12 (Three epistemic grounds for the intent gap)**.** *(i)* ***Requirements are generative, not enumerative****. A person cannot list their requirements completely before seeing the system's behaviour. The requirement "do not leak privacy" may not surface at all until the system is seen actually collecting data. This is not logical undecidability but a constructive limitation of human cognition:* preferences are activated by the environment, not present beforehand as a complete list.

(ii) ***A specification is lossy compression of intent****. Intent lives in a causal, embodied, evolutionarily shaped mind; a specification is a finite symbolic object. Lossy compression means that information is necessarily lost, and which parts are lost cannot be known in advance. This is a phenomenon of information theory, not of metamathematics.*

(iii) ***The distinction between verification and validation has not been overturned****. Verification answers "does the implementation conform to the specification"; validation answers "does the system meet the real need". The distinction exists and persists precisely because the latter cannot be mechanically reduced to the former.*

Combining Proposition 7.11 with 7.12 gives the paper's precise statement of the problem:

Principle 7.13 (The precise statement of the intent gap)**.** The specification-transformation process is not a step that can be mechanically guaranteed *in general*; the reason is not logical undecidability but that human preferences are dynamic, situational, and not fully knowable before they are encountered.

This correction does not weaken the claim; it replaces the load-bearing wall. It also shows that machines can help clarify requirements, propose counterexamples, generate scenarios, find contradictions and check a specification for internal consistency, whereas which specification is finally adopted, and whether it fully expresses the goal, still requires an identifiable accountable party.

Remark 7.14 (The intent gap is the frame problem)*.* The three grounds of Proposition 7.12 are not three independent findings; they point jointly to an older problem. To formalise an

intent one must first decide which facts are *relevant*; and deciding relevance requires already knowing which conclusions are to be drawn, which is exactly the frame problem: without knowing the answer in advance, one cannot enumerate the relevant premises.

The intent gap is therefore not a defect of formal methods but an instance of the frame problem. The identification has two consequences:

(a) It explains why stronger formal tools do not solve the problem: tools improve *derivation*, whereas the problem lies in *the enumeration of premises*;

(b) It explains why *interaction* is the only effective means. The frame problem is unsolved in general and is evaded only in systems that possess a perception–action loop: an agent need not enumerate relevance in advance, because the environment gives feedback on its actions. The probing loop of Chapter 5 and the traceable refinement of this section both move that evasion mechanism up to the requirements layer.

Interactive refinement is therefore forced, not selected from among candidate approaches.

7.6 The Asymmetry Between Verification and Generation

A second formulation requiring correction is the description of "grading is easier than sitting the exam" as "the universal dividend of P versus NP". The description does not hold: whether P equals NP remains open, and it is not known in general whether finding a solution is necessarily harder than checking one.

This correction must be distinguished from a second point: the open status of P versus NP does *not* make the phenomenon "verification is cheaper than generation" fictitious.

Proposition 7.15 (Structural asymmetry)**.** *The structural asymmetry of polynomial-time certificate checking is* the definition of *the class* NP *itself: that a solution can be verified efficiently is a basic fact about computation, and it does not depend on whether* $\mathrm{P} \neq \mathrm{NP}$ *has been proved.*

Argument. The definition of NP is "there exists a polynomial-time verifier V and a certificate w of polynomial length such that $x \in L \iff \exists w : V(x, w) = 1$". The definition itself asserts the existence and the efficiency of verification, and asserts nothing about the complexity of finding w. The effectiveness of the verifier architecture therefore rests on the definition itself and is independent of the eventual answer to P versus NP. ■

This cost scale indicates the precise boundary of "verification is cheap". The limit is not the open status of P versus NP but *certificate length*: checking a proof can be done in polynomial time, while the proof itself can be astronomically long. Hence:

Principle 7.16 (The two components of verification cost)**.** "Verification is cheaper than generation" must be decomposed into two independent propositions:

(a) **Checking time is cheap**: given a certificate, the decision is efficient (Proposition 7.15);

(b) **Certificate size is cheap**: the certificate itself is short enough to be stored, transmitted and re-checked.

(a) is guaranteed by the definition of NP; (b) is *not*, and in many practical systems it is the real bottleneck. A verification regime that satisfies only (a) is unusable in engineering, because no one can re-check a certificate they cannot finish reading.

Principle 7.16(b) is measurable: for a given prover and class of properties it suffices to measure the distribution of certificate sizes. It also explains why a "small kernel plus large proof" architecture must in practice always be paired with a *readable* proof-summary layer (invariants, prose statements of contracts, counterexamples) rather than supplying only proof terms.

Example 7.17 (Historical cost scale of formal verification)**.** The initial functional-correctness proof of the microkernel seL4 took about 11 person-years (excluding investment in reusable tools and frameworks); the proof scripts of the time ran to about 200,000 lines and verified about 8,500 lines of C code [25]. That scale indicates the historical cost structure of formal verification. AI may substantially reduce the cost of writing proofs, finding invariants and repairing proofs, but specification design, model selection, proof architecture and the trusted base remain real work.

7.7 Grades of Evidence

Different kinds of evidence provide support of different strength, and conflating them is the most common error in this area. Table 7.3 gives the grades of evidence together with the corresponding "what must not be claimed" list.

Table 7.3: Grades of evidence

Type of evidence	Conclusions it can support	Conclusions that must not be claimed
Machine-checked formal proof	For the stated proposition, holds under the given logic, definitions, axioms, libraries and checker assumptions	Does not prove that the proposition fully expresses the intent; does not cover unmodelled environments; does not by itself establish latency or energy consumption
Verified-compiler theo-rem/translation certificate	Between specific source and target semantics, the defined preservation or refinement relation holds, or a single translation result is verified	Does not mean that the linker, operating system, hardware and external services are verified
Static analysis and abstract interpretation	Proves or conservatively estimates the properties of the analysed program, relative to a sound approximation	Does not guarantee precision or the absence of false alarms; is not a measurement of actual performance
Testing and mutation testing	Finds counterexamples, assesses the specified behaviour on the listed tests, and provides evidence over a finite sample	Does not prove that all inputs and all executions are safe; a pass rate is not a general proof of semantic equivalence of specifications
Statistical performance evidence	An empirical distribution under specified data, sampling procedure, hardware and confidence method	Is not a hard-real-time worst-case execution time; cannot be extrapolated unconditionally to a deployment whose distribution has changed
Runtime monitoring	Gives an online decision about the observed traces under the monitor's semantics	If observation is incomplete, "not detected" must not be stated as "all executions satisfy"

Principle 7.18 (Three states that must be distinguished)**.** "A violation was detected", "no violation has yet been observed" and "all executions are proved safe" are three different states, and none may be substituted for another.

Definition 7.19 (Artefact evidence list)**.** Every artefact should carry ten items of evidence, listed in Table 7.4. The purpose of the list is to make "may this binary be trusted" an auditable question rather than a question about the identity of its author.

Items 9 and 10 are the two most easily omitted and the most important: the former makes the boundary of the guarantee explicit, the latter turns assurance from a one-off check into a lifecycle mechanism.

Table 7.4: Artefact evidence list

No.	Evidence item
1	Specification version and requirement traceability identifiers
2	Formal specification and environmental assumptions
3	Hash of the candidate source code
4	Compiler, flags, target architecture and dependency versions
5	Proof and checker versions
6	Resource-analysis model and results
7	Test and monitoring strategy
8	Trusted computing base inventory
9	**Known uncovered behaviour**
10	**Re-verification rules** (which changes trigger recomputation of the affected evidence)

7.8 Side Effects and Constraint Mechanisms

The example in Problem 1.4 mixes side effects of several kinds. Their mechanical mechanisms become visible only when the kinds are separated.

Definition 7.20 (Four classes of side effect)**.** See Table 7.5.

Table 7.5: Classification of side effects and the corresponding mechanical mechanisms

Class	Example	Mechanical mechanism	Point of verification
Functional	A generated function modifies global state it must not modify	Type systems, effect types (which make "what will be touched" part of the signature), ownership systems	Compile time, at zero runtime cost
Resource	1 ns becomes 1 μs; memory blows up	Fuel/gas metering, worst-case execution time analysis, quotas	Both compile time + runtime
Security	Unauthorised access, sandbox escape	Capability models, sandboxes, bytecode verifiers	At admission + at runtime
Semantic drift	The specification itself is wrong	**No mechanical mechanism**	Human attestation and consensus

The first three classes are already covered by mechanical mechanisms, and are arranged in depth (proof > types > testing > metering > isolation > monitoring, each layer having a simpler trust base than the layer above it). Only the fourth class, a specification that departs from the intent, always requires humans.

Remark 7.21 (Assurance scope of three engineering prototypes). Three common engineering mechanisms cover different kinds of property and must not be advertised side by side:

(a) A browser sandbox guarantees *execution isolation and a permission boundary*, not functional correctness. It is the largest untrusted-code runtime on earth: every day it executes hundreds of millions of pieces of code of unknown provenance that have never been reviewed; it never reviews, it isolates.

(b) A bytecode verifier verifies *the safety properties its rules cover* (no out-of-bounds access, bounded loops, no dangling pointers), not the correctness of arbitrary business logic. A deterministic verifier of a few tens of thousands of lines adjudicates the admission of millions of lines of free code; this is precisely an instance of "compiler-like assurance that does not depend on anyone's will".

(c) A resource-metering mechanism guarantees *constraints under an abstract resource accounting*, not real wall-clock time. It turns a performance side effect from something "a human must notice" into "a budget violation automatically refuses execution", but it need not catch a degradation from 1 ns to 1 μs, because it meters abstract operation costs rather than actual hardware latency. Real-time performance assurance requires timing analysis and a restricted runtime environment, a completely different engineering discipline.

7.9 Runtime Assurance

Formal proofs depend on environmental assumptions, and the actual environment may change. Beyond static proof, therefore, the executing system must remain safe when an assumption fails.

Principle 7.22 (Layered defence)**.** An ideal architecture combines static proof (for known deterministic behaviour) with dynamic monitoring (for unknown or non-deterministic behaviour), forming a layered defence: whatever can be proved is proved first; residual risk is detected by independent monitoring; serious violations have their impact limited by isolation or rollback.

Remark 7.23 (The monitor's own boundary)*.* Even a runtime monitor that is itself correct does not thereby possess the complete facts of an execution. Missing observation traces, ambiguous events, unreliable transport and concurrency that cannot be seen all limit its conclusions. The observation mechanism of the monitoring system must therefore itself be included in the assurance chain.

Remark 7.24 (Four conditions for contract composition)*.* The composition of local properties depends on four things: the environment genuinely satisfies the subcomponent assumptions; the connection topology is well posed; the feedback condition is checkable; and the observation and time semantics are compatible. "Individually correct" for components and "globally correct" for the system are not interchangeable.

Principle 7.25 (The conditional form of an invariant)**.** A usable assume–guarantee contract should state simultaneously: the environmental conditions it depends on, the behaviour it guarantees outwards, the side effects it may produce, the resources it must consume and occupy, and the constraints that must hold when it is composed with other subsystems.

7.10 Artefact Binding

The last conjunct of the assurance condition (2.11), namely that the object actually executed is the object verified, is frequently omitted from existing discussions. It is not a marginal detail: even if a verifier correctly accepts a binary, the preceding verification conclusion does not transfer automatically if the deployment system uses a different file, or if the loading environment changes a critical premise. Supply-chain attacks aim precisely at this crack.

The complete trust chain must therefore include content-addressed build outputs, a cryptographic binding between the verification evidence and the binary, and re-verification at deployment. In the notation of (2.12), $\mathrm{digest}(B_{\mathrm{deployed}}) = \mathrm{digest}(B_{\mathrm{verified}})$ and $E_{\mathrm{deployed}} \models A_{\mathrm{claimed}}$.

Principle 7.26 (A three-way distinction for reproducible builds)**.** Three properties of different strength must be distinguished:

(a) **Reproducible**: the same inputs produce a byte-for-byte identical output;
(b) **Verifiable**: an independent third party can rebuild from the inputs and compare;

(c) **Verified**: a proof that a particular binary satisfies its declared properties.
(a) and (b) are preconditions for (c) to take effect; (c) does not imply (a). The three are not interchangeable.

7.11 The Reliability Bound of the Consensus Layer

§2.6 showed that the intent layer admits no mechanical solution. What does it admit? This section gives a precise answer and proves a counter-intuitive result.

Theorem 7.27 (Consensus reliability bound)**.** *Let n verifiers judge independently the proposition "whether a specification faithfully expresses the intent", each erring with probability at most $p < 1/2$. Then the probability that majority voting errs satisfies*

$$\Pr\big[\textit{majority errs}\big] \;\le\; \exp\Big(-2n\Big(\tfrac{1}{2}-p\Big)^2\Big). \tag{7.1}$$

Proof. Let X be the number of verifiers who err, $X = \sum_{i=1}^{n} X_i$, with the X_i independent and $\Pr[X_i = 1] \le p$, so $\mathbb{E}[X] \le np$. A majority errs exactly when $X \ge n/2$. By Hoeffding's inequality [58], for $t > 0$, $\Pr[X - \mathbb{E}[X] \ge t] \le \exp(-2t^2/n)$. Taking $t = n(1/2 - p)$ (so that $X \ge n/2$ implies $X - \mathbb{E}[X] \ge t$) gives

$$\Pr\Big[X \ge \tfrac{n}{2}\Big] \;\le\; \exp\Big(-\frac{2n^2(1/2-p)^2}{n}\Big) \;=\; \exp\Big(-2n\big(\tfrac{1}{2}-p\big)^2\Big). \qquad \blacksquare$$

This result is the quantitative form of the jury theorem [59]. It relies on independence, however, and independence is a fragile assumption; its failure has an equally precise form.

Proposition 7.28 (Bound dominated by a correlated error mode)**.** *Suppose that the errors of n verifiers have the following structure: with probability q a* common error mode *C occurs (for instance a shared training-data bias, a shared domain misunderstanding or a shared incentive), in which case all verifiers err simultaneously; with probability $1 - q$ the verifiers err independently, with error rate $p' < 1/2$. Then*

$$\Pr\big[\textit{majority errs}\big] \;\ge\; q. \tag{7.2}$$

In particular, when q is bounded below by a positive constant, increasing n cannot push the probability that the majority errs below q.

Proof. When $C = 1$ all n verifiers err, so the majority necessarily errs. Hence $\Pr[\text{majority errs}] \ge \Pr[C = 1] = q$. Taking the limit in n does not change this lower bound. $\blacksquare$

Corollary 7.29 (Diversity, not number)**.** *By Proposition 7.28, the governing variable of consensus reliability is* the independence of errors *(that is, the size of q), not the number of verifiers. The central goal of consensus design is therefore to reduce q: admit participants with different backgrounds, models with different training provenance, reviewers with different interests, and explicitly designed* adversarial *roles.*

Remark 7.30 (Use of this result). Corollary 7.29 yields a criterion directly usable in engineering judgement: when someone proposes "adding a few more agents to review this", the right question is "are the new reviewers independent of the existing ones in their error modes". If they are not, increasing the number increases cost without increasing reliability. This also explains why "reviewing with another model from the same source" does not constitute a guarantee structurally.

7.12 The Incentive Structure of the Intent Layer

Proposition 7.28 attributes consensus reliability to the parameter q but gives no method of estimating q. This section identifies the nature of that omission: the principal sources of q are *collusion* and *shared interest*, so it is not a statistical parameter but a problem of *incentive design*.

Principle 7.31 (The composition of q)**.** The common error mode decomposes into three sources:

(a) **Cognitive isomorphism**: the verifiers share training provenance, domain assumptions or habits of thought;
(b) **Interest isomorphism**: the verifiers' payoff is tied to "passing" rather than to "being right";
(c) **Information isomorphism**: the verifiers read the same material under review, so their errors are determined by the same defect.

The three sources correspond to three different interventions: admitting heterogeneous participants (for (a)), changing the payoff structure (for (b)), and introducing independent information channels (for (c)). Acting on the first alone is the common practice and the least effective.

Principle 7.32 (Proper scoring rules)**.** For the second source, the correct remedy is to tie a verifier's reward to the *accuracy* of the judgement rather than to its *lean*. A proper scoring rule [70] supplies a ready-made instrument: the reward function attains its unique maximum at the true belief, thereby removing the incentive for both "going along with the majority" and "pleasing the submitter". A concrete form is a prediction market on the proposition "does this specification faithfully express the intent", settled on the defects and rework data observed subsequently. The mechanism has two by-products:

(a) It provides a means of reducing q other than replacing people;
(b) It yields a posterior estimate of q: it suffices to measure the historical correlation of verifier errors.

This section closes the gap left by §7.11. The bound there is conditional, and its condition (independence) is here reduced to a quantity that can be designed, measured and paid for. Chapter 9 writes it as hypothesis H_{11}.

7.13 The Consolidated Assurance Condition

Combining the parts of this chapter yields a complete, conditional statement of assurance.

Definition 7.33 (Two-layer trust structure)**.** The chain from intent to execution is a composite of a two-layer trust structure:

$$\underbrace{I \xrightarrow{\text{consensus}} \Phi}_{\text{non-formalisable layer}} \longrightarrow \underbrace{\Phi \xrightarrow{\text{mechanical}} P \xrightarrow{\text{mechanical}} B \xrightarrow{\text{mechanical+monitoring}} E}_{\text{formalisable layer}} \tag{7.3}$$

In the formalisable layer, trust is guaranteed by mechanical verifiers; in the non-formalisable layer, trust is guaranteed by adversarial consensus; and the boundary between the two layers, the record of the consensus, and the traceability of the whole chain are guaranteed mechanically by cryptographically anchored infrastructure.

Theorem 7.34 (Consolidated assurance condition)**.** *Let I be an intent, Φ an attested specification, B a candidate implementation (binary) and E a runtime environment, with A the*

declared environmental assumptions. Then "property φ holds within the declared scope Σ" holds if and only if the following conditions hold simultaneously:

$$\textit{(i)}\quad \Pr\big[\Phi \textit{ is faithful to } I\big] \geq 1-\varepsilon_{cons}, \qquad \varepsilon_{cons} \textit{ determined by (7.1) and } q; \tag{7.4}$$

$$\textit{(ii)}\quad \mathrm{Tr}(B,E) \subseteq \mathrm{Allowed}(\Phi); \tag{7.5}$$

$$\textit{(iii)}\quad E \models A; \tag{7.6}$$

$$\textit{(iv)}\quad \mathrm{digest}(B_{deployed}) = \mathrm{digest}(B_{verified}); \tag{7.7}$$

$$\textit{(v)}\quad \varphi \textit{ is preserved under the interpretation relation } R \textit{ in use.} \tag{7.8}$$

Condition (i) is probabilistic, and its reliability is governed by q; conditions (ii)–(v) are mechanical, and their strength is limited by TCB and by the semantic model. Equation (7.3) and Theorem 7.34 together give the complete answer to Problem 1.4:

Principle 7.35 (Answer to Problem 1.4)**.** "Humans no longer read the code" is attainable; "humans need no longer answer for the expression of the goal" is not. The workable division of labour is: humans establish and bear responsibility for the goal and the constraints; generators perform large-scale generation, search and optimisation; mechanised verification and the execution boundary judge and constrain implementations; and the system always carries traceable assurance evidence.

Question	**This paper's judgement**
Q1 Can it be proved that the intent and the specification agree?	Not by general automatic methods. Controlled languages, domain ontologies, examples and human attestation can narrow the problem, but cannot cover implicit requirements that were never expressed (Proposition 7.12)
Q2 How can a specification be proved sufficient?	Consistency, satisfiability, realisability, vacuity and formal coverage metrics can all be checked; none of them proves that the specification contains every undeclared prohibition
Q3 How do local guarantees yield an end-to-end guarantee?	It requires that relations compose, that environmental assumptions close and that observation maps are compatible (Proposition 2.4); liveness and real-time properties usually require additional conditions
Q4 Can functional and resource guarantees be unified?	Several constraints can be expressed in a joint behaviour–cost semantics, but time, memory, energy and information flow need different analysis models; joint expression is not unified solving
Q5 How can trust in the generative model be reduced?	The generator proposes candidates and an independent checker verifies them. This reduces dependence on the generator's correctness but does not remove the requirement to trust the logical kernel, the theorem statements, the external assumptions and the correspondence of artefacts
Q6 Can a given specification always be synthesised effectively?	No. Finite grammars, restricted logics and specific templates make some problems solvable; for arbitrary program semantic properties there are boundaries of undecidability and search-space explosion

Question	**This paper's judgement**
Q7 Can actual latency be strictly guaranteed?	A usable worst-case execution time bound is obtainable under specific assumptions about hardware, inputs, scheduling and the cost model; benchmarks and asymptotic complexity are not by themselves sufficient to prove a bound at the nanosecond scale
Q8 Is a unified theory needed?	The better-founded choice is to establish shared proof interfaces and evidence formats while retaining the different semantics, assumptions and analysis tools of each domain. A unified formalism may be one goal, but it is not a necessary precondition

Remark 7.36 (Summary of conclusions on the central questions Q1–Q8).

The empirical content of this chapter is directions 1–3 (Chapter 9) and hypotheses H_7, H_8 and H_{11}. Propositions 7.11 and 7.12 are arguments rather than theorems, and what can be challenged in them is whether the three grounds are exhaustive. Theorem 7.27 and Proposition 7.28 are applications of standard probability theory, and their conclusions depend on the independence assumption.

Chapter 8

Levels as Units of Verification-Obligation Decomposition

The first seven chapters use levels as a tool for comprehension (Chapter 5), for design (Chapter 6) and for assurance (Chapter 7). This chapter shows that the three are three uses of one object, and argues for the direction among them most worth pursuing: promoting levels from an aid to comprehension to the *unit into which verification obligations decompose*.

8.1 The Cognitive Scale

Definition 8.1 (Cognitive scale)**.** Given a study object S and a task T, the **cognitive scale** of S, $\mathrm{L}_0, \mathrm{L}_1, \mathrm{L}_2, \ldots$, is divided by the type of question a level answers rather than by the physical location of the code:

L_0 **Computational principles** What performs the computation? What constrains it? Memory, instruction execution, function calls, concurrency, synchronisation, virtual memory, I/O, caching, networking, persistence, types and the memory model.

L_1 **Infrastructure and runtime** Which shared capabilities does the system provide, and how is their correct cooperation guaranteed? Allocators, task schedulers, event loops, object systems, executors, storage-engine interfaces, resource managers, lifecycle mechanisms. This is the level at which the six functions of Chapter 4 reside.

L_2 **Components and subsystems** Which duties exist? Where are the boundaries? How do they cooperate? The transaction subsystem, the rendering engine, the entity–component system, the network protocol stack, the replication module, the query executor.

L_3 **Domain function and business semantics** What does the system promise externally? Which behaviour is correct? Query semantics, game rules, layout rules, transaction isolation semantics, permissions and business invariants.

L_4 **and above: scenario and system-level behaviour** How does the whole system behave in real scenarios? End-to-end workflows, failure recovery, system-level performance, deployment topology, business objectives.

L_∞ **Purpose and responsibility** What should be wanted? Which trade-offs are acceptable? This level belongs to humans; the reason is given in §8.4.

Principle 8.2 (The cognitive scale is not a physical hierarchy)**.** The levels of Definition 8.1 denote *cognitive scales*, not necessarily the physical layers of the code. The L_3 domain function of a game engine may call the L_1 runtime interface directly; the transaction semantics of a database may equally impose constraints on the storage layer.

Principle 8.3 (Relativity and recursiveness of levels)**.** The same component may occupy different levels under different study tasks. An asynchronous runtime is infrastructure (L_1) to an ordinary application developer, and to its own developers a complex system requiring deep modification (it has a complete $L_0 \dots L_4$ of its own). Levels are therefore defined relative to the study object and the current task.

Principle 8.3 has an important corollary: *every large system contains a complete level ladder internally*, and this embedded ladder is a distinct thing from the external technology-stack ladder. This explains directly the conclusion of Chapter 4: every large system grows a "small operating system" inside its own domain, and that is precisely its embedded L_1.

Remark 8.4 (Difference from the causal hierarchy). The levels of this paper resemble the causal hierarchy [66] in form: an intervention at a lower level does not automatically transmit to a higher level, just as a modification of the implementation layer does not automatically become a modification of the contract layer. The two differ, however. The causal hierarchy distinguishes *types of query* (observation, intervention, counterfactual); the levels of this paper distinguish *observable domain and guarantee obligation* (Definition 2.1). Conflating them yields a concrete error: the belief that being able to ask a question at a higher level amounts to being able to intervene at that level, when the latter requires an interface the former does not provide.

8.2 The Level Contract

Definition 8.5 (The level-contract quadruple)**.** A level is recognised only if one can write down the quadruple

$$\mathrm{LC}(\Lambda) \;=\; \big(\text{participants},\ \text{guaranteed invariants},\ \text{contract form},\ \text{verification method}\big), \quad (8.1)$$

together with the fifth item this paper additionally requires, the **oblivious set** $\mathrm{Obl}(\Lambda)$ (Definition 2.7), that is, what the level is permitted never to care about.

Principle 8.6 (Criterion of level reality)**.** A level is real if and only if the five items of Definition 8.5 can be written down. A "level" whose fifth item (the oblivious set) cannot be written down is not a level but a directory or a namespace.

Applying Definition 8.5 to the instances of Table 8.1 shows the concrete form taken by one and the same set of questions at different levels.

The most informative column of Table 8.1 is the last. It makes explicit the other half of "what this level is responsible for", namely "what this level is permitted not to know". That column is the one most often missing from the design documents of large systems, and the most valuable.

8.3 Decomposition of Verification Obligations

This is the central claim of the chapter.

Principle 8.7 (The three identities of a level)**.** One and the same level structure carries three functions at once:

(i) **unit of comprehension**: it fixes the lower bound on "how much information is needed to understand a system", namely the composite of the level quotients (Proposition 2.8);

(ii) **unit of design**: it fixes the structure of "the read-set of a typical change" (Theorem 6.3);

Table 8.1: Instantiation of the level-contract quadruple on a relational database engine

Level	Participants	Guaranteed invariants	Contract form	May ignore
L_0	pages, cache lines, instructions	the locality principle; atomicity of a cache line	hardware timing and the memory model	upper-level semantics; the notion of a transaction does not exist
L_1	buffer pages, log records, locks, memory contexts	a dirty page is not reused before it is flushed; the log reaches disk before the data	the `palloc` contract, the buffer pin/release protocol	SQL semantics; query-plan shape
L_2	parser, planner, executor, access methods	the tuples consumed by the executor satisfy the visibility of the current snapshot	the executor interface (open/next/close)	the physical layout of storage; the log format
L_3	transactions, statements, sessions	the visibility and atomicity declared by the isolation level	SQL semantics and the definition of isolation levels	buffer-management policy; index structures
L_4	applications, operations, clients	an end-to-end workflow remains recoverable under failure	service-level objectives and recovery-point objectives	the shape of the internal execution plan

(iii) **unit of verification obligation**: it fixes *which properties can be assured mechanically, which must be monitored at runtime, and which must be left to the human division of responsibility.*

Item (iii) is the part that existing discussions touch on but do not develop.

A level is naturally a unit of verification-obligation decomposition because it supplies two things at once: an *observable domain* ($\mathcal{O}_i$, what the level can see) and an *invariant set* ($\mathcal{I}_i$, what the level must guarantee). The form of a verification obligation is precisely to decide a property over some observation domain, so every verification obligation attaches naturally to some level.

Definition 8.8 (Level attribution of a verification obligation)**.** Given a property φ and a level Λ, φ is **attributed to** Λ if φ can be expressed entirely within the observable domain $\mathcal{O}_\Lambda$ of Λ. The level of attribution of φ is the *lowest* level that can express it.

Three classes of obligation follow:

(a) **Mechanically decidable at the level**: φ is expressible within the observable domain of Λ and admits an exhaustible decision procedure. Assigned to mechanical assurance.

(b) **Monitorable at the level but undecidable**: φ is expressible within the observable domain but can be decided only at execution time, over traces already observed. Assigned to runtime assurance, with the three states distinguished by Principle 7.18.

(c) **Not expressible at the level**: φ requires cross-level information or concerns purpose itself. Assigned to human attestation or consensus assurance.

Table 8.2 concretises the claim of this chapter. For a property, three questions are asked in order: to which level does it belong; which assurance can that level provide; and what are the assumptions of that assurance. These three questions replace questions that admit no answer, such as “do we need formal verification”.

8.4 The Bearer of Responsibility

Principle 8.9 (Responsibility cannot be transferred)**.** Deciding what is wanted and bearing the consequences of that decision are two facets of one act of agency. Hence L_∞ does not belong

Table 8.2: Distribution of assurance types across levels (example)

Property	Level of attribution	Assurance mode	Evidence type
memory safety	L_0–L_1	mechanical (type system / ownership / sandboxing)	compile-time checking, verifier
no out-of-bounds access and bounded loops	L_0–L_1	mechanical (bytecode verifier)	static proof at admission
no unauthorised I/O	L_1–L_2	mechanical (capability model / effect types)	compile-time + runtime boundary
functional postconditions	L_2–L_3	mechanical (Hoare / refinement types / contracts)	proof terms, SMT decision
resource bounds	L_1–L_2	mechanical (resource analysis) + environmental assumptions	symbolic bounds, worst-case execution time analysis
real-time deadlines	L_2–L_4	mechanical (within the hardware model) + runtime	timing analysis, monitoring
isolation-level semantics	L_3	mechanical (model checking / contracts)	model-checking results
end-to-end availability	L_4	monitoring + consensus	operational metrics, service-level objectives
the specification is faithful to the intent	L_∞	consensus	adversarial review, attestation records

to machines, not because machines are not intelligent enough, but because responsibility must be borne by a subject capable of bearing consequences.

This principle must be made precise by the results of §7.11. The residual responsibility of humans is *not individual but collective*: not "one person reviews the specification" but "a cognitive network reaches consensus on the specification through adversarial debate". That network may include human experts of different backgrounds, models of different training provenance, and explicitly designed adversarial roles.

Principle 8.10 (Diversity is irreplaceable)**.** What makes humans irreplaceable in the age of AI is not that individual human judgement is better than that of an AI, but that humans are an irreducible *source of diversity* in the cognitive network. By Corollary 7.29, the reliability of consensus is governed by the independence of error modes, and the ultimate source of independence is *different histories of problem formation*. The value of humans therefore lies in the diversity of their perspectives, not in the precision of their computation.

8.5 Level-Language Mismatch

Principle 8.11 (Consistency of level language)**.** The gravest error in reading code is to understand the current level in the language of the wrong level.

Example 8.12 (Concrete forms)**.** (a) Understanding business validation logic by pointer analysis, treating "is this transaction permitted" as a question of memory aliasing.

(b) Understanding buffer management in business semantics, looking for commercial rules in a page-replacement policy.

(c) Understanding database isolation levels in the language of scheduler fairness: both are concurrency problems, but the correctness criterion of the former is a throughput and latency distribution, and that of the latter is visibility and atomicity.

(d) Solving multi-tenant isolation with a type system, because isolation is a cross-cutting concern (Principle 6.20) and a type system can express only part of it.

The antipattern of Example 8.12(d) has a more serious form: it is not merely a wrong language but an attempt to press a cross-cutting concern into the level structure. By Principle 6.20, such an attempt is structurally doomed to fail, and its failure is often misdiagnosed as "layering that is not thorough enough".

The empirical content of this chapter is the reproducibility of Definition 8.8: whether different analysts give the same level attribution to the same property, and whether the assignment of Table 8.2 reduces the omission of properties that should be assured mechanically but are left to manual review. With a falling omission rate as the metric, the claim is testable.

Chapter 9

The Falsifiable Research Programme

Every claim in this paper is no more than an interpretive framework unless it is converted into testable form. This chapter performs that conversion, following the distinction between hard core and protective belt in the methodology of research programmes [65]: claims that are not load-bearing belong in the protective belt, not reinforced into the hard core.

9.1 Methodology: From Interpretive Framework to Testable Hypotheses

Principle 9.1 (Demotion procedure)**.** For every claim, carry out the following operations: write down its *observable consequences*; write down what observation would *refute* it; write down the *minimal experiment* that would yield that observation; and write down the *control condition* of that experiment. Any claim for which these four steps cannot be completed is to be marked explicitly as an interpretive framework rather than an empirical assertion.

By Principle 9.1, the claims of this paper fall into three classes:

- **Proved**: propositions 2.8, 3.5, 2.16, 6.7, 2.4, 5.8, 7.15, 2.25, 4.15, 5.10, 6.14; theorems 4.4, 4.9, 6.3, 7.27; proposition 7.28; corollaries 4.5, 4.11, 7.29, 7.8.
- **Argued under explicit assumptions**: principles 1.5, 1.6, 1.7, 1.8; propositions 7.11, 7.12.
- **Advanced as falsifiable hypotheses**: the hypotheses H_1–H_{12} below. In addition, the structural proposal of Remark 4.17 (promoting the level family to a strict fibration) and the renormalisation analogy of Remark 4.21 are not counted among the theorems; their status is that of a proposal and a heuristic.

9.2 Twelve Falsifiable Hypotheses

Hypothesis 9.2 (Structural convergence above a random baseline)**.** After controlling for system size, domain, platform, programming language and framework, large software systems from different domains exhibit structural similarity on their abstract skeleton (Definition 2.12) significantly above a random baseline. *Operationalisation*: construct a collection of architecture graphs and skeleton maps by the five-step design of §3.6, take the structural metrics of the third step as the feature vector, and test whether the similarity distribution of cross-domain pairs stochastically exceeds a null model, such as degree-sequence-preserving graph rewiring. *Falsification condition*: the similarity distribution does not differ significantly

from the null model. *Risk*: freedom in the choice of skeleton map; the map and the metrics must be pre-registered.

Hypothesis 9.3 (Convergence strength monotone in domain binding)**.** Conjecture 3.10 holds: structural similarity decreases monotonically in the domain binding Dom (Definition 3.9). *Operationalisation*: annotate Dom at each level (estimated from the fraction of domain axioms on which that level's semantics depends) and regress similarity on Dom. *Falsification condition*: the monotonicity fails, or the coefficient is not significant.

Hypothesis 9.4 (Phase-transition threshold determined by $n \times m$)**.** Whether a system builds a unified substrate is predicted better by the product of the number of live entities and the number of shared resources (together with the frequency of recurrent problems and the frequency of evolution) than by lines of code. *Operationalisation*: for a set of open-source projects, annotate manually or semi-automatically whether a self-built implementation of any of the six functions appears; fit separate logistic regressions on a proxy for $n \times m$ (order of magnitude of concurrent entities, number of kinds of shared resource) and on lines of code; compare their explanatory power. *Falsification condition*: lines of code explain the outcome no worse than $n \times m$. *Risk*: automatic estimation of n and m is difficult; the number of concurrent entities and of mutexes, connection pools, caches and the like can serve as a first proxy.

Hypothesis 9.5 (Read-set scaling outperforms traditional size metrics)**.** The change read-set $\mathbb{E}[\rho]$ (Definition 6.1) and its *scaling exponent* α with respect to system size (Proposition 6.14) correlate with defect rate and change failure rate better than traditional size metrics such as module count, lines of code and cyclomatic complexity, and better than the current magnitude of the read-set $\mathbb{E}[\rho]$ itself. *Operationalisation*: on several open-source projects with a complete commit history and defect tracking, estimate the magnitude and the scaling of the read-set over the same time windows, regress each in turn on defect density in a panel design, and compare their explanatory power. *Falsification condition*: the scaling provides no predictive power beyond the magnitude and the existing metrics.

Hypothesis 9.6 (Layered model plus active probing outperforms exhaustive reading)**.** Under a constraint on unit cost (token budget or person-hours), a comprehension process that uses a layered cognitive model plus active probing attains a higher task success rate (scored against (U1)–(U5) of Definition 5.3) than exhaustive reading. *Operationalisation*: given the same unfamiliar code base and the same set of tasks (predict the impact surface, locate a defect, add a feature), compare three strategies: sequential reading, layered summaries only, and layered summaries plus a probing loop. Measure task success rate and success rate per unit cost. *Falsification condition*: the active-probing strategy is not significantly better, or its advantage comes entirely from the number of probes rather than from the structure of the strategy.

Hypothesis 9.7 (Leak pairs are automatically discoverable)**.** The leak pairs of Proposition 6.7 can be found by automated differential testing, and the discovery rate is negatively correlated with module boundary quality. *Operationalisation*: for several libraries, generate execution pairs that are "identical at the interface and different in internal state", detect divergence in external behaviour, and compare against human-annotated boundary quality scores. *Falsification condition*: automatically generated execution pairs fail to trigger leaks effectively, or the discovery rate is unrelated to boundary quality.

Hypothesis 9.8 (Specification quality is the present bottleneck)**.** At the present state of the art, the principal bottleneck on the path from intent to verifiable execution is specification quality, not code generation capability or proof search capability. *Operationalisation*:

take existing benchmark evidence (specification generation tasks, evaluations that separate code correctness from specification quality, proof success rates) as initial evidence, and on a new controlled benchmark compare the marginal contribution of each of the three links to end-to-end success rate. *Initial evidence*: a component-wise evaluation on 189 Lean tasks reports, for the best model, code correctness 72.6%, specification soundness and completeness 52.3%, and proof success rate 4.9% (single trial) [8]; an evaluation of 245 operating-system kernel specification generation tasks shows that models remain deficient in long context and specification completeness [3]; an evaluation that instantiates specifications as trusted tests points out that proof difficulty interferes with the assessment of specification quality [4]; a traceable refinement method reports accuracy gains on the specification-generation subtask [5]; a mutation-driven method reports precision and recall [6]. These are empirical results on specific benchmarks, not general reliability theorems, but they support the judgement that the three dimensions should be evaluated separately. *Falsification condition*: with the capability of the other links controlled, the marginal contribution of specification quality is not significantly greater than that of the other links.

Hypothesis 9.9 (Consensus reliability is dominated by correlated error modes)**.** On intent-attestation tasks, the reliability gain from adding verifiers is bounded above by the correlated error mode q (Proposition 7.28); introducing heterogeneous verifiers (people of different backgrounds, models from different sources, adversarial roles) improves reliability more than merely adding homogeneous verifiers. *Operationalisation*: construct intent-attestation tasks, configure several verifier groups (homogeneous or heterogeneous), measure how the error rate of the majority judgement varies with group size and heterogeneity, and compare with the predictions of Equations (7.1) and (7.2). *Falsification condition*: the rate at which the error rate of homogeneous verifiers decreases with n does not differ significantly from that of heterogeneous verifiers (that is, q is in fact small).

Hypothesis 9.10 (The artefact binding gap is a measurable risk)**.** The binding gap (Equation (2.12)) between the verified artefact and the artefact actually executed is a measurable risk in real deployments and can be significantly reduced, and reducing it yields an observable reliability benefit. *Operationalisation*: introduce content addressing, reproducible builds and deploy-time verification into several deployment pipelines, and measure the change in the proportion of build nondeterminism, the rate of deployment drift events and the rate of related incidents. *Falsification condition*: the gap is not measurable in practice, or the cost of closing it far exceeds the benefit.

Hypothesis 9.11 (The six functions are the surviving set under coarse-graining)**.** Under the coarse-graining sequence of Definition 4.18, the structures that survive are exactly the slots of $\mathcal{K}_6$ and their relational graph (Conjecture 4.19). *Operationalisation*: apply mechanical coarse-graining to several repositories following Principle 4.20, and record how the surviving set changes with granularity. *Falsification condition*: the surviving set contains structures outside the six functions, or some of the six functions are washed out already at intermediate granularity. *Risk*: the choice of coarse-graining threshold introduces an arbitrary parameter; the threshold should be subjected to a sensitivity analysis.

Hypothesis 9.12 (Incentive structure reduces correlated error)**.** On intent-attestation tasks, adopting an appropriate scoring rule (Principle 7.32) reduces the error rate of the majority judgement more than merely increasing the number of verifiers, and a measurable fall in error correlation accompanies it. *Operationalisation*: on the same set of specification-attestation tasks, compare three configurations: few unincentivised verifiers, many unincentivised verifiers, and few verifiers under a scoring rule; measure the error rate and the correlation coefficient of errors between verifiers. *Falsification condition*: neither the error rate nor the

error correlation coefficient of the scoring-rule configuration is better than that of the many-unincentivised configuration.

Hypothesis 9.13 (Rate–distortion slope predicts refactoring benefit)**.** The slope $\partial L_T/\partial \ell$ of the rate–distortion curve (Remark 6.12) predicts which refactorings reduce defect rate or rework rate in practice. *Operationalisation*: for a set of completed refactorings, estimate ex ante the direction of displacement along the curve and the slope, and ex post regress the changes in defect rate and change failure rate on them. *Falsification condition*: the slope provides no predictive power beyond naive variables such as the size of the change.

Table 9.1: Summary of hypotheses and experimental designs

ID	Hypothesis	Principal measurement	Principal risk
H_1	Structural convergence above a random baseline	Skeleton similarity vs. null model	Excessive freedom in the choice of skeleton map
H_2	Convergence strength monotone in domain binding	Regression of similarity on Dom	Dom is hard to annotate independently
H_3	Threshold determined by $n \times m$	Comparison of logistic-regression explanatory power	Automatic estimation of n and m
H_4	Read-set scaling outperforms traditional size metrics	Panel regression of read-set scaling on defect density	Accuracy of the read-set proxy
H_5	Layered model plus active probing outperforms exhaustive reading	Task success rate per unit cost	Subjectivity of tasks and scoring
H_6	Leak pairs are automatically discoverable	Differential-testing discovery rate	Difficulty of triggering valid execution pairs
H_7	Specification quality is the bottleneck	Marginal contribution of the three links	Ground truth and coverage of the benchmark
H_8	Consensus dominated by q	Error rates of homogeneous and heterogeneous verifiers	Construction and measurement of heterogeneity
H_9	Artefact binding gap is measurable	Build nondeterminism and drift event rate	Availability of real deployments
H_{10}	The six functions are the coarse-graining surviving set	Surviving set under the coarse-graining sequence	Arbitrariness of the coarse-graining threshold
H_{11}	Incentive structure reduces correlated error	Error rate and error correlation coefficient	Experimental ethics and cost of incentive schemes
H_{12}	Rate–distortion slope predicts refactoring benefit	Regression of defect rate before and after refactoring	Estimability of the slope

9.3 Three Priority Research Directions

The three directions below connect requirements engineering with formal methods, compilation correctness with resource analysis, and program contracts with runtime security. They share one characteristic: each has a precisely stated question, a falsifiable hypothesis, a method of testing and a principal risk.

Direction One: Specification-Quality Evidence Chain for Real Intent

The precise question. Given a set R of natural-language requirements, a domain model D and a generated specification S, how is an independent evaluation process to be built

that checks separately: requirement coverage; exclusion of forbidden behaviour; retention of permitted behaviour; explicitness of environmental assumptions; and the satisfiability and implementability of the specification?

Existing foundations. Requirements traceability research, requirement formalisation and verification, vacuity detection, and recent specification-generation and joint-evaluation benchmarks. The key connection is *semantic traceability* in requirements engineering and *specification-versus-program counterexamples* in formal methods.

Falsifiable hypothesis. Compared with a single "proof passed" signal or a single "similarity to a reference specification" score, atomic requirement traceability plus positive and negative testing plus specification-strength and mutation checks plus independent domain review identifies the *under-constraint*, *over-constraint* and *intent omission* of a specification earlier, that is, it is the concretisation of Hypothesis H_7.

Method of testing. On small benchmarks such as payments, permission control and idempotent interfaces, establish requirement atoms annotated by multiple stakeholders, permitted and forbidden trace sets, and environmental preconditions; compare four approaches: pure generation, requirement decomposition, traceability testing, and mutation refinement with expert assistance. Measure precision and recall, error categories, the rate of missed prohibitions, the over-constraint rate, expert agreement and cost.

Falsification condition. If adding the traceability and mutation layers does not significantly reduce the rate at which hidden negative requirements are missed, or if the gain comes only from leaked reference test answers, the hypothesis fails.

Principal risk. The "gold standard" for real intent may itself be contested.

Direction Two: Joint Functional–Resource Translation Certificates for Actual Artefacts

The precise question. Given a source program P, a functional contract φ, a resource budget B and a fixed target platform model H, can a certificate C be generated for *each compilation* such that an independent verifier checks semantic preservation and analysable resource constraints at the same time?

Existing foundations. Semantic-preservation proofs for verified compilers, per-translation validation, translation certification, proof-carrying code, automatic amortised resource analysis and worst-case execution time analysis. The substantive connection is to place the "behaviour relation" and the "cost relation", currently verified *separately*, inside a *cost-carrying behaviour semantics*.

Working model. The program semantics is extended to a pair (τ, c), where τ is an observable trace and c a cost; the translation proof should guarantee that target traces are explicable by the source semantics, and should prove $c_{\text{target}} \leq B$ under an explicit cost model, or else give a conservative relation relative to the source cost.

Remark 9.14 (An assumption that must be avoided). It must not be presupposed that optimised compilation always lowers cost monotonically. Optimisation may improve average

performance while worsening the worst case; the cost relation must therefore be a conservative relation comparable in both directions, not an assumption of monotonicity.

Method of testing. Choose a reproducible sublanguage and target architecture, and compare three routes: verified compilation, translation validation, and ordinary compilation with post-hoc verification. Record certificate size, verification time, coverage of program categories, tightness of resource bounds, false-rejection rate, and robustness to changes in compiler optimisation.

Falsification condition. If the resource proofs do not remain sound once calibrated against a real platform, or if certificate generation and verification cost far more than the baseline with no safety benefit, the scope should be narrowed to specific safety-critical domains.

Principal risk. The target processor microarchitecture and scheduling model are not precise enough.

Direction Three: Composition Contracts and Runtime Assurance for Untrusted Components

The precise question. When a service is composed of verified components, generated components that are not fully verified, and external services, how are assumption–guarantee contracts, information-flow policies, resource budgets and runtime monitoring to be used to give an auditable overall safety proposition together with an upper bound on failure impact?

Existing foundations. Modular assumption–guarantee reasoning, contract composition for dynamical systems, information-flow non-interference, verified kernels, and studies of uncertainty in runtime verification. The substantive connection is to place the formal contracts of subcomponents and the runtime detection and enforcement of contract premises and guarantees within a single system argument.

Falsifiable hypothesis. When every component has explicit assumptions, entry and exit events are observable, and contract violations can trigger isolation or rollback from an independent monitor, a composable conditional guarantee can be given for a specified safety property without proving the internal implementation of each component correct.

Method of testing. Build a multi-component payment flow: an idempotent request service, an identity and permission service, a generative risk-decision component, an external database and a network simulator. Under fault injection, compare three groups (no contracts, static contracts, contracts plus monitoring and rollback) and measure duplicate charges, cross-user data leakage, detection latency, isolation success rate and performance cost.

Falsification condition. If the events observable by the monitor are insufficient to decide the critical properties, or if the rollback mechanism cannot itself remain safe, the architecture cannot provide the guarantee it claims.

Principal risk. For subtle semantic properties that go unobserved, runtime assurance can constrain only the detectable and enforceable part.

9.4 Six-Stage Implementation Roadmap

Table 9.2: Research implementation roadmap

Stage	Core tasks	Deliverables
1	Define the benchmark and the threat model: choose a payment service and an idempotent interface as the running case; fix the assets, roles, observable events, forbidden behaviour, environmental assumptions and time limits	Set of requirement atoms, positive and negative trace sets, list of environmental assumptions (attested by stakeholders)
2	Specification quality assessment: compare pure generation, traceability refinement, and mutation- and counterexample-driven approaches; check for under-constrained specifications, over-constrained specifications, vacuous satisfaction, unsatisfiability and unimplementability	Specification quality assessor; for each requirement identifier, the corresponding specification clause, test and attestation record
3	Program synthesis in a restricted language: choose an explicit sublanguage and a logic of controlled decidability; separate candidate generation from proof checking	Candidate programs, proof obligations, counterexample classification; distinguish unsatisfiable, unknown, timeout and inexpressible
4	Artefact and resource proofs: choose a mature compilation or translation-validation route; specify reproducible experimental hardware or a simulation model	Compilation-chain metadata, semantic-preservation or translation certificates, resource bounds and model boundaries
5	Composition and runtime assurance: write assumption–guarantee contracts for external services and not-fully-verified modules; add independent monitoring, sandboxing, auditing and safe rollback	System-level contract composition results, fault-injection report, residual risk list
6	End-to-end evaluation: measure assurance coverage, false negatives, false positives, development cost, proof cost, artefact verification cost, resource budgets and failure recovery	A public benchmark capable of falsifying the research hypotheses, a tool prototype and an experimental report

9.5 Success Criteria

The criteria below assess whether the roadmap succeeds. They are designed to be independently checkable:

(S1) **Explicit object**: the object of each guarantee (which artefact, which semantics, which environment) can be uniquely identified.

(S2) **Real assumptions**: the environmental assumptions relied upon can be tested, not assumed by default.

(S3) **Independent evidence**: the judgement of evidence does not depend on the generator's self-assessment.

(S4) **Distinguishable failure categories**: unsatisfiable, unknown, timeout, inexpressible, and candidate refuted by counterexample must be reported separately and not merged into a single "failure".

(S5) **Measurable cost**: generation cost, proof cost, verification cost and runtime cost can be measured separately.

(S6) **Falsifiability**: the refutation of at least one H_i would weaken the core claims of the programme.

Remark 9.15 (Self-examination of the programme). One core claim of this paper is that "generation is untrusted and verification must be independent". The principle applies to the paper itself: its load-bearing propositions must be independently checkable and must not depend on any unproved conjecture. Remark 7.36 and the list of non-claims in Chapter 11 are the concrete results of that self-examination.

Chapter 10

Literature Map and Positioning

This chapter places the paper's claims in their literature context, states their relation to existing theory, and marks where the gaps lie.

10.1 Six Groups of Literature

First Group: Modern Specification Generation and Joint Evaluation. This is the frontier closest to the intent → specification step. The relevant literature reports measurable results: a specification-generation benchmark for operating-system kernel verification tasks still shows model limits on long context and specification completeness [3]; a benchmark that evaluates code correctness, specification quality and proof generation separately reports a significant gap among the three dimensions (on a particular task set, code correctness, specification soundness and completeness, and single-pass proof success rate are 72.6%, 52.3% and 4.9% respectively) [8]; an evaluation framework that instantiates specifications as trusted tests points out that proof difficulty interferes with the assessment of specification quality [4]; a traceable refinement method decomposes natural-language requirements into atomic requirements and establishes requirement-level traceability [5]; a mutation-driven method tests specification strength with semantically non-equivalent variants [6]; and the route of generating specifications from code should be studied separately from the route of generating specifications from intent [7]. Vacuity detection [9] supplies a theoretical route to identifying the case in which verification passes while the specification holds vacuously. Together these results support one important judgement: *code generation ability, specification expressiveness and formal proof ability are distinct research variables and should be evaluated separately.*

Second Group: Program Semantics, Refinement and Observable Behaviour. Operational semantics, denotational semantics, the lattice model of abstract interpretation [20], monadic semantics [17, 18], domain theory, refinement relations [16] and separation logic [21] together supply the mathematical tools for the question of when two systems at different levels implement the same behavioural specification. Semantic refinement is not simple information compression: it must define the *behavioural relation* between levels.

Third Group: Type Theory, Constructive Mathematics and Proof Extraction. Constructive type theory [19], the correspondence between proofs and programs [47], and the mechanisms that connect proof objects with constructive computation supply the mathematical foundation for "an AI proposes candidate proofs and a mechanical kernel checks them". One inference must be avoided: *if a specification has already been translated into a type, then the specification necessarily expresses human intent faithfully.* The correspondence guarantees

only the stipulated structural correspondence among propositions, types, proofs and terms within a chosen logic and type system; how human intent is turned into a proposition is a separate process that requires verification. Refinement types [44] and SMT solving [45] are the engineering representatives of this direction; automated theorem-proving environments and retrieval-augmented proof search [13, 14] are its continuation in the age of AI.

Fourth Group: From Proof to Machine Code. This is the most mature group. A verified compiler turns the compilation transformation itself into a proved object [22]; the back ends of a verified compiler cover several target architectures [24]; a refinement proof for an operating-system kernel shows how far the formal verification of large-scale low-level system software can reach [25]; translation validation checks a *single* compilation result independently [23]; translation certification establishes a verifiable relational certificate for a *specific artefact* [26]; and proof-carrying code lets a code provider attach a proof that a machine can check independently [15].

The key lesson of this group is that *verifying the correctness of a compiler and verifying the correctness of a particular binary are two different strategies.* The former proves that the compiler satisfies a universal property; the latter verifies one build output independently. The two can be combined, without staking all trust on a single tool.

Fifth Group: Resource Correctness. Automatic amortised resource analysis can derive symbolic resource bounds for a program under a particular language and cost model, with a soundness proof; worst-case execution time analysis derives timing bounds for particular hardware and assumptions; static cache analysis shows the influence of the hardware model on bound inference. Three notions of time must be distinguished: algorithmic complexity gives a bound on the growth of cost as input size varies; worst-case execution time analysis gives an upper bound under a particular platform and assumptions; measurement provides latency data under particular experimental conditions. *None of the three substitutes for another.* Equally, linear use is not optimal use: a program may consume each input resource exactly once and still use a poor algorithm or introduce excessive latency.

Sixth Group: Runtime Verification, Contract Composition and the Emergence of Levels. This group contains two classes of material of different kinds, which must be treated separately.

The first class is *runtime assurance and contracts*: assume–guarantee reasoning requires a subsystem to give the corresponding guarantee when its input assumption is satisfied [35]; contract composition for dynamical systems gives composition rules and checking methods under arbitrary interconnection topologies [36, 37, 38]; work on the uncertainty of runtime verification shows that observation, event transport and trace integrity are components of monitoring reliability [34]; online monitoring of temporal logic [46] and information-flow security [42] address trace properties and access restrictions respectively.

The second class is *explanatory frameworks*: the near-decomposability of complex systems provides an explanation for level structures [39]; minimum description length supplies the statistical and information-theoretic background for "abstraction is compression" [56, 40, 41]; computational thermodynamics gives a physical lower bound on computation [32]. This group can supply an explanatory framework for architecture, compression, model selection and complex systems, but it *cannot automatically supply a correctness proof for a particular program.*

10.2 The Five Layers Rearranged by Mathematical Function

Rearranged by which segment of the chain it solves, this literature yields a structure deeper than "an AI writes code and a verifier checks it":

Table 10.1: The five layers rearranged by mathematical function

Layer	Function	Representative lines of work
A	Expressibility of the objective: formal specification generation, intent traceability, requirement decomposition, counterexample generation	Specification-generation benchmarks, traceable refinement, mutation-driven strength testing, vacuity detection, requirement formalisation
B	Realisability of the objective: logical consistency, program synthesis, constraint solving, realisability decision	Counterexample-guided inductive synthesis, syntax-guided synthesis, program-skeleton completion, constructive type theory, SMT solving
C	Cross-layer semantic preservation: operational semantics, refinement relations, proof checking, compiler correctness, binary verification	Verified compilers, verified compiler back ends, kernel refinement proofs, proof-carrying code, translation validation
D	Resource and environment assurance: complexity, resource bounds, runtime monitoring, assume–guarantee contracts, execution constraints	Automatic amortised resource analysis, worst-case execution time analysis, runtime verification, contract composition, information-flow security
E	Structure and physical limits: model compression, levels of complex systems, thermodynamics and physical realisation	Minimum description length, near-decomposability of complex systems, computational thermodynamics, molecular self-assembly

The most important finding is that these areas *do not share one ready-made, fully unified mathematical object*. Each has nonetheless solved a different problem along this chain. The research opportunity therefore lies in defining the composition relations among them, not in inventing the entire mathematical toolkit from scratch.

10.3 The Unoccupied Intersection

Principle 10.1 (Shape of the literature map)**.** Every step from specification to binary has a mature literature; automated synthesis from proof to implementation has an active frontier; but the gap from *intent* to *specification* has no formal solution, only consensus structures, and the literatures of social epistemology, Byzantine fault tolerance and adversarial review exist in entirely different disciplines and have not been systematically coupled with formal verification systems.

On these grounds, the most valuable position to occupy is the following.

Principle 10.2 (Proposed research position)**.** Integrate adversarial consensus in the style of social epistemology with mechanical admission in the style of proof-carrying code inside one framework: let a consensus-attested specification be the direct input to a proof-carrying artefact, and anchor consensus records, specification versions and artefact identity together cryptographically. As far as the review conducted for this paper extends, few works address both literatures at once.

This position is valuable because it satisfies three conditions at once: it corresponds to a real and still unsolved technical gap (no mechanical solution at the intent layer); both of its components already have mature theoretical foundations; and its composition rules can be written out explicitly (Definition 7.33 and Theorem 7.34) and can therefore be tested.

10.4 Relation to Existing Theory

Table 10.2: Relation of this paper to existing theory

Theory	How this paper uses it, and how it does not use it
Near-decomposability of complex systems	*Used*: as the explanation of why level structures are stable, and recast into a testable sufficient-statistic condition (Definition 6.6, Proposition 6.7). *Not used*: as evidence that "all complex systems necessarily have the same levels"; the source work does not prove this, nor does it give a general theorem about verification cost.
Abstract interpretation and Galois connections	*Used*: as the formal tool for semantic preservation between levels (Definition 2.2). *Not used*: no claim that all inter-level relations can be expressed as Galois connections; where they cannot, the paper falls back on simulation relations or on an explicit human argument.
Minimum description length	*Used*: as the formal background for "comprehension is compression" and the source of the stopping rule for descent (Definition 2.15, Proposition 2.16). *Not used*: no claim that the shortest description is the most correct description. MDL is a model-selection principle; its conclusions depend on the chosen model family, encoding and data, and it cannot by itself determine what purpose a system should achieve.
Information hiding and deep modules	*Used*: as the normative basis for module boundaries. *Not used* as it stands: the paper adds a testable version of it, namely whether an interface is a sufficient statistic of behaviour (Definition 6.6), because how well information is hidden is not itself measurable.
Correspondence between organisational structure and system structure	*Used*: as one of the mechanisms of high-level convergence and one of the causes of divergence. *Not used*: not as a technical necessity; it is a source of constraints, not a source of uniqueness for solutions.
Program logic and refinement	*Used*: as the mathematical interface for the specification–implementation relation. *Not used*: no claim that safety refinement automatically guarantees liveness, timing and fairness properties.
Proof-carrying code	*Used*: as the classic implementation of a trust structure in which generation is free and admission is strict, and as the predecessor of the fourth pillar of this paper. *Not used*: its premise is not ignored: the receiver must fix a safety policy in advance, and the correctness of the policy itself is an external problem.

Theory	How this paper uses it, and how it does not use it
Assume–guarantee reasoning	*Used*: as the source of the rules by which local guarantees compose into a global guarantee (Proposition 2.4). *Not used*: its four composition conditions are not ignored; a component's being "correct in isolation" and a system's being "correct as a whole" are not interchangeable.
Resource analysis	*Used*: as evidence that resource correctness can enter formal analysis. *Not used*: a symbolic bound is not equated with a time bound on a particular processor; complexity order, worst-case execution time, statistical quantiles and measurement are not conflated.
Computational thermodynamics	*Used*: as the basis for the boundary condition that the physical cost of computation is incompressible. *Not used*: it is not used to predict the power consumption of a piece of ordinary code, nor folded into semantic correctness. (The paper mentions it only in Chapter 11, and not as a load-bearing structure.)
The halting problem and Rice's theorem	*Used*: as the boundary at which a general synthesis algorithm and a general decision algorithm cannot exist. *Not used*: they are not used to prove that "intent can never be formalised" (Proposition 7.11).
P and NP	*Used*: only the structural asymmetry entailed by the *definition* of the class NP (Proposition 7.15). *Not used*: no open assertion of "P $\neq$ NP" or "P = NP" serves as a load-bearing structure.

Principle 10.3 (Positioning of this paper)**.** The contribution of this paper lies at the *interface layer*, not in a new semantic theory: it promotes levels from a comprehension aid to the unit into which verification obligations decompose; it formalises oblivious sets, near-decomposability and read-sets as computable or measurable objects; it attributes the necessity of internal infrastructuralisation to the completeness of the operational semantics rather than to code volume; and it writes the trust structure as the composite of the two modes of trust generation and gives a bound on consensus reliability. These interfaces connect existing theory rather than replace it.

Chapter 11

Objections and Limits

This chapter performs a self-audit: it removes claims that cannot bear load from the structure, and records explicitly what has been removed.

11.1 List of Non-Claims

The following list is binding: no other part of the paper may be read as supporting the claims below.

(N1) The paper does *not* claim that all large software eventually converges on one and the same high-level architecture (Hypothesis 3.3 is explicitly rejected).

(N2) The paper does *not* claim that code volume by itself causes internal infrastructuralisation; it claims that the cause is the product of entity count and resource count, together with the recurrence of problems and the frequency of evolution.

(N3) The paper does *not* claim that the six functions must be *built in-house*; it claims that they must be *defined*, and may be adopted from a host.

(N4) The paper does *not* claim that Simon's near-decomposability is a universal law. The paper recasts it as a conditional, testable sufficient-statistic criterion (Definition 6.6) and concedes that the original source provides no general theorem about verification cost.

(N5) The paper does *not* claim that minimum description length can decide what purpose a system ought to achieve. MDL is a model-selection principle, not a theory of intent.

(N6) The paper does *not* claim that "comprehension is compression" is a theorem in every domain. Within inductive inference it is a defensible theoretical position; outside that range it is a strong heuristic.

(N7) The paper does *not* claim that a mechanical system never errs. It claims that mechanical decision does not depend on the generator's subjective will, and that a mechanical system can still be defective, can fail, and can embody an incomplete model.

(N8) The paper does *not* claim that trust can be fully mechanised. The TCB can be reduced, verified and isolated, but it cannot be eliminated outright.

(N9) The paper does *not* claim that the intent layer can be closed mechanically. It can be *approached* by adversarial consensus, and the reliability of that approximation is governed by the correlated error mode.

(N10) The paper does *not* claim that "adding more agents to review" raises reliability. By Corollary 7.29, reliability rises only when the correlated error mode falls.

(N11) The paper does *not* claim that layering can accommodate everything. A cross-cutting concern cannot be localised structurally into any single layer (Principle 6.20).

(N12) The paper does *not* claim that formal verification will become cheap *automatically* in

the AI era. That is a hypothesis worth investigating, not an established fact (§7.6).

(N13) The paper does *not* claim that compiler correctness equals system correctness. Linking, loading, the runtime, external libraries, hardware and the operating system need not lie within the scope of the proof.

(N14) The paper does *not* claim that resource guarantees can be unified into a single method. Time complexity, symbolic resource bounds, worst-case execution time, statistical quantiles and direct measurement are evidence of different grades.

(N15) The paper does *not* claim that "no violation detected" equals "every execution satisfies". Three states must be distinguished (Principle 7.18).

11.2 Arguments Removed from the Load-Bearing Structure

The paper deliberately rejects several arguments that are common in discussions of this kind but do not hold. They are rejected not because they are useless, but because they state heuristics as theorems, and thereby make the whole framework depend on unproved conjectures.

Table 11.1: Arguments removed from the load-bearing structure

Rejected argument	**Why it fails**	**Replacement in this paper**
Deriving from Gödel's incompleteness theorem that "a permanent gap between intent and specification necessarily exists"	The incompleteness theorem concerns limits of provability in particular formal systems; it is not a theorem about the relation between human mental intent and formal specification	Replace it with three epistemic grounds: requirements are generative; specification is lossy compression; and the distinction between verification and validation (Proposition 7.12)
Deriving from Rice's theorem that "intent cannot be formalised"	Rice's theorem concerns the undecidability of non-trivial semantic properties of programs; it has nothing to do with human intent	Use it only as a boundary on general synthesis and general decision algorithms (Q6 of Remark 7.36)
Calling "grading is easier than answering" a dividend of P $\neq$ NP	Whether P equals NP is still open; the argument depends on an unproved conjecture	Use only the structural asymmetry implied by the definition of the class NP (Proposition 7.15)
Asserting as a mathematical theorem that "the verifier must be minimal, deterministic and never isomorphic"	A small trusted kernel is a valuable architectural principle; small does not mean free of defects	TCB minimisation: make the untrusted part inspectable, and concentrate the unavoidable trust on a basis that is as small, and as explicit in its properties, as possible

Rejected argument	**Why it fails**	**Replacement in this paper**
Asserting that "a dead thing does not betray", hence mechanical assurance is absolutely reliable	A mechanical system can be defective, can fail, and can embody an incomplete model	Distinguish "does not betray" (does not depend on the generator's will) from "cannot err" (impossible)
Asserting that Simon's law of levels proves that every complex system necessarily organises itself into levels	The original source offers an explanation, not a universal law, and no general theorem about verification cost	Recast as a conditional, testable sufficient-statistic criterion (Definition 6.6)
Asserting that linear logic makes wasted resources structurally impossible	The exponential structure of linear logic still permits particular forms of duplication and discarding; linear use is not optimal use	Use a composite structure: usage constraints + resource-bound analysis + performance models + runtime instrumentation + qualified worst-case execution time analysis
Asserting that formal verification must become cheap in the AI era	Historical cost magnitudes show that specification design, model selection and proof architecture remain real work (Example 7.17)	Downgrade it to a hypothesis and give a measurable cost metric (criterion S5 of Chapter 9)
Asserting that "comprehension is compression" is a universal law	Its rigorous formalisation applies to inductive inference, and beyond that range it degenerates into a heuristic	Restrict the scope of application and tie it to task-relative sufficiency (Definition 2.14)
Asserting a substantive connection between this framework and the Langlands programme	That programme studies the functorial duality between arithmetic objects and automorphic representations; the "free generation / deterministic admission" this framework needs is only the *smell of a duality*, not a functor. To write it in would substitute rhetoric for structure	Adopt a duality that pays its way: the LP/SMT duality. Farkas certificates, unsatisfiable cores and unrealisability certificates give a theorem-level correspondence between a synthesis problem and its infeasibility proof, and are already used in engineering
Asserting that the Landauer principle constrains software design	The scale of software is some twelve orders of magnitude away from that bound; the erasure bound is not a constraint surface for architecture	Adopt the memory wall and locality: moving one bit costs about 10^9 times the erasure bound in energy, and this is the physics that actually fixes the memory hierarchy, cache policy and data layout

Rejected argument	Why it fails	Replacement in this paper
Asserting that homotopy type theory is a necessary foundation for this framework	That theory extends the foundations of mathematics, but gives no software guarantee that resource analysis or verified compilation does not already give	Restrict the role of type theory to what has been proved: dependent types expressing postconditions and invariants [19]

11.3 Possible Objections and Responses

Objection 1: read-sets and oblivious sets cannot be measured precisely in real systems, so the formalisation is decorative. Response: the read-set of Definition 6.1 can be counted directly from version history, using "the set of modules touched by each change" as a proxy; it is a computable quantity (Hypothesis H_4 exists precisely to test the validity of that proxy). The oblivious set is not required here to be extracted automatically; it is required to be *explicitly declared*. It is a design artefact, not an analysis artefact. Once declared, it can be checked in part by the leak-pair diagnostic (Corollary 6.8).

Objection 2: the core conclusion that "a large system grows a kernel" is too broad and almost unfalsifiable. Response: Theorem 4.9 has explicit premises (more than one concurrently live entity, finite contended resources, reachable faults), and its conclusion is falsifiable: a well-defined system that satisfies the premises but lacks one of the six functions suffices to refute it. The empirical part rests on Hypothesis H_3, whose falsifier is that "lines of code explain the data no worse than $n \times m$".

Objection 3: the two-layer trust structure places too much responsibility on consensus, and consensus often fails in practice. Response: this is exactly the problem Proposition 7.28 addresses. The proposition does not defend consensus; it gives the form of its failure and its design variables. It also explains a common phenomenon: a "review panel" composed of models of common origin is structurally not a guarantee, however large it is.

Objection 4: does treating the level as the unit into which verification obligations decompose merely rename the existing practice of "layered verification"? Response: in part, yes, and the paper concedes it. The difference lies in three points: (a) the assignment rule given here (Definition 8.8) is an operational decision, not a slogan; (b) the paper explicitly introduces the class of "obligations this layer cannot express" and requires that it be assigned to consensus or to human approval rather than ignored; (c) the paper gives a quantitative bound on the reliability of the consensus layer, which makes "assign it to consensus" a decision that can be assessed rather than merely trusted.

Objection 5: defining technical debt as a description-length gap is uncomputable and therefore useless. Response: the value of the definition lies not in numerical computation but in the direction and the criterion it gives (Principle 6.11(c)): deciding whether a refactoring is worthwhile is equivalent to deciding whether it makes the system easier to generate from a shorter description. That is a question that can be used in review, even though $\ell^\star$ is uncomputable.

Objection 6: the paper mixes engineering, mathematics and social epistemology; does it exceed the defensible range of a single discipline? Response: the criticism holds. Two mitigations are in place. First, every class of claim is labelled with its strength (the strength trichotomy of Chapter 9); second, the citations that are *used* are explicitly separated from those that are *rejected* (Table 10.2). Cross-disciplinarity is not itself a defect, provided that no discipline's conclusion is used outside its scope.

11.4 Open Problems

The following questions are open:

(O1) **Independent measurability of** Dom **and** Conv. For Conjecture 3.10 to become a testable proposition, both quantities must be measurable independently of the conclusion. The paper gives only the direction, not a reproducible annotation protocol.

(O2) **Formalisation of the completeness argument for the six functions**. The proof of Theorem 4.9 is a structural enumeration, not a derivation in a formal system. Writing it as a completeness theorem for a given operational-semantics framework remains unfinished work.

(O3) **Whether cross-layer semantics always compose**. Proposition 2.4 gives sufficient conditions, but whether those conditions can be satisfied simultaneously in systems containing nondeterminism, real-time constraints and reflexivity is not known.

(O4) **How to measure q for the consensus layer**. Proposition 7.28 attributes reliability to the correlated error mode, but gives no operational method for estimating q. This is the crux of whether the direction can be put into practice.

(O5) **Whether a joint semantics of resources and function is decidable**. A costed trace semantics can express joint constraints, but the size of its decidable fragment and the tightness of its bounds are unsettled.

(O6) **Compatibility of artefact binding with dynamically generated implementations**. If an implementation can be regenerated at runtime in response to load, the static identity assertion of Equation (2.12) must be replaced by a stronger question: which changes preserve the existing contract, which require re-verification, and how to fall back to a known safe state when verification fails.

(O7) **Consistency of level assignment**. Whether Definition 8.8 yields consistent assignments across different analysts requires empirical testing.

11.5 Methodological Limitations

(L1) For every reference cited, the depth of checking is limited to the formal publication page, the author's or project's home page, and preprint metadata; the full text was *not read through in full for each entry*. Every concrete number given in the text should therefore be treated as a *second-hand report awaiting re-checking*; several entries with recent nominal years are retained as recorded in the original source, and no independent checking was done here.

(L2) The formalisation in this paper consists of *working definitions established for the purposes of this paper*, and should not be mistaken for an existing unified standard. In particular, the three items "oblivious set", "level contract four-tuple" and "level assignment of verification obligations" are working concepts proposed here.

(L3) The three epistemic grounds (Proposition 7.12) are arguments rather than theorems; the challengeable point is whether the three are exhaustive.

(L4) No experiments were performed. Every hypothesis in Chapter 9 remains in the "to be tested" state.

Principle 11.1 (Self-audit conclusion)**.** After this scrutiny, three minimal independently testable propositions remain in the load-bearing structure:

(i) a structural asymmetry between verification and generation, implied by the definition of the class NP;

(ii) the level is the unit into which assurance obligations decompose, because every obligation belongs to some observable domain;

(iii) the gap between intent and specification has epistemic grounds, and therefore cannot be closed mechanically in general.

Every remaining claim (including "compression", "emergence" and "internal infrastructuralisation") has either been downgraded to a falsifiable hypothesis or confined to an explicit scope of application.

Chapter 12

Conclusion

12.1 Answers to the four questions

Problem 12.1 (Generality of low-level determinism)**.** The answer has three parts. The low level is nearly unique because it is *over-constrained*: the number of constraints is at least the number of design degrees of freedom, and the solution set degenerates into a discrete point set (Proposition 3.5). The upper levels admit many solutions because problem-level constraints are far fewer than the degrees of freedom, and the solution set is a high-dimensional manifold. The upper levels nevertheless converge, because a *second set of constraints*, imposed by the human organisations that build and maintain the system, compresses the manifold into a few sharp peaks (Principle 3.8).

The accurate answer is therefore this: convergence exists, but not towards the engineering optimum; it is convergence towards a *structure that a bounded organisation can navigate*. What converges is carried by *relations* rather than by components (Principle 3.11); its strength decreases monotonically with domain binding (Conjecture 3.10).

Problem 12.2 (Coercion by scale)**.** The conclusion is a *conditional necessity* result. The independent variable that decides whether a unified substrate is required is the product of the number of live entities and the number of shared resources, not the line count (Theorem 4.4, Corollary 4.5). The six functions (identity and lifecycle, allocation and reclamation, scheduling, naming and addressing, communication and synchronisation, and fault isolation and recovery) are necessary conditions for the operational semantics to be well defined (Theorem 4.9). They may be built or adopted from a host; the criterion is whether the invariant they carry is domain-critical (Corollary 4.11).

Problem 12.3 (Comprehension and design)**.** The conclusion is that comprehension is inverse compression and design is forward compression, the two reading directions of one structure. The optimal descent depth of comprehension is given by the stationarity condition of rate–distortion (Proposition 2.16); the measurable objective of design is the expected read-set (Definition 6.1), and minimising it requires the enforcement of each invariant at a single site (Theorem 6.3). Whether an abstraction leaks is measurable, not merely felt (Proposition 6.7, Corollary 6.8). For an AI the advantage lies not in reading more but in probing faster (Proposition 5.8).

Problem 12.4 (Correctness without review)**.** The conclusion is this: it is attainable that no human reviews the implementation line by line, and unattainable that no human remains responsible for the expression of the goal. "Correctness" must be decomposed into five dimensions (Definition 7.1); the intermediate process must be defined as a seven-stage verifiable

refinement chain (Definition 7.2); trust must be a composite of the two modes of trust generation (Definition 7.33); and artefact binding is the fourth pillar and cannot be omitted (Eq. (2.12)). The consolidated assurance condition (Theorem 7.34) gives the complete answer.

12.2 Compression as the through-line

Gathering the whole paper together yields a single thread: **compression**.

- Complex systems compress themselves by *levels*, so that every large system grows a kernel internal to its domain (Chapter 4);
- Bounded intelligence compresses information by *comprehension*, so that a finite cognition accommodates an unbounded world (Chapter 5);
- Engineering compresses the cost of change by *architecture*, so that a local modification does not require global comprehension (Chapter 6);
- Civilisation compresses trust by *rules*, so that a million lines of code of unknown provenance can be run safely (Chapter 7).

The mechanism of compression is the same in all four cases: quotienting by the kernel of the abstraction map (Proposition 2.8). *All the information a level exposes to the level above is the quotient of lower-level behaviour by that kernel.*

12.3 The core proposition

If the whole paper is to be compressed into one testable sentence, this paper chooses the following:

Principle 12.5 (Core proposition)**.** A complex system is comprehensible to a bounded intelligence if and only if it is compressible; and it is compressible if and only if it admits a separation of scales in which every level has an explicit, declared *oblivious set*.

Three consequences follow directly.

First, it turns "comprehensibility" from a subjective impression into a structural property: comprehensibility is equivalent to the existence of a quotient map under which task-relevant predictive power is preserved. This makes comprehensibility an *objective function* of design rather than a retrospective assessment.

Second, it redefines "architecture" as *manufactured separation of scales*. Nature obtains levels free from separation of scales and renormalisation; software levels do not emerge spontaneously and must be paid for in interfaces, encapsulation and invariants, and they are liable to leak. The core deliverable of architecture is therefore not a module list but the *declaration of the oblivious set of every level*.

Third, it gives a concrete rule of action for an AI: do not attempt to exhaust low-level information; maintain a layered, falsifiable compression model that counterexamples can update; obtain gradient by probing rather than by reading; and take task sufficiency as the stopping condition.

12.4 Closing remarks

The paper begins with three engineering intuitions and concludes with a joint proposition about structure and assurance. What is established is not the answers but *which questions have no answer*, and why:

- the gap between intent and specification has no mechanical solution, and its grounds are epistemological rather than logical; it is in substance the frame problem, and is therefore avoidable only in interaction;
- the basis of trust for mechanical assurance can be narrowed but not eliminated; and "verification is cheap" must be decomposed into two independent propositions, cheap checking time and cheap certificate size, of which the latter is the usual bottleneck;
- the reliability of consensus assurance is dominated by correlated error modes, so that *diversity* rather than *number* is the design variable, and diversity is ultimately a problem of incentive design, not of statistics;
- the structural convergence of large systems is probabilistic, domain-specific and shaped by organisational constraints, rather than unique;
- the duty of architecture is not to lower the cost of a single change but to lower the *exponent* with which that cost grows with size.

Set side by side, these items share one shape: *complexity compression is a necessity driven by size, and the form the compression takes is a contingency driven by domain; while the boundary of assurance falls exactly between what a machine can decide and what must be validated by the party that bears the consequences.*

The price is the surrender of a picture expressible as a single theorem; the gain is that every claim sits where it can be tested. For a framework about how to establish trustworthy conclusions on top of untrusted complexity, this is the only self-consistent ending: *it must pass its own test.*

Appendix A

Supplementary Proofs and Arguments

This appendix supplies complete proofs, or fuller arguments, for several results in the main text. Where a complete proof is already given there, it is not repeated.

A.1 Notes on Proposition 3.5

The proof given in the main text rests on the transversality theorem. Two points are added here.

Why "independent constraints" are necessary. If two constraints are not independent (for instance, when the second is a consequence of the first), their contributions to the codimension do not add, $\operatorname{codim} \mathcal{S} < c$, and the solution set has dimension greater than $d - c$. The conclusion of Proposition 3.5 must therefore be read under the premise that the constraints are independent and transversal. That premise usually holds in low-level design, where each constraint originates in a distinct pre-existing object (binary compatibility, the debugger, exception unwinding, the system-call interface), and often fails in high-level design, where several "constraints" are in fact different formulations of one preference.

Why "solution-set degeneracy" does not mean "unique solution". When $c \geq d$ the solution set is finite or empty, and a finite set may contain more than one element. Low-level design is nonetheless "nearly unique" in practice because, over and above the formal constraints, an implicit economic constraint is at work: maintaining two incompatible calling conventions at once carries labour and toolchain costs, and that cost induces a strict preference order among the admissible options.

A.2 Complete Argument for Theorem 4.4

The main text gives the principal argument. The choice of cost function and the reasonableness of the assumptions are supplied here.

The definition of cost. Let the unit interaction cost c be the whole of the work required to handle one interaction between a particular entity and a particular resource correctly: design, implementation, testing and long-term maintenance. In the no-substrate case every entity must do this work independently, and must handle the resource invariants independently as well.

The magnitude of the consistency obligation. For a resource r, n independent implementations must agree on the invariants of r. If consistency is not enforced, the deviation of any one implementation breaks the global invariant. If it is enforced, the enforcement itself requires that all n sites know the invariant, so the number of enforcement sites is $\Omega(n)$. Neither case improves on the conclusion "read-set $\geq k$" (Theorem 6.3). In other words, the absence of a substrate is not merely more costly: it forces invariants to be scattered, and thereby *permanently raises* the read-set lower bound of the system. This is the second benefit of a unified substrate, and it is independent of amortisation.

Relaxing assumption (b). Assumption (b), that every entity participates in all m resources, is the worst case. If entity e participates in only $|\mathcal{R}(e)|$ resources, the no-substrate cost is $\sum_e |\mathcal{R}(e)|$. As long as $\mathbb{E}_e[|\mathcal{R}(e)|]$ grows with m (that is, resource use is diffuse and broad), the product-form growth still holds. The qualitative conclusion of (4.2) therefore does not depend on the worst-case assumption.

A.3 Supplement to the Item-by-Item Argument for Theorem 4.9

The main text argues item by item, showing that the omission of any one function yields undefined behaviour. Two remarks on the strength of that argument are added here.

Strength of the (F3) scheduling argument. The argument relies on the claim that $\rightarrow$ must be a function, or that its non-determinism must be explicitly specified. If the semantics of a system is defined as the set of *all* interleavings, that is, if non-determinism is taken as part of the semantics and no selection among it is made, then the selection function may indeed be absent. In that case, however, the *implementation* of the system must still select one interleaving in every concrete execution, so the scheduling function remains present at the implementation level even though it does not appear in the abstract semantics. The formulation used in this paper, "either in the system's own code or in its host", covers this case.

Strength of the (F6) isolation argument. The argument relies on fault reachability. If the execution environment of a system guarantees the absence of faults (pure functional computation, for instance, or execution in a restricted environment verified to be fault-free), then (F6) may be absent or may degenerate to the trivial. This is precisely why some minimal systems can do without the complete six functions, and it shows that the premises of Theorem 4.9 are not redundant.

A.4 The Read-Set Lower Bound of Theorem 6.3

The main text proves $|\rho(c)| \geq |\mathrm{S}(\iota)|$. Why that inequality is tight is explained here.

If $\mathrm{S}(\iota) = \{M_0\}$ and the interface of M_0 is sufficient (Definition 6.6), then every change that could break ι must be expressed at the interface of M_0, so an agent performing the change need only understand M_0 and its call sites. Then $|\rho(c)| = 1 + O(\text{call sites})$, and the lower bound 1 is attained (when the change itself occurs inside M_0).

If $\mathrm{S}(\iota) = \{M_0, M_1\}$ and both modules bear partial responsibility for ι, the agent must understand both *and the consistency rule between them*, whose comprehension cost is strictly greater than that of the two modules together. The read-set therefore grows superlinearly in

$|\mathrm{S}(\iota)|$, and the linear relation is only a lower bound. The bound given in the main text is consequently conservative.

A.5 Exhaustiveness of the Three Grounds of Proposition 7.12

This argument has limits. The three grounds take the following form:

(g1) Preferences are generative: there exist an intent I and a situation s such that some component of I is fixed only after the bearer of the intent encounters s.

(g2) Specifications are lossy compressions: there exist an intent I and a specification Φ such that Φ cannot carry all information relevant to I.

(g3) Validation does not reduce to verification: there exist a requirement R and an implementation P such that "P satisfies Φ" is true while "P satisfies R" is false, and the procedure that decides the latter cannot be reduced to the procedure that decides the former.

The three grounds are not independent of one another: (g1) is a source of (g2), and (g3) is a consequence of (g2). A more accurate formulation may therefore be that *the root of the intent gap is the composite of (g1) and (g2).* This paper retains the three parallel formulations because they correspond to different engineering remedies: (g1) to interactive refinement and counterexample generation, (g2) to traceability and coverage analysis, and (g3) to an independent validation process and an allocation of responsibility.

Whether a fourth ground exists is an open problem (Chapter 11, items (O)).

A.6 Tightness of the Bound in Theorem 7.27

Equation (7.1) is the Hoeffding bound, which is not tight for large n. For $p < 1/2$ the probability of a majority error is in fact given by the central limit theorem, $\Pr[\text{majority errs}] \approx \Phi\big(-\sqrt{n}\,(1/2-p)/\sigma\big)$, whose exponent is $\Theta(n(1/2-p)^2)$ with a better constant.

This paper chooses the Hoeffding bound over a tighter estimate for three reasons:

(a) it requires no assumption about the distribution of individual errors, only independence, boundedness and a mean;

(b) its form is simpler, and it is therefore less easily misread as an exact value;

(c) the central conclusion of this paper (Proposition 7.28) does not depend on the tightness of the bound: the lower bound q given by a correlated error mode is independent of the constant in the upper bound.

A.7 A Stronger Form of Proposition 7.28

The main text gives $\Pr[\text{majority errs}] \geq q$. A finer form is available. Let the common error mode C and the individual independent errors I_i satisfy: verifier i errs if and only if $C = 1$ or $I_i = 1$. The event that the majority errs is then

$$\{C = 1\} \;\cup\; \Big\{C = 0,\; \sum_{i=1}^{n} I_i \geq \tfrac{n}{2}\Big\},$$

so that

$$\Pr[\text{majority errs}] \;=\; q + (1-q)\,\Pr\Big[\sum_{i=1}^{n} I_i \geq \tfrac{n}{2}\Big] \;\geq\; q.$$

For fixed $q > 0$, moreover, $\lim_{n\to\infty} \Pr[\text{majority errs}] = q$. This yields a statement stronger than a lower bound of q: *consensus reliability has an asymptotic floor determined by the*

correlated error mode; increasing the number of verifiers can only approach that floor and cannot break through it. This is the quantitative basis of Corollary 7.29.

A.8 Notes on Proposition 5.8

The argument in the main text uses the definition of information gain. One clarification is added here: Proposition 5.8 does not say that passive reading is without value. Passive reading supplies the *prior*, in the Bayesian sense, whereas active probing supplies the *likelihood*. A prober with no prior does not know what to probe; a reader with only a prior cannot update. The first four of the seven steps of the protocol in §5.4 are therefore reading (establishing the prior), and the last three are probing (acquiring the gradient). Their order is not arbitrary.

Appendix B

Templates

This appendix provides three templates for direct use. They convert the formal objects of the main text into engineering documents.

B.1 Architecture Map Template

The template records the level ladder of a system. Each row is one level.

Table B.1: Architecture map

Level	Participant	Guaranteed invariants	Assumed of the level below	May be ignored	Verification method
L_4					
L_3					
L_2					
L_1					
L_0					

The following requirements apply when filling in the table:

(1) The "may be ignored" column must not be left blank. A blank means the level is not a level (Principle 8.6).

(2) "Guaranteed invariants" must be written as propositions that can be checked, or at least decided.

(3) "Verification method" must state which class of assurance in Table 8.2 the invariant falls under (mechanical, monitoring, consensus).

B.2 Module Contract Template

Identity Module name, the level it belongs to, unique identifier.
Responsibility What problem it solves (one sentence, no longer than one line).
Boundary What it is responsible for, and what it is explicitly not responsible for.
External interface The functions, types or interfaces through which the level above uses it.
Precondition P What the caller must satisfy.
Postcondition Q What is guaranteed on return.
Invariant I Where it is enforced; why circumvention is impossible.
Side effects E^+ / E^- What is permitted to occur; what is explicitly forbidden.

Resource budget R Upper bounds on time, space, bandwidth and concurrency, and their cost model.

Failure semantics F What happens on error, cancellation, retry and recovery.

Environmental assumptions A The hardware, operating system, load and external-service conditions on which the conclusions depend.

Ownership and lifecycle How key objects are created, initialised, closed and destroyed; who owns them.

Concurrency model Thread-safety guarantees, lock ordering, reentrancy.

Dependencies Which modules it depends on; whether the dependency is on an abstract interface or on a concrete implementation.

Oblivious set What this module is permitted never to care about (required; see Definition 2.7).

Verification requirements Which tests or checks can demonstrate the above conclusions; whether there is an isolable unit of verification.

Leak-pair risk Known scenarios in which "the interface is the same but the external behaviour differs".

Open questions Design questions not yet settled.

B.3 Decision Record Template

Identifier and title `ADR-XXXX`: decision title.

Status Draft / accepted / superseded / deprecated.

Context What situation forces the decision to be made.

Decision drivers The constraints and goals that bear on the decision (performance, consistency, team size, rate of evolution, cost).

Candidate options Listed one by one, each with its assumptions and its cost.

Decision Which one is chosen, and the reason in one sentence.

Impact What the decision makes easier and what it makes harder.

Full argument for the excluded options Why the other options were rejected. This item is the most valuable part of a decision record: it records the information and judgement available at the time, so that future readers need not derive it again.

Verification How the consequences of the decision are checked; which external evidence would show that it is wrong.

Reassessment conditions What circumstances should prompt this decision to be revisited.

Principle B.1. If an architecture is a physical fact like a low-level ABI, there is no need to write a decision record. *The ubiquity of decision records is itself evidence that high-level architecture has no single correct answer.*

Appendix C

Glossary

Term	Definition and location
Level	The six-tuple $(X, \mathcal{B}, \mathcal{O}, \mathcal{A}, \mathcal{C}, \mathcal{I})$; a cognitive scale, not a physical directory. Definition 2.1.
Abstraction map	The map $\mathcal{O}_i$ projecting lower-level behaviour onto upper-level behaviour. Definition 2.2.
Concretisation map	The map γ_i expanding an upper-level behaviour into the set of its lower-level implementations. Definition 2.2.
Galois connection	$\mathcal{O}_i \dashv \gamma_i$, stating what abstraction discards and what it retains. Definition 2.2.
Inter-level soundness	$\mathcal{O}_i(\mathcal{B}_{i+1} \restriction A) \subseteq \mathcal{B}_i$. Definition 2.3.
Oblivious set	The kernel $\ker \mathcal{O}_i$ of the abstraction map, that is, the set of differences the level can in principle never distinguish. Definition 2.7.
Compression ratio	The information that cannot be recovered under the abstraction map. Definition 2.10.
Architecture relational graph	A typed multigraph whose edges fall into five classes: abstraction, dependency, composition, control and contract. Definition 2.11.
Abstract skeleton	The graph obtained by replacing each component with its duty role, retaining the type of each relation and erasing its instance. Definition 2.12.
Task	The pair $T = (Q, d)$, where Q is a query family and d a loss on answers. Definition 2.13.
T-sufficiency	The model answers every query as the real system does. Definition 2.14.
Description length	The length $\ell(M)$ of a model under a fixed encoding. Definition 2.15.
Behaviour contract	The eight-tuple $(P, Q, E^+, E^-, R, I, F, A)$. Definition 2.18.
Observable traces	The set $\mathrm{Tr}(B, E)$ of observable execution traces an artefact may produce in an environment. Definition 2.19.
Assurance	The four-tuple (property, object, premises, evidence); the condition under which it holds is (2.11). Definition 2.20.
Trusted computing base (TCB)	The set of components that must be trusted unconditionally for the assurance to hold. Definition 2.21.

Term	Definition and location
Mechanical assurance	Assurance given by a single deterministic, exhaustively inspectable decision procedure. Definition 2.22.
Consensus assurance	Assurance given by the adversarial scrutiny of multiple independent participants. Definition 2.22.
Internal infrastructuralisation	The three-stage process: identification of commonality → unification of mechanisms → upper-layer composition. Definition 4.1.
Participant-resource structure	The five-tuple $(E, \mathcal{R}, \to, \text{Ready}, \text{Fault})$. Definition 4.2.
Unified substrate	The set of mechanisms through whose declared interfaces every interaction of an entity with a resource passes. Definition 4.3.
The six functions	Identity and lifecycle; allocation and reclamation; scheduling; naming and addressing; communication and synchronisation; fault isolation and recovery. Definition 4.8.
Read-set of a change	The set $\rho(c)$ of modules that must be read to complete a change correctly. Definition 6.1.
Enforcement sites of an invariant	The set $\mathrm{S}(\iota)$ of modules that must be implemented correctly for the invariant to hold. Definition 6.2.
Near-decomposability	A module's interface observations are a sufficient statistic for its external behaviour. Definition 6.6.
Leak pair	Two executions with equal interface trace and different external behaviour. Proposition 6.7.
Technical debt	The difference between the description length of the code and its shortest generating description. Definition 6.10.
Verifiable refinement chain	The seven-stage chain from purpose to certified deployment, (G1) . . . (G7). Definition 7.2.
The five dimensions of correctness	Functional, side-effect, resource, system and goal-expression correctness. Definition 7.1.
Four classes of side effect	Functional, resource, security and semantic drift. Definition 7.20.
Artefact evidence package	The ten items of evidence every artefact should carry. Definition 7.19.
Cognitive scale	L_0 principles of computation, L_1 infrastructure and runtime, L_2 components and subsystems, L_3 domain function and business semantics, L_4 and above: scenarios and system-level behaviour, L_∞ purpose and responsibility. Definition 8.1.
Level contract four-tuple	(participant, invariant assured, contract form, mode of verification), plus the oblivious set. Definition 8.5.
Level attribution of a verification obligation	A property φ belongs to the lowest level that can express it. Definition 8.8.
Domain binding	The fraction Dom of the level's axioms that are domain-specific. Definition 3.9.
Two-layer trust structure	A composite in which the intent layer is assured by consensus and the formalisable layer by mechanical assurance. Definition 7.33.

Symbol Index

Symbol	Meaning
Λ_i	level i
$\mathcal{B}_i$	behaviour set of level i
$\mathcal{O}_i, \gamma_i$	abstraction and concretisation maps
$\mathcal{A}_i, \mathcal{C}_i, \mathcal{I}_i$	environmental assumptions, cost model, invariant set
$\mathrm{Obl}(\Lambda)$	oblivious set
κ	compression ratio
$\mathcal{G}, \phi$	architecture relational graph, abstract skeleton
T, Q, d	task, query family, loss functional
ℓ, L_T	description length, task loss
S	behaviour contract
$\mathrm{Tr}(B, E), \mathrm{Allowed}(S)$	set of observable traces, set of permitted traces
$\mathfrak{A}$, TCB	assurance four-tuple, trusted computing base
Π	participant-resource structure
$\mathcal{K}_6$	the six functions
n, m, k, F	number of entities, number of resources, interfaces per entity, fixed cost of the substrate
$\rho(c), \mathrm{S}(\iota)$	read-set, set of enforcement sites
D	technical debt
$\mathrm{Dom}, \mathrm{Conv}$	domain binding, convergence strength
$q, p, \varepsilon_{\mathrm{cons}}$	correlated error probability, individual error rate, consensus error bound

Bibliography

[1] J. Mucha, A. Kaufmann, and D. Riehle. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering*, 29:119–141, 2024. https://doi.org/10.1007/s00766-023-00412-z

[2] K. Mokos and P. Katsaros. A survey on the formalisation of system requirements and their validation. *Array*, 7:100030, 2020. https://doi.org/10.1016/j.array.2020.100030

[3] S. Li, J. Jiang, T. Zhao, and J. Shen. OSVBench: Benchmarking LLMs on specification generation tasks for operating system verification. In *Proceedings of AAAI 2026*, 40(37):31698–31707, 2026. https://doi.org/10.1609/aaai.v40i37.40437

[4] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu. How powerful are LLMs in generating formal program specifications? In *Proceedings of ICML 2026*, PMLR 306:145221–145249, 2026. https://proceedings.mlr.press/v306/yang26am.html

[5] Z. Ye, A. Z. H. Yang, H. Su, Z. Liao, S. Tenka, Z. Qin, U. Ghai, D. Song, and S. Kong. Intent-aligned formal specification synthesis via traceable refinement (VeriSpecGen). arXiv:2604.10392, 2026. https://arxiv.org/abs/2604.10392

[6] L. Ma, S. Liu, Y. Li, Q. Wu, H. Wang, and L. Bu. SpecSyn: LLM-based synthesis and refinement of formal specifications for real-world program verification. arXiv:2604.21570, 2026. https://arxiv.org/abs/2604.21570

[7] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of ICSE 2025*, 2025. https://doi.org/10.1109/ICSE55347.2025.00129

[8] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song. VERINA: Benchmarking verifiable code generation. In *Proceedings of ICLR 2026*, 2026.

[9] O. Kupferman and M. Y. Vardi. Vacuity detection in temporal model checking. In *CHARME 2003*, 2003. https://doi.org/10.1007/3-540-48153-2_8

[10] A. Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis / Technical Report EECS-2008-177, UC Berkeley, 2008. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[11] R. Alur et al. Syntax-guided synthesis. In *Proceedings of CAV 2013*, 2013. https://www.sygus.org/

[12] T. Kurabayashi, H. Tanno, H. Kirinuki, and T. Oshima. A survey of program synthesis. *Computer Software*, 40(2):95–128, 2023. https://doi.org/10.11309/jssst.40.2_95

[13] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS 2023 Datasets and Benchmarks*, 2023.

[14] P. Song, K. Yang, and A. Anandkumar. Towards large language models as copilots for theorem proving in Lean. arXiv:2404.12534, 2024. https://arxiv.org/abs/2404.12534

[15] G. C. Necula. Proof-carrying code. In *Proceedings of POPL 1997*, pages 106–119, 1997. https://doi.org/10.1145/263699.263712

[16] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. https://doi.org/10.1145/363235.363259

[17] E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. https://doi.org/10.1016/0890-5401(91)90052-4

[18] G. D. Plotkin and J. Power. Notions of computation determine monads. In *FOSSACS 2002*, LNCS 2303:342–356, 2002. https://doi.org/10.1007/3-540-45931-6_24

[19] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. https://homotopytypetheory.org/book/

[20] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of POPL 1977*, pages 238–252, 1977. https://doi.org/10.1145/512950.512973

[21] A. W. Appel and S. Blazy. Separation logic for small-step Cminor. In *TPHOLs 2007*, LNCS 4732:5–21, 2007. https://doi.org/10.1007/978-3-540-74591-4_3

[22] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. https://doi.org/10.1145/1538788.1538814

[23] G. C. Necula. Translation validation for an optimizing compiler. In *Proceedings of PLDI 2000*, 2000. https://doi.org/10.1145/349299.349314

[24] Y. K. Tan, M. O. Myreen, R. Kumar, A. Fox, S. Owens, and M. Norrish. The verified CakeML compiler backend. *Journal of Functional Programming*, 29, 2019. https://doi.org/10.1017/S0956796818000229

[25] G. Klein et al. seL4: Formal verification of an OS kernel. In *Proceedings of SOSP 2009*, pages 207–220, 2009. https://sel4.systems/Research/publications.html

[26] J. O. G. Krijnen, M. M. T. Chakravarty, G. Keller, and W. Swierstra. Translation certification for smart contracts. *Science of Computer Programming*, 233:103051, 2024. https://doi.org/10.1016/j.scico.2023.103051

[27] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936. https://doi.org/10.1112/plms/s2-42.1.230

[28] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the AMS*, 74(2):358–366, 1953.

[29] J. Hoffmann and S. Jost. Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science*, 32(6), 2022. https://doi.org/10.1017/S0960129521000487

[30] R. Wilhelm et al. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Transactions on Embedded Computing Systems*, 7(3), 2008. https://doi.org/10.1145/1347375.1347389

[31] M. Lv, N. Guan, J. Reineke, R. Wilhelm, and W. Yi. A survey on static cache analysis for real-time systems. *LITES*, 3(1), 2016. https://doi.org/10.4230/LITES-v003-i001-a005

[32] R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5:183–191, 1961. https://doi.org/10.1147/rd.53.0183

[33] I. Georgescu. 60 years of Landauer's principle. *Nature Reviews Physics*, 3:770, 2021. https://doi.org/10.1038/s42254-021-00400-8

[34] R. Taleb, S. Hallé, and R. Khoury. Uncertainty in runtime verification: A survey. *Computer Science Review*, 50:100594, 2023. https://doi.org/10.1016/j.cosrev.2023.100594

[35] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: Methodology and case studies. In *Proceedings of CAV 1998*, 1998.

[36] M. Sharf, B. Besselink, K. H. Johansson, et al. Assume/guarantee contracts for dynamical systems: Theory and computational tools. *IFAC-PapersOnLine*, 54(5):25–30, 2021. https://doi.org/10.1016/j.ifacol.2021.08.469

[37] M. Sharf, B. Besselink, and K. H. Johansson. Contract composition for dynamical control systems: Definition and verification using linear programming. *Automatica*, 164:111637, 2024. https://doi.org/10.1016/j.automatica.2024.111637

[38] B. Shali, A. J. van der Schaft, and B. Besselink. Design and control for implementation of simulation-based assume-guarantee contracts. *IEEE Transactions on Automatic Control*, 71(1):65–80, 2026. https://doi.org/10.1109/TAC.2025.3587874

[39] H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[40] P. D. Grünwald. The minimum description length principle in coding and modeling. *IEEE Transactions on Information Theory*, 52(2), 2006. https://doi.org/10.1109/TIT.2005.862128

[41] P. Grünwald and T. Roos. Minimum description length revisited. arXiv:1908.08484, 2019. https://arxiv.org/abs/1908.08484

[42] J. A. Goguen and J. Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy 1982*, pages 11–20, 1982. https://doi.org/10.1109/SP.1982.10014

[43] M. C. Feser, S. Chaudhuri, and I. Dillig. Synthesizing data structure traversals from local specifications. In *Proceedings of PLDI 2015*, 2015. https://doi.org/10.1145/2737924.2738007

[44] P. M. Rondon, M. Kawaguchi, and R. Jhala. Liquid types. In *Proceedings of PLDI 2008*, 2008. https://doi.org/10.1145/1375581.1375602

[45] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS 2008*, LNCS 4963, 2008. https://doi.org/10.1007/978-3-540-78800-3_24

[46] M. Leucker and C. Schallhart. Runtime verification for LTL and TLTL. In *TACAS 2009*, 2009. https://doi.org/10.1007/978-3-642-00768-2_17

[47] W. A. Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, 1980.

[48] On the generation of requirements specifications from software engineering models: A systematic literature review. *Information and Software Technology*, 51(9):1291–1307, 2009. https://doi.org/10.1016/j.infsof.2009.04.001

[49] D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[50] M. E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968.

[51] F. P. Brooks. *The Mythical Man-Month*. Addison-Wesley, 1975.

[52] B. W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, 1984.

[53] J. Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, 2018.

[54] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[55] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948.

[56] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[57] A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

[58] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[59] M. de Condorcet. *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. Paris, 1785.

[60] L. C. Paulson. The foundation of a generic theorem prover. *Journal of Automated Reasoning*, 5(3):363–397, 1989.

[61] M. T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Second edition, Pragmatic Bookshelf, 2018.

[62] E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[63] D. C. Dennett. Real patterns. *The Journal of Philosophy*, 88(1):27–51, 1991.

[64] W. V. O. Quine. Two dogmas of empiricism. *The Philosophical Review*, 60(1):20–43, 1951.

[65] I. Lakatos. Falsification and the methodology of scientific research programmes. In *Criticism and the Growth of Knowledge*, Cambridge University Press, 1970.

[66] J. Pearl. *Causality: Models, Reasoning, and Inference*. Second edition, Cambridge University Press, 2009.

[67] T. Leinster. *Higher Operads, Higher Categories*. London Mathematical Society Lecture Note Series 298, Cambridge University Press, 2004.

[68] B. Jacobs. *Categorical Logic and Type Theory*. North-Holland, 1999.

[69] P. Alvaro, N. Conway, J. M. Hellerstein, and W. R. Marczak. Consistency analysis in Bloom: A CALM and collected approach. In *Proceedings of CIDR 2011*, 2011.

[70] G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[71] C. E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7(4):142–163, 1959.

[72] R. E. Kalman. On the general theory of control systems. In *Proceedings of the First International Congress on Automatic Control*, 1960.